

01 C++工作原理

2024年11月14日 0:01

预处理：

```
#include <iostream>
#define INTEGER int
#if ...
.....
#endif
```

“#” 后的**预处理语句**在编译前就被处理了。

`#include`本质上只是复制粘贴。它让编译器遍历我们需要引入的头文件，然后将其复制到cpp文件里，就好像我们在代码中写过那些内容一样。（甚至可以只在.h中只写一个“}”，在函数结尾处引入它，代替手敲“}”）

`#include <iostream>` 实际上会使我们的代码多出整整50610行。

`#if & #endif` 包裹的代码当满足条件时才会被编译，否则编译后的代码不含这部分，就像被删除了一样。

Main函数：

```
int main(){...}
```

同C语言，main函数是程序的入口，但C++中main函数并不必要写明return 0，默认返回0。

打印 “hello world”：

```
cout << "hello world!" << endl;
cin.get();
```

“<<”实际上是一个函数，并非运算符。它**等效于**“.print()”。

endl换行。

cin.get()函数会暂停程序，直到按下回车键。

设置：

Debug & Release:

debug模式运行更慢，但方便调试，release模式运行速度经过优化，且会生成文件（默认为.exe类型）。

（比如当一个有返回值的函数被调用，却没有相应的变量接收它的返回值，这时，无速度优化编译依然会编译该语句，即使它不发挥任何作用；而速度优化编译则会选择跳过该语句，什么都不做）

x86 & x64:

x86指windows32, x64指windows64

注:

- 编译器只编译.cpp文件, 不编译.h文件。头文件在预处理时同.cpp文件被部分编译。
- .cpp文件被编译成.obj文件, 再通过link连接合成一个.exe文件。

函数声明与定义:

声明: 告诉编译器有该函数 (包括名称和变量类型) ;

```
void Log(const char* message);
```

定义: 写明函数的名称和函数体;

```
void Log(const char* message) {  
    cout << message << endl;  
}
```

当编译发生时, 链接器会试图在项目内所有cpp文件中寻找所声明函数的定义, 如果有声明却无定义, 就会出现编译错误。

C++中文件不重要:

不同于Java中, 公开类名必须与文件名相同等等要求, C++中的文件仅仅是为编译器提供源代码, 格式、命名并不重要 (默认识别.cpp/.c/.h三种文件)。

编译 & 链接:

编译:

1. 编译生成的obj文件内部是纯二进制代码, 无可读性, 但也可以修改设置, 使其输出汇编代码。
2. 在汇编代码中可以发现, 默认release模式输出的汇编代码比默认debug模式输出的汇编代码简洁了很多。可以在设置中使debug模式使用速度优化, 也可以使release模式禁用速度优化。
3. 代码中出现的常数计算式 (如 $5*2$) 都会在编译时自动计算出结果, 并将直接将结果发送给寄存器。这一过程称为“常数折叠”。

链接:

1. 链接只在编译后进行, 一般在运行/调试/生成时进行, 如果只按ctrl F7进行编译, 链接不会被触发。

2. 当需要输出exe程序时，链接会为每个单独的obj文件建立联系，形成一个完整的应用程序。但如果只有一个obj，同样也会触发链接。此时的主要任务是找到程序的入口，如果没有定义入口，就会出现LNK错误。（程序的入口并不一定是main函数，可以自定义名称）
3. 如果A函数内调用了外部定义的函数B，即使在当前cpp文件内A函数未被调用，当函数B的定义缺失时，链接器仍会报错。除非用static修饰A函数，确保它只会在当前cpp文件内被调用。
4. 当同一个cpp文件内定义完全相同的函数时，编译器会报错。而当不同的cpp文件内定义完全相同的函数时，链接器会报错（这个错误通常发生在头文件内定义函数的时候。由于#include复制粘贴的本质，会无意间在不同的cpp文件内重复定义函数，因此一般只在头文件内声明函数，在源文件内定义函数）。

错误代码中，“C”开头代表编译错误（compile），“LNK”开头代表链接错误（link）。

02 变量和函数

2024年11月14日 9:23

数据类型：

在C++中，所有原始数据类型都有其规定的用途，但实际上，我们并不需要完全遵守所谓的“规定”，因为这些数据类型的本质区别只在于占用的内存大小。

- char: 1byte
- short: 2byte
- int: 4byte
- long: 4byte
- long long: 8byte
- float: 4byte
- double: 8byte
- bool: 1byte

正如前面所说，数据类型的赋值并没有完全的严格规定，比如，你可以为一个char变量赋值65 (A的ASCII码值)，或者为一个int变量赋值“A”。

事实上，bool值为真，0为假，只需要1bit的空间即可，但是由于计算机可寻址的最小内存单位是字节，所以bool值的内存大小仍认为是1byte。

指针和引用：

以bool为例：

- 指针: bool*
- 引用: bool&

前期不做详细介绍。

函数/方法：

- 函数在类中出现，就称为方法。
- 每执行一个函数，编译器都会生成一个call指令，创建一个堆栈结构来存储返回地址，同时，程序会在内存中反复跳跃，因此过多的函数会导致程序的运行速度降低。

头文件 (.h)：

头文件通常作为一个存储各种函数的声明的公共空间。本质上起到简化“复制粘贴”的作用。通过在cpp文件中引用头文件，可以避免复制粘贴大量冗长的函数声明。

#pragma once

新建头文件时，编译器会自动生成预处理语句“#pragma once”。

它的作用是监督该头文件，组织单个头文件被多次include，并转换为单个翻译单元。

由于include的本质是复制粘贴，所以当头文件内定义结构体时，重复include会导致结构体的重复定义，编译出错。由于头文件内也可以include其他头文件，这种嵌套会导致该错误会更多。

易发生。

```
#include <iostream>
#include "iostream"
```

引用头文件可以使用尖括号<>或双引号""，区别在于：

- 尖括号通常用于包含标准库头文件或者第三方库头文件。当使用尖括号时，编译器会在标准库目录和通过编译器选项（如gcc的-I参数）指定的目录中查找头文件。
- 双引号则用于包含用户自定义的头文件。当使用双引号时，编译器首先会在当前源文件所在的目录中查找头文件，如果没有找到，再按照尖括号的方式进行查找。

```
#include <iostream>
#include <stdlib.h>
```

C++标准库为了区别于C标准库，其头文件无后缀名。而C标准库的头文件一般以.h为后缀。

03 条件&分支语句 (if & else)

2024年11月16日 19:44

布尔值:

通常来说, 0为假, 1为真;

事实上, 可以认为0为假, 非0为真。

前面已经提到, 布尔值虽然理论上使用1bit的内存就足够, 但是仍会划分1byte的内存。此时计算机要做的事情就是检查这1byte内存中的8个bit是否含0以外的数据。

如果全为0, 则显然整个byte存储的值为0, 判定为假; 反之, 整个byte存储的值不为0, 判定为真, 具体的值并不重要。

(因此nullptr显然在内存上与0相同, 他们本质相同)

```
char* ptr = nullptr;
00007FF76E4223F2  mov     qword ptr [ptr], 0
int zero = 0;
00007FF76E4223FA  mov     dword ptr [zero], 0
```

if语句的本质:

在debug模式下, 进入反汇编, 我们可以得到源代码和汇编码的详细对比。

```
int x = 6;
00007FF6C29923EB  mov     dword ptr [x], 6 |
bool comparisonResult = x == 5;
00007FF6C29923F2  cmp     dword ptr [x], 5
00007FF6C29923F6  jne     main+34h (07FF6C2992404h)
00007FF6C29923F8  mov     dword ptr [rbp+0F4h], 1
00007FF6C2992402  jmp     main+3Eh (07FF6C299240Eh)
00007FF6C2992404  mov     dword ptr [rbp+0F4h], 0
00007FF6C299240E  movzx   eax, byte ptr [rbp+0F4h]
00007FF6C2992415  mov     byte ptr [comparisonResult], al
if (comparisonResult) {
00007FF6C2992418  movzx   eax, byte ptr [comparisonResult]
00007FF6C299241C  test    eax, eax
00007FF6C299241E  je      main+5Ch (07FF6C299242Ch)
    Log("hello world!");
00007FF6C2992420  lea     rcx, [string "hello world!" (07FF6C299BCB0h)]
00007FF6C2992427  call    Log (07FF6C2991361h)
}
cin.get();
00007FF6C299242C  mov     rcx, qword ptr [__imp_std::cin (07FF6C29A3190h)]
00007FF6C2992433  call    qword ptr [__imp_std::basic_istream<char, std::char_traits<char> >::get (07FF6C29A3150h)]
```

当我们新建一个bool值时, 机器会判定条件是否满足, 若不满足, **jne指令 (jump not equal)** 会被触发, cpu会跳到相应的指令, 获得一个0值; 若满足, 则不触发jne, 正常向下执行, 获得一个1值, 之后**主动跳到 (jmp)** 该区域倒数第二句指令, 即将寄存器内的值设为刚刚得到的值。

遇到if语句时, cpu会将括号内的条件语句对应的bool值存入寄存器, 再执行**test指令**, 该指令的实质就是将刚刚放入寄存器的值**与0做比较**, 所以后面紧跟着的是**je指令 (jump equal)**, 即若条件语句等于0, 就跳过if语句。

若if语句与else语句搭配使用，情况会稍有不同。

```
    if (comparisonResult) {
00007FF7856C2418  movzx     eax,byte ptr [comparisonResult]
00007FF7856C241C  test      eax,eax
00007FF7856C241E  je        main+5Eh (07FF7856C242Eh)
        Log("hello world!");
00007FF7856C2420  lea       rcx,[string "hello world!" (07FF7856CBC0h)]
00007FF7856C2427  call      Log (07FF7856C1361h)
    }
00007FF7856C242C  jmp      main+6Ah (07FF7856C243Ah)
    else {
        Log("no hello world!");
00007FF7856C242E  lea       rcx,[string "no hello world!" (07FF7856CBC10h)]
00007FF7856C2435  call      Log (07FF7856C1361h)
    }
```

test指令后的je指令会直接跳入else语句（不过else语句只会紧跟着if语句出现，所以本质上没有改变，只是可以换种理解的思路），再者，if语句后会多出一句jmp指令，用于在执行完if语句内的指令后跳过接下来的else语句。

补充关于debug和release模式的不同：

以下是同一段代码分别在debug模式和release模式下的反汇编代码。

```
    int x = 6;
00007FF6C29923EB  mov       dword ptr [x],6 |
    bool comparisonResult = x == 5;
00007FF6C29923F2  cmp       dword ptr [x],5
00007FF6C29923F6  jne       main+34h (07FF6C2992404h)
00007FF6C29923F8  mov       dword ptr [rbp+0F4h],1
00007FF6C2992402  jmp       main+3Eh (07FF6C299240Eh)
00007FF6C2992404  mov       dword ptr [rbp+0F4h],0
00007FF6C299240E  movzx     eax,byte ptr [rbp+0F4h]
00007FF6C2992415  mov       byte ptr [comparisonResult],al
    if (comparisonResult) {
00007FF6C2992418  movzx     eax,byte ptr [comparisonResult]
00007FF6C299241C  test      eax,eax
00007FF6C299241E  je        main+5Ch (07FF6C299242Ch)
        Log("hello world!");
00007FF6C2992420  lea       rcx,[string "hello world!" (07FF6C299BC0h)]
00007FF6C2992427  call      Log (07FF6C2991361h)
    }
    cin.get();
00007FF6C299242C  mov       rcx,qword ptr [__imp_std::cin (07FF6C29A3190h)]
00007FF6C2992433  call      qword ptr [__imp_std::basic_istream<char, std::char_traits<char> >::get (07FF6C29A3150h)]
```

debug模式下的反汇编代码

```
    int x = 6;
    bool comparisonResult = x == 5;
    if (comparisonResult) {
        Log("hello world!");
    }
    cin.get();
00007FF771551004  mov       rcx,qword ptr [__imp_std::cin (07FF771552080h)]
00007FF77155100B  call      qword ptr [__imp_std::basic_istream<char, std::char_traits<char> >::get (07FF771552088h)]
```

release模式下的反汇编代码

可以看到，release模式下，编译器会在编译时就将许多常数化的变量预先处理，也就是之前提到的常数折叠。（比如编译器明知x一定等于6，则x一定不等于5，那么comparisonResult一定为false，则if语句没有意义。所以这些内容根本不会被编译成汇编码）这就是为什么release模式下的执行速度会更快。

04 循环与控制流语句 (for/while & continue/break/return)

2024年11月20日 21:38

循环语句：

for循环：

```
for (int i = 0; i < 5; i++) {  
    Log("hello, world!");  
}
```

```
int i = 0;  
bool condition = true;  
for (; condition; ) {  
    Log("hello, world!");  
    i++;  
    if (i >= 5) {  
        condition = false;  
    }  
}
```

以上两段代码功能完全一样，唯一的区别是变量*i*的作用域有所不同；

while循环：

理解for循环的原理后，可以尝试将for循环与while循环进行转化：

```
int i = 0;  
while (i < 5) {  
    Log("hello, world!");  
    i++;  
}
```

do while循环：

```
int i = 0;  
do {  
    Log("hello, world!");  
    i++;  
} while (i < 5);
```

do while与while循环的区别在于执行语句和判断语句的顺序不同。

do while语句中do后的代码块内的语句至少会被执行一次。

控制流语句：

continue, break, return。

05 指针和引用

2024年11月20日 22:38

指针只是一个保存内存地址的整数。本质上，指针的类型没有实际意义。

指针：

```
int var = 8;
int* ptr = &var;
*ptr = 10;
```

指针的声明与变量类似，不同的是需要在类型前加上 “*” 。

且指针可以为void类型，而变量不可以。

在变量前使用 “&” 来取地址；在指针前使用 “*” 来反向引用该指针指向的地址所存储的变量。

前面已经强调指针的类型本质上没有意义，但是它仍有作用。它明确了该指针指向的内存地址的大小。

如：int存储的内存地址一定为4字节，char存储的内存地址一定为1字节。

所以，可以用下面的方式向内存申请任意大小的空间：

```
char* buffer = new char[8];
memset(buffer, 0, 8);
```

但指针本质上也是一个变量，它同样存储在内存中，所以会存在“指向指针的指针”，例如双指针和三指针。

```
char** ptr = &buffer;
```

引用本质上仅仅获取现有变量的地址，它不占用任何内存空间。

引用：

```
int a = 5;
int& ref = a;
```

以上代码中，ref仅仅引用了a，它不占用任何空间，而且编译器在编译时也只会生成a一个变量。可以理解为ref是a的另一个名字。

在调用函数时，引用比较常用。比如想要通过函数修改当前作用域内的变量大小：

```

void Increment(int& value) {
    value++;
}

int main() {
    int a = 5;
    Increment(a);
    cout << a;
    cin.get();
}

```

此时打印出a的值为**6**;

以上代码可通过取地址和指针的反向引用来实现，效果完全一样：

```

void Increment(int* value) {
    (*value)++;
}

int main() {
    int a = 5;
    Increment(&a);
    cout << a;
    cin.get();
}

```

所以可以看出，**引用就是指针的语法糖**。

实际上，**指针**的功能远比**引用**更加强大，但在某些场合，**引用**更加简洁易用。

补充：

引用在声明时必须初始化，且一旦初始化之后无法更改引用的对象。

```

int a = 5;
int b = 8;
int& ref = a;
ref = b;

```

比如上述代码的结果就是，a将被赋值为b，ref依然引用a。

那么如何更改引用？答案是：不行，只能使用指针。

```

int* ref = &a;
ref = &b;

```

06 类

2024年12月5日 20:13

类本质上可以理解为语法糖。类的存在并没有给一种语言添加任何新的额外功能，也就是说：**用类能解决的问题，没有类也可以解决；不用类无法解决的问题，有类也没法解决。类仅仅提供便利。**

类与结构体：

类与结构体本质上的区别只有默认访问权限不同：类的成员默认为私有，而结构体的成员默认为公开。仅此而已。

其实C++保留结构体只是为了与C语言保持向后兼容，结构体完全可以用public修饰的类来替代。

不过一般来说，为了保持类继承的层次结构完整，**涉及到继承的、功能较为复杂的部分都会使用类。**结构体更适合处理**只包含一些变量和对于这些变量的简单操作的集合。**

类的基础写法：

```
#include <iostream>
using namespace std;

class Log {
public:
    const int LogLevelError = 0;
    const int LogLevelWarning = 1;
    const int LogLevelInfo = 2;

private:
    int m_LogLevel = LogLevelInfo;

public:
    void SetLevel(int level) {
        m_LogLevel = level;
    }

    void Error(const char* message) {
        if (m_LogLevel >= LogLevelError) {
            cout << "[Error]:" << message << endl;
        }
    }

    void Warn(const char* message) {
        if (m_LogLevel >= LogLevelWarning) {
            cout << "[Warning]:" << message << endl;
        }
    }
}
```

```
    }  
  
    void Info(const char* message) {  
        if (m_LogLevel >= LogLevelInfo) {  
            cout << "[Info]:" << message << endl;  
        }  
    }  
};
```

与以往接触的面向对象语言 (JAVA,C#) 相比, C++ 的类写法略有不同:

- public、private 等权限修饰符不能直接写在成员声明前, 要另起一行顶格写, 统筹之后的所有声明, 直到遇见另一个权限修饰符;
- 类结束的右大括号 "}" 后需要加 ";" 。

07 静态 (static)

2024年12月8日 21:54

static修饰符出现的位置不同，其作用也不同，通常包括以下情况：

- 出现在类或结构体之外；
- 与类和结构体一起使用；
- 局部静态变量 (local static) 。

出现在类或结构体之外：

比如当前有两个文件，分别是Main.cpp和Static.cpp，其内容分别是：

```
//Static.cpp
static int s_var = 5;
```

```
//Main.cpp
int s_var = 10;
```

按照之前所学，当运行Main程序时，由于链接器的存在，两个源文件中声明的重复的变量，会出现链接错误。但此时并没有出现该错误。是因为static修饰符将相应的变量限定为只对定义它的翻译单元（就是.cpp）可见，所以在别的源文件内声明同名的变量不会引起链接错误。

可以尝试将static删去，再运行程序便会出现链接错误：

```
l>Static.obj : error LNK2005: "int s_var" (?s_var@@3HA) 已经在 Main.obj 中定义
l>D:\代码\C++代码\Test_1\x64\Debug\Test_1.exe : fatal error LNK1169: 找到一个或多个多重定义的符号
```

相反的，我们可以尝试使用extern关键字来外部引用变量：

```
//Static.cpp
static int s_var = 5;
```

```
//Main.cpp
extern int s_var;
int main() {
    cout << s_var;
    cin.get();
}
```

此时会出现另一个错误，原因是一样的，static修饰符将相应的变量限定为只对Static.cpp可见，在其他的源文件内试图引用均会失败：

```
l>Main.obj : error LNK2001: 无法解析的外部符号 "int s_var" (?s_var@@3HA)
l>D:\代码\C++代码\Test_1\x64\Debug\Test_1.exe : fatal error LNK1120: 1 个无法解析的外部命令
```

有趣的是，如果不在main函数内尝试打印s_var（或者以什么别的方式尝试使用该变量），并

不会出现任何错误。或者，如果改写成 “`extern int s_var = 10;`” 同样不会出现错误，也就是说初始化覆盖了引用，使该变量变得可用了。

可以总结为static出现在类或结构体之外时，可以理解为相对于.cpp文件的“private”。一般情况下，如果你不想要让一个变量成为项目的全局变量，尽可能多地使用static修饰符。

与类和结构体一起使用：

```
#include <iostream>
using namespace std;

struct Entity {
    static int x, y;
};

int Entity::x;
int Entity::y;

int main() {
    Entity e1, e2;
    e1.x = 1;
    e1.y = 2;
    e2.x = 2;
    e2.y = 3;
    cout << e1.x << " " << e1.y << endl;
    cout << e2.x << " " << e1.y << endl;
}
```

上面的代码运行的结果实际上会是2行一样的 “2 3”，因为e1.x和e2.x引用的都是同一个变量。这就是结构体里的静态变量。

实际上，e1.x这种写法是不规范的，想要引用Entity的静态变量x，应该写 “Entity::x”，这样从代码来看就不会产生歧义了，e1.x和e2.x本质上都是Entity::x。

值得注意的是，当删除这两段代码后，编译器会出现链接错误。

```
int Entity::x;
int Entity::y;
```

```
1>Main.obj : error LNK2001: 无法解析的外部符号 "public: static int Entity::x" (?x@Entity@@2HA)
1>Main.obj : error LNK2001: 无法解析的外部符号 "public: static int Entity::y" (?y@Entity@@2HA)
```

这是由于我们只在类中声明了静态变量，但是却没有对其定义，链接器尝试为它们分配内存地址时就会出现错误。

正常情况下，类或者结构体内声明的静态变量都需要在类的外部进行定义。只有当静态变量是预先已知的常量时，才可以使用以下方法，同时进行声明和初始化。

```
static const int x = 10;
static const int y = 20;
```

静态方法（函数）与静态变量有所区别，它不需要在外部再次定义。但重要的一点是**静态方法无法访问非静态变量**。

```
#include <iostream>
```

```
using namespace std;

struct Entity {
    static int x, y;
    static void Print() {
        cout << x << " " << y << endl;
    }
};

int Entity::x;
int Entity::y;

int main() {
    Entity::Print();
    cin.get();
}
```

如果此时将静态变量x, y前的static修饰删去（当然，定义的部分也要删去），则会出现错误：

```
1>D:\代码\C++代码\Test_1\Main.cpp(7,11): error C2597: 对非静态成员“Entity::x”的非法引用
1>D:\代码\C++代码\Test_1\Main.cpp(5,6): message : 参见“Entity::x”的声明
```

这是因为非静态变量属于实例，静态方法脱离实例存在，直接调用静态方法会让它不知道该关联哪个实例。当代码修改成如下：

```
#include <iostream>
using namespace std;

struct Entity {
    int x, y;
    static void Print(Entity e) {
        cout << e.x << " " << e.y << endl;
    }
};

int main() {
    Entity e;
    e.x = 1;
    e.y = 2;
    Entity::Print(e);
    cin.get();
}
```

程序可以正常运行。因此并不是静态方法无法访问非静态变量这么简单粗暴，本质上是参数传递的问题。

实际上，这种写法就是非静态方法的本质，只不过非静态方法将作为参数传递的实例隐藏起来了。

局部静态变量（Local Static）：

变量有两个重要属性：生命周期和作用域。

生命周期指的是该变量从创建到删除，在内存中存在的时间长度。

作用域是该变量可被访问的范围。

```
#include <iostream>
using namespace std;

void Function() {
    static int i = 0;
    i++;
    cout << i << endl;
}

int main() {
    Function();
    Function();
    Function();
    Function();
    Function();
    cin.get();
}
```

上述代码打印的结果是**1 2 3 4 5**；如果没有static修饰，打印的结果将会是**1 1 1 1 1**。

一般情况下，函数内声明的变量的生命周期截止在函数执行完毕，作用域就是函数内部。

static修饰的局部静态变量的作用域不变，但是**生命周期延长到了整个程序的生命周期**。也就是说，自从第一次执行该函数后，之后每次执行该函数，访问的都是第一次创建的那个变量。

这是一段单例的实现代码：

```
#include <iostream>
using namespace std;

class Singleton {
private:
    static Singleton* s_Instance;
public:
    static Singleton& Get() { return *s_Instance; }

    void Hello(){}
};

Singleton* Singleton::s_Instance = nullptr;

int main() {
    Singleton::Get().Hello();
    cin.get();
}
```

我们可以使用静态局部变量来优化单例类的代码，让其更简洁，且功能保持不变（甚至更优）：

```
class Singleton {
private:
    static Singleton* s_Instance;
public:
    static Singleton& Get() {
```



```
        static Singleton instance;  
        return instance;  
    }  
  
    void Hello(){}  
};
```

我删去了单例类的静态成员变量s_Instance（所以不需要再初始化它了），而是在Get方法里使用静态局部变量来作为返回的实例引用。这样的话，该单例类的实例只在首次调用Get方法时被创建，这是一种懒加载，且这种方式是线程安全的，又避免了显式的空指针检查。

08 枚举 (enum)

2024年12月8日 21:55

当你想要使用整数来表示某些状态或者某些数值时，枚举将会非常有用。

枚举的本质就是整数。

再次回顾之前写的Log类：

```
#include <iostream>
using namespace std;

class Log {
public:
    const int LogLevelError = 0;
    const int LogLevelWarning = 1;
    const int LogLevelInfo = 2;

private:
    int m_LogLevel = LogLevelInfo;

public:
    void SetLevel(int level) {
        m_LogLevel = level;
    }

    void Error(const char* message) {
        if (m_LogLevel >= LogLevelError) {
            cout << "[Error]:" << message << endl;
        }
    }

    void Warn(const char* message) {
        if (m_LogLevel >= LogLevelWarning) {
            cout << "[Warning]:" << message << endl;
        }
    }

    void Info(const char* message) {
        if (m_LogLevel >= LogLevelInfo) {
            cout << "[Info]:" << message << endl;
        }
    }
};
```

我们可以发现，标注出的地方很适合使用枚举。于是代码可以修改为如下：

```
#include <iostream>
using namespace std;
```

```

class Log {
public:
    enum Level {
        LevelError = 0, LevelWarning, LevelInfo
    };

private:
    Level m_LogLevel = LevelInfo;

public:
    void SetLevel(Level level) {
        m_LogLevel = level;
    }

    void Error(const char* message) {
        if (m_LogLevel >= LevelError) {
            cout << "[Error]:" << message << endl;
        }
    }

    void Warn(const char* message) {
        if (m_LogLevel >= LevelWarning) {
            cout << "[Warning]:" << message << endl;
        }
    }

    void Info(const char* message) {
        if (m_LogLevel >= LevelInfo) {
            cout << "[Info]:" << message << endl;
        }
    }
};

```

枚举的本质就是整数，它的默认存储类型是int，可以根据需要修改，但必须都是整型。比如：

```

enum Example : unsigned char {
    A,B,C
};

```

这样可以更大限度地节约内存。常用的类型有：int、short、long、long long以及它们对应的无符号整型。

09 构造函数&析构函数

2024年12月20日 20:39

构造函数：

与Java不同（int与float默认初始化为0），C++中的所有基本类型都需要手动初始化。

所以我们更加需要构造函数。

构造函数**当且仅当**在实例化类的对象时自动被调用。所以，当使用类的静态方法时，它不会被调用。

C++中每个类都有默认的空构造函数。

```
#include <iostream>
using namespace std;

class Entity {
public:
    int X, Y;
    void Print() {
        cout << X << " " << Y << endl;
    }
};

int main() {

    Entity e;
    e.Print();
    cin.get();
}
```

当没有构造函数的时候，以上代码在有的编译器内会报错，而在vs中可以正常运行，不过打印出的X,Y值分别如下：

-858993460 -858993460

至于X,Y默认为什么是这个数值，我们可以找到它们在内存中的地址。

地址: 0xFFFFFFFFCCCCCCCC

我的电脑是64位系统。这个地址是一个十六进制数，由16个十六进制数字组成。每个十六进制数字代表4位二进制数，因此16个十六进制数字总共代表64位二进制数。

0xFFFFFFFFCCCCCCCC是C++在VS中的默认填充值，这个值转化成二进制后先取反码再取补码，再转化成十进制就可以得到-858993460。

不过，当我使用另一种方法实例化对象时，情况却不同了：

```
int main() {

    Entity* e = new Entity();
    e->Print();
    cin.get();
}
```

x, y的值为0。

```
0 0
```

这是因为使用new关键字是在**堆**上为对象分配内存，编译器在申请内存的时候就顺便把内存里的内容（原本是-858993460）刷新成0了，而上一种写法是在**栈**上直接创建对象，没有申请内存的步骤，所以也就没有把内存刷新为0的这一步。

所以，并不是x,y被初始化为0。它们依然没有被初始化，而是它们等于内存里的值，内存里的值被改变了而已。

现在回到正题，我们写好构造函数。

```
#include <iostream>
using namespace std;

class Entity {
public:
    int X, Y;
    Entity() {
        X = 0;
        Y = 0;
    }

    Entity(int x, int y) {
        X = x;
        Y = y;
    }

    void Print() {
        cout << X << " " << Y << endl;
    }
};
```

也可以使用它的另一个重载：

```
Entity* e = new Entity(1,2);
```

不过当我们想要创建一个静态类时，我们不希望这个类存在任何对象。可以用以下两种方法**禁用它的构造函数**：

- 用private修饰构造函数：

```
private:
    Entity() {
        X = 0;
        Y = 0;
    }
```

```

        Entity(int x, int y) {
            X = x;
            Y = y;
        }
};

```

- 删除构造函数:

```
Entity() = delete;
```

(这句代码的意思是把这个类默认存在的空构造函数删除)

析构函数:

与构造函数相反, 析构函数在对象被销毁时自动被调用。

```

#include <iostream>
using namespace std;

class Entity {
public:
    int X, Y;
    //Entity() = delete;

    Entity() {
        cout << "Created entity" << endl;
        X = 0;
        Y = 0;
    }

    //析构函数使用~修饰
    ~Entity()
    {
        cout << "Destroyed entity" << endl;
    }

    void Print() {
        cout << X << " " << Y << endl;
    }
};

void Function() {
    Entity e;
    e.Print();
}

int main() {
    Function();
    cin.get();
}

```

此处如果不封装Function函数, 则无法看到析构函数打印的信息, 因为对象在栈上被创建, 在main函数结束后才会被销毁。

但是当代码稍作修改如下时，就必须手动删除对象才能够触发析构函数：

```
#include <iostream>
using namespace std;

class Entity {
public:
    int X, Y;
    Entity() {
        cout << "Created entity" << endl;
        X = 0;
        Y = 0;
    }

    ~Entity() {
        cout << "Destroyed entity" << endl;
    }

    void Print() {
        cout << X << " " << Y << endl;
    }
};

void Function() {
    Entity* e = new Entity();
    e->Print();
    // 注意这里需要手动删除对象
    delete e;
}

int main() {
    Function();
    cin.get();
}
```

这是因为使用new关键字新建对象会使对象在堆上被创建，从而不会在函数结束时自动销毁，需要手动释放内存。

10 继承

2025年1月10日 22:59

```
#include <iostream>
using namespace std;

class Entity {
public:
    float X, Y;
    void Move(float xa, float ya) {
        X += xa;
        Y += ya;
    }
};

class Player : public Entity{
public:
    const char* Name;

    void PrintName() {
        cout << Name << endl;
    }
};

int main() {
    Player player;
    player.PrintName();
    cin.get();
}
```

上述代码实现了简单的继承。值得注意的是声明子类时，可见性可选 `public`、`protected`、`private` 共三种，关于他们的区别，详细的学习会在13节展开，这里先贴出AI的回答：

在C++中，类的继承可以使用三种访问修饰符：`public`、`protected` 和 `private`。这些修饰符决定了基类（被继承的类）的成员在派生类（继承的类）中的访问权限。对于你提供的代码，`Entity` 类前面的 `public` 修饰符可以改为 `protected` 或 `private`，但它们的含义和影响有所不同。

1. `public` 继承

- **定义：**使用 `public` 继承时，基类的公有成员（`public`）在派生类中保持公有，基类的保护成员（`protected`）在派生类中保持保护，基类的私有成员（`private`）在派生类中不可访问。
- **示例：**

```
cpp
class Entity {
public:
    float X, Y;
protected:
    int protectedVar;
```



```
private:
    int privateVar;
};

class Player : public Entity {
public:
    void Print() {
        cout << X << " " << Y << endl; // 可以访问 X 和 Y
        cout << protectedVar << endl;    // 可以访问 protectedVar
        // cout << privateVar << endl;    // 编译错误, privateVar 不可访问
    }
};
```

2. protected 继承

- **定义:** 使用 `protected` 继承时, 基类的公有成员和保护成员在派生类中都变为保护成员, 基类的私有成员在派生类中仍然不可访问。
- **示例:**

```
cpp
class Player : protected Entity {
public:
    void Print() {
        cout << X << " " << Y << endl; // 可以访问 X 和 Y, 但它们现在是 protected
        cout << protectedVar << endl;    // 可以访问 protectedVar
        // cout << privateVar << endl;    // 编译错误, privateVar 不可访问
    }
};
```

- 在这种情况下, `X` 和 `Y` 在 `Player` 类中是保护成员, 这意味着它们不能被 `Player` 类的对象直接访问, 但可以被 `Player` 类的派生类访问。

3. private 继承

- **定义:** 使用 `private` 继承时, 基类的公有成员和保护成员在派生类中都变为私有成员, 基类的私有成员在派生类中仍然不可访问。
- **示例:**

```
cpp
class Player : private Entity {
public:
    void Print() {
        cout << X << " " << Y << endl; // 可以访问 X 和 Y, 但它们现在是 private
        cout << protectedVar << endl;    // 可以访问 protectedVar
        // cout << privateVar << endl;    // 编译错误, privateVar 不可访问
    }
};
```

- 在这种情况下，X 和 Y 在 Player 类中是私有成员，这意味着它们不能被 Player 类的对象或 Player 类的派生类直接访问。

选择合适的继承方式

- **public 继承**：最常用，表示“是一个”（is-a）关系。例如，Player 是一个 Entity。
- **protected 继承**：较少使用，通常用于中间类，这些类不打算被直接实例化，但会被进一步派生。
- **private 继承**：表示“有一个”（has-a）关系，但通常更推荐使用组合（composition）而不是私有继承。

在你的具体场景中，如果 Player 是一个 Entity，使用 public 继承是最合适的，因为它清晰地表达了这种“是一个”关系，并且允许 Player 类的对象直接访问继承自 Entity 的公有成员。

来自 <<https://kimi.moonshot.cn/chat/cu0j7pknvj4gbroajm40>>

简而言之：

- **public**：会完全继承父类，不做任何修改，如果子类中没有任何新加的内容，那么他将等于父类。
- **protected**：会把父类中所有的公有成员和保护成员继承过来，并转化为保护成员。
- **private**：会把父类中所有的公有成员和保护成员继承过来，并转化为私有成员。

公有成员：任何范围下都可访问。

保护成员：外界无法直接访问，但可以被子类继承。

私有成员：外界无法直接访问，不可被子类继承。

11 虚函数

2025年1月10日 23:27

在讲虚函数之前，两个现象可以总结出C++继承的一个特点：

1、如果父类中非私有的方法没有在子类中出现，那么子类会自动继承。

```
#include <iostream>
#include <string>
using namespace std;

class Entity {
public:
    std::string GetName() { return "Entity"; }
};

class Player : public Entity{
private:
    std::string m_Name;

public:
    Player(const std::string& name)
        : m_Name(name){}

    //std::string GetName() { return m_Name; }
};

int main() {
    Entity* e = new Entity();
    cout << e->GetName() << endl;

    Player* p = new Player("LJJ");
    cout << p->GetName() << endl;

    cin.get();
}
```

比如上述代码的输出结果就是两个 “Entity” 。

如果取消注释，就是预期的一行 “Entity” ，一行 “LJ” ，没有问题。

2、子类继承了父类的方法，但是碰到使用多态的情况：

```
#include <iostream>
#include <string>
using namespace std;

class Entity {
```

```

public:
    std::string GetName() { return "Entity"; }
};

class Player : public Entity{
private:
    std::string m_Name;

public:
    Player(const std::string& name)
        : m_Name(name){}

    std::string GetName() { return m_Name; }
};

int main() {
    Entity* e = new Entity();
    cout << e->GetName() << endl;

    Player* p = new Player("LJJ");
    cout << p->GetName() << endl;

    //让类型为Entity的指针指向其子类Player的对象
    Entity* entity = p;
    cout << entity->GetName() << endl;

    cin.get();
}

```

```

Entity
LJJ
Entity

```

输出结果如上，entity指针明明指向了Player对象，但是仍然调用了Entity类中的GetName方法。

我们希望的多态并不是这样的，子类应该是父类的超集，它可以适用父类适用的所有情况，同时有自己的附加功能。但现在显然子类的附加功能没有实现。

通过上面两种情况，可以看出C++继承的一个特点：**子类关心谁是父类，但父类并不关心谁是子类。**

这个问题的关键在于**静态绑定（Static Binding）和动态绑定（Dynamic Binding）的概念。

- **静态绑定**：在编译时确定调用哪个函数。C++ 中的普通函数调用（**非虚函数**）使用静态绑定。编译器根据指针或引用的静态类型（即声明类型）来决定调用哪个函数。
- **动态绑定**：在运行时确定调用哪个函数。C++ 中的**虚函数**（Virtual Functions）使用动态绑定。编译器根据对象的实际类型来决定调用哪个函数。

默认情况下，编译器使用的都是静态绑定，它会根据声明的类型“不加检查地”调用该类型的函数，即使实际指向的对象可能是是声明类型的子类。要想使用动态绑定，就要引入**虚函数**。这只需要在代码中做出两处简单的修改。

```

class Entity {
public:
    virtual std::string GetName() { return "Entity"; }
};

class Player : public Entity{
private:
    std::string m_Name;

public:
    Player(const std::string& name)
        : m_Name(name){}

    std::string GetName() override { return m_Name; }
    //注意，这里的GetName方法虽然没有使用virtual修饰，但override同样赋予了它虚函数的特性
    //它同样是虚函数
};

```

```

Entity
LJJ
LJJ

```

现在我们得到了预期的输出结果。

虚函数运作的基础是**虚函数表 (Virtual Table)**：

1. 虚函数表的定义

- 虚函数表是一个数组，存储了类中所有虚函数的地址。每个包含虚函数的类都有一个虚函数表。虚函数表在编译时生成，并在运行时被对象使用。
- 每个对象在内存中都有一个指针（通常称为虚表指针，vptr）指向其所属类的虚函数表。通过这个指针，对象可以找到并调用其虚函数。

2. 虚函数表的工作原理

- 当创建一个包含虚函数的类的对象时，编译器会自动在对象的内存布局中插入一个指针（vptr），指向该类的虚函数表。
- 当通过基类指针或引用调用虚函数时，编译器会通过vptr找到虚函数表，然后通过虚函数表中的地址调用相应的虚函数。

显然，使用虚函数是有代价的：

- 空间上，需要额外的内存来存储虚函数表以及虚表指针。
- 时间上，动态绑定要求每次调用虚函数前都要在虚函数表中进行遍历查找来确定需要映射到的虚函数。

但是这对于性能的影响是微乎其微的，一般还是建议使用虚函数。

12 接口与纯虚函数

2025年1月11日 20:21

其他语言（比如JAVA）有接口关键字interface，但是C++没有，接口在C++中也用类表示，特点是这种类中只存在纯虚函数。

```
#include <iostream>
#include <string>
using namespace std;

class Printable {
public:
    virtual std::string GetClassName() = 0;
};

class Entity : public Printable{
public:
    virtual std::string GetName() { return "Entity"; }
    std::string GetClassName() override { return "Entity"; }
};

class Player : public Entity{
private:
    std::string m_Name;

public:
    Player(const std::string& name)
        : m_Name(name){}

    std::string GetName() override { return m_Name; }
    std::string GetClassName() override { return "Player"; }
};

void Print(Printable* obj) {
    cout << obj->GetClassName() << endl;
}

int main() {
    Entity* e = new Entity();
    Print(e);

    Player* p = new Player("LJJ");
    Print(p);

    cin.get();
}
```

上述代码中的Printable类就是接口，因为它只包含一个纯虚函数，而Entity和Player类都实现了这个接口。

注意：接口是无法被实例化的。继承了接口的类，如果不将接口的纯虚函数实现，也是无法进行实例化的。

补充：

上述代码中有这些片段：

```
virtual std::string GetName() { return "Entity"; }  
std::string m_Name;
```

使用 `std::` 前缀的目的是明确这些标识符属于标准库，从而避免命名冲突。

可以使用 `using namespace std;` 这句代码来将标准库中的所有标识符引入全局命名空间，从而省去使用 `std::` 前缀的麻烦，**但是并不推荐这样做，因为这会引起命名冲突，以及在大型项目中引发许多难以调试的错误。**

更合适的做法是：

```
using std::string; // 引入特定的标识符  
using std::cout;  
using std::endl;
```

另外，关于这句代码可能对初学者来说有些难懂：

```
Player(const std::string& name)  
    : m_Name(name){}
```

这句代码是一个构造函数的定义，它使用了**初始化列表**（initializer list）来初始化类的成员变量，表示将传入的参数 `name` 的值赋给成员变量 `m_Name`。初始化列表用于在构造函数体执行**之前**初始化类的成员变量。这种方法效率更高，尤其是对于一些复杂的对象（如 `std::string`）。

它等价于如下代码：

```
Player(const std::string& name) {  
    m_Name = name; // 在构造函数体中赋值  
}
```

但是在构造函数体中赋值的方法在很多情况下不适用，如：

- 初始化**引用**成员变量；
- 初始化**常量**成员变量；
- 初始化**没有默认构造函数的类**的成员变量；

所以尽量使用初始化列表，这种方法效率更高。使用初始化列表初始化多个成员变量代码参考如下：

```
class MultiInit {
private:
    int a;
    double b;
    std::string c;
public:
    MultiInit(int x, double y, const std::string& z) : a(x), b(y), c(z) {
        // 构造函数体
    }
};
```


13 可见性

2025年1月11日 21:04

C++中类的成员的可见性只对于组织代码起到作用，对于实际程序运行没有任何影响。

- 1.private段（缺省）：只有类中的成员函数可以访问该段的成员。
- 2.protected段：只有类中成员函数和它的派生类可以访问段中成员。
- 3.public段：类中成员函数、类中的对象、派生类的成员函数及对象都可以访问该段中的成员。

值得一提的是，C++中类的成员默认可见性是private；结构体的成员默认可见性是public。

一个类的友元（friend）也可以访问该类的私有成员。

关于这部分理论知识非常简单，到这里就结束了。重要的是该如何合理使用这三种可见性。

有些人总是将类所有的数据成员写成私有，并一一奉上公开的setter与getter方法，这种习惯是完全没有必要的。也许很多时候我们应该这么写，但是“一直如此”就肯定不是正确的。我们需要视情况选择合理的办法。

举个例子，在pygame中开发游戏是一个非常底层的任务，我们需要从头开始写游戏循环等各种底层逻辑，包括屏幕刷新。

这就会导致一个现象：我将某个物体的x，y坐标赋值成某个向量，但是它的位置并没有发生任何改变。因为还差一步：刷新屏幕显示。

所以，在代码中，尽管物体类中的x，y是公开成员变量，我可以在外界直接访问，但是这样做会引发难以调试的错误。所以我们这个时候就需要将x,y设为私有成员变量，再写一个公开成员函数setPosition()。在这个函数中同时完成对x,y的赋值与刷新屏幕refresh()。

这样不仅代码的可读性更强，而且程序的健壮性也得到了保证。

14 数组

2025年1月12日 12:58

指针是C++数组工作的基础。

C++中的数组声明很简单:

```
int example[5];
```

具体操作也和其他语言类似，不再赘述。

C++中如果试图访问数组之外的索引，会出现**内存访问违规（Memory access violation）**。

简单来说，就是数组在试图访问不属于自己的内存。在debug模式下，程序将会崩溃报错，来帮助调试；但是在release模式下，我们可能不会得到任何报错消息，然后这个操作就会悄悄地改变其他地方的内存，它有可能是源代码中另一个变量的内存，这就会引发严重的错误。

```
#include <iostream>
using namespace std;

int main() {
    int example[5];
    for (int i = 0; i < 5; i++) {
        example[i] = 0;
    }
    cout << example[0] << endl;
    cout << example[-1] << endl;
    cin.get();
}
```

值得一提的是，C++中不存在数组[-1]索引的写法（这在其它语言中通常代表数组的最后一个元素）。也就是说，上述代码的 `cout << example[-1] << endl;` 实际上就是内存访问违规操作。程序最后打印出的结果也证实了这一点。

0
-858993460

补充小细节:

```
for (int i = 0; i < 5; i++) {
    example[i] = 0;
}
```

循环中的边界条件一般写为 $i < 5$ 而不是 $i \leq 4$ ；因为小于等于比较需要额外进行等于的比较，更加占用内存。

数组是顺序存储结构，数组实际上类似一个指针，它指向数组第一个元素的内存地址。当我们通过索引访问数组的元素时，实际上就是找到这个指针指向的内存地址，再加上元素相应类型的内存大小*索引数来作为偏移量，最终能够准确的找到数组里的某一个元素。这也能够解释为什么数组都要以 0 作为第一个元素的索引值。

[illegible]

以例子中的example数组为例，它是一个长度为5的整型数组，每个整型占4字节的内存，那么该数组就会占用一段连续的20字节大小的内存，每4个字节一个划分，代表其中的一个元

素。

```
#include <iostream>
using namespace std;

int main() {

    int example[5];
    int* ptr = example;    //验证了数组可以直接为指针赋值

    for (int i = 0; i < 5; i++) {
        example[i] = 0;
    }

    example[2] = 5;
    cout << example[2] << endl;

    *(ptr + 2) = 6;
    cout << example[2] << endl;

    //这两个操作本质上完全一样

    cin.get();
}
```

其中这一步操作 `*(ptr + 2) = 6;` 中 `+2` 实际上是加上 `4 * 2` 共8字节的偏移量，前面的因数取决于指针的类型。它也可以写成 `*(int*)((char*)ptr + 8) = 6;`

数组与指针一定程度上可以等同，但是二者仍有些区别：

- **类型不同：**数组是一个具有固定大小和类型的数据结构，而指针是一个变量，它存储着另一个变量（或内存块）的地址。指针的大小是固定的（在大多数平台上，指针大小为4字节或8字节），而数组的大小由其元素数量和类型决定。
- **内存分配和释放：**数组在声明时就分配了固定大小的内存，其内存分配是在栈上（局部数组）或全局数据区（全局数组），不需要手动释放。指针可以指向动态分配的内存（通过new或malloc等），需要手动释放内存（通过delete或free）。
- **可修改性：**数组的大小和内存地址是固定的，不能改变。而指针的值（即它指向的地址）是可以改变的，可以指向不同的内存位置。

刚刚的例子中的写法是在**栈**上创建数组，我们还可以使用new关键字在**堆**上为数组申请内存。

```
#include <iostream>
using namespace std;

int main() {
    int example[5];

    int* another = new int[5];
    delete[] another;

    cin.get();
}
```

注意：栈上创建的数组，当程序运行到函数的结束花括号之外时，就会被销毁。但是堆上申请的内存需要及时手动释放，否则只能等到程序结束后才会销毁。

关于数组应该在栈上还是在堆上创建，没有哪个更好的说法，而是视情况而定。

比如，当一个函数的返回值是一个数组，且这个数组是在函数内部创建的，这个时候就**必须**要在堆上创建这个数组。

但是在堆上创建数组也有它的缺点，那就是**间接寻址**所产生的**内存碎片**和**缓存丢失**以及额外的性能损耗。一个简单的例子：

```
#include <iostream>
using namespace std;

class Entity {
```

```
public:
    int example[5];

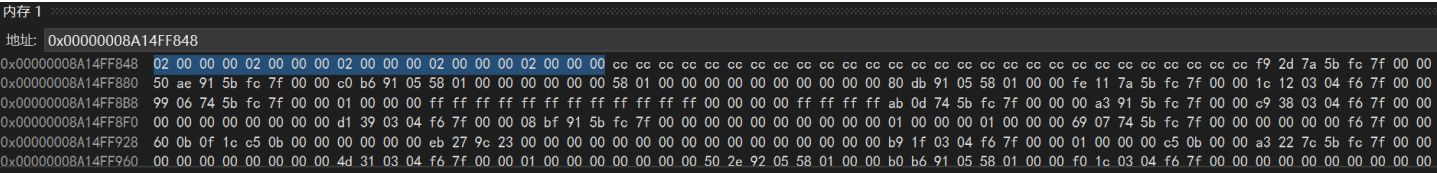
    Entity() {
        for (int i = 0; i < 5; i++) {
            example[i] = 2;
        }
    };

int main() {

    Entity e;

    cin.get();
}
```

这时Entity中的数组是在栈上创建的。打断点调试后访问对象e的地址：



发现e的地址就存放着example数组。

我们再换个写法，在堆上创建example数组：

```
#include <iostream>
using namespace std;

class Entity {
public:
    int* example = new int[5];

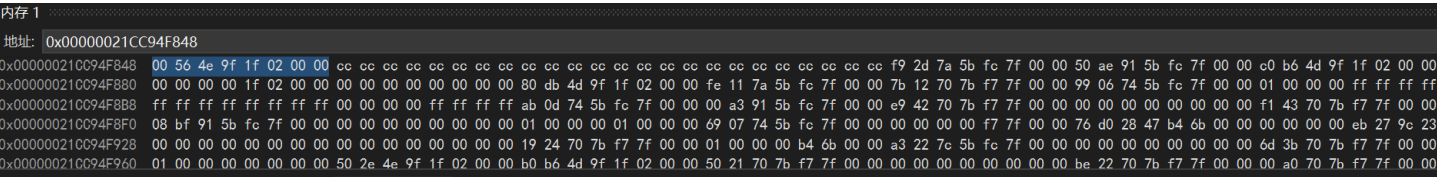
    Entity() {
        for (int i = 0; i < 5; i++) {
            example[i] = 2;
        }
    };

int main() {

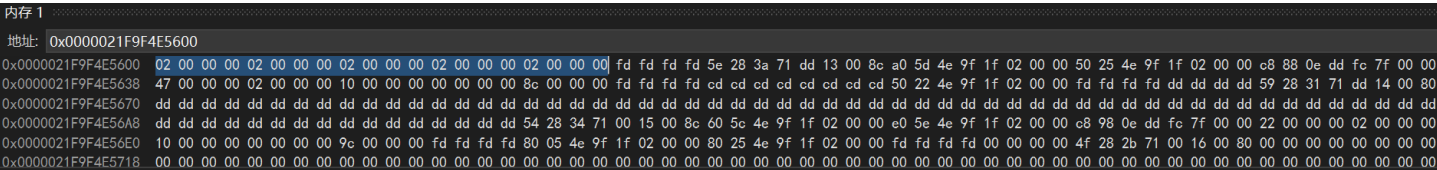
    Entity e;

    cin.get();
}
```

此时运行后访问对象e的地址：



发现这里并不是example数组的内存，而是另一个内存地址。将这个内存地址逐字节倒过来写（因为字节存储次序的原因），再次访问，我们才能找到example数组：



这就是间接寻址，所以也不奇怪它会多消耗额外的性能。

不幸的是，c++的原始数组并不支持获取它的长度。比如在其它语言中常见的size(),length()之类的方法，这些都没有。

```
int a[5];
int count = sizeof(a) / sizeof(int);
```

也许我们可以通过这种方式来计算数组的长度，但前提必须是已知数组的类型，以及这个数组必须是**静态数组**（换句话说，栈上创建的数组）。

当数组为**动态数组**时（堆上创建的数组），`sizeof(a)`的返回值就会变成该指针类型的大小，而不是数组的内存大小，从而引发严重的错误。

所以必须养成自己维护自己数组大小的习惯，比如使用常量变量来规定数组的大小：

```
const int size = 5;
int* example = new int[size];
```

或者：

```
static const int size = 5;
int example[size];
```

不过在C++11中提供了**`std::array`**，解决了这个问题。它可以通过`size()`方法获取长度，也可以自己进行边界检查，额外的开销是不可避免的，但是它仍然好用。

15 字符串

2025年1月14日 18:53

字符：

C++默认使用ASCII字符进行文本编码。

默认情况下char类型占用1字节的内存，特殊情况是指日文、中文、韩文等语言的一些字母，它们的字符可能占多个字节。

当使用char指针时，可以用字节来做指针运算。

char也可以用来分配内存缓冲区，比如你想分配1kb的内存，那么可以分配1024个char。

字符串：

char属于基本数据类型，字符串本质上是字符数组。

```
const char* name = "ljj";
```

C++中声明字符串与其他语言不同，并不是以string为数据类型进行声明。

注意，在较新版本的C++中声明字符串时的const是必须的，说明字符串在C++中是不可变的。

从内存中观察字符串，它的每个字节对应的就是一个字符的ASCII码。且每个字符串的结尾都会有一个**空终止字符**，也就是0，它标志着一个字符串的结束。

```
#include <iostream>
using namespace std;

class Entity {
public:
};

int main() {

    const char* name = "ljj";
    char name2[3] = { 'l', 'j', 'j' };

    cout << name << endl;
    cout << name2 << endl;

    cin.get();
}
```

比如上面这段代码，当我们手动创建字符数组来模拟字符串的时候，因为缺少空终止字符，程序会在打印完应有的3个字符后继续打印，直到遇见内存中的下一个0。而直接创建的字符串不用担心这个问题，编译器会自动在末尾补上空终止字符。

```
ljj
ljj烫烫烫烫烫烫烫烫烫烫烫烫烫烫烫贴-;I?
```

要修正这个错误，只需要在字符数组最后加上 '\0' 或 0，当然字符数组的大小也要增加1。

```
char name2[4] = { 'l','j','j' ,'\0'};

char name2[4] = { 'l','j','j' ,0};
```

我们也可以选择使用 `std::string`，它的本质也是 `char*` 或者 `char` 数组，但是作为官方封装好的数据类型，使用起来会更加方便。

C++ 中的字符串不能直接相加，因为这种操作本质上是在将两个字符指针相加，这是非法的。但是有两种方法来实现字符串拼接：

```
#include <iostream>
using namespace std;

class Entity {
public:
};

int main() {

    //错误写法
    //std::string name = "ljj" + "hello";

    std::string name = "ljj";
    name += ",hello";

    std::string name2 = std::string("ljj") + ",hello";

    cout << name << endl;
    cout << name2 << endl;

    cin.get();
}
```

注意，如果将 `std::string` 类型的变量直接作为参数传递给函数的话，实际上传递进去的实际上是原字符串的复制体，之后在函数体内进行任何对字符串修改的操作都不会影响到原字符串。而且复制字符串实际上是一个相当影响性能的操作，不建议这样写。下面是例子：

```
#include <iostream>
```

```
using namespace std;

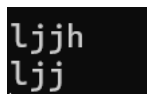
void PrintString(std::string string) {
    string += "h";
    cout << string << endl;
}

int main() {

    std::string name = "ljj";

    PrintString(name);
    cout << name << endl;

    cin.get();
}
```



可以看到原字符串name没有被改变。

通常情况下，通过**常量引用**来传递字符串是合适的做法，不过这样的话就无法进行修改字符串的操作了。但是一般字符串都存储在常量区，如果你需要直接修改一个字符串的内容，这个行为本身就是不合理的。（常量引用在将类的对象作为参数传递时也经常使用。）

```
void PrintString(const std::string& string) {
    cout << string << endl;
}
```

其他字符类型：

```
#include <iostream>
using namespace std;

int main() {

    const char* name = u8"Ljj";
    const wchar_t* name2 = L"Ljj";
    const char16_t* name3 = u"Ljj";
    const char32_t* name4 = U"Ljj";

    cin.get();
}
```

- **char**: 通常为1个字节（8位），其大小是固定的，能够表示的字符范围有限，一般用于表示ASCII字符集中的字符，可以表示128个不同的字符（扩展ASCII可以表示256个字符）。
- **wchar_t**: 大小不是固定的，依赖于编译器和平台。在Windows平台上，wchar_t通常为2个字节，可以表示65536个不同的字符；在Linux和macOS平台上，wchar_t通常为4个字节，能够表示的字符数量更多。它主要用于表示宽字符，以支持多种字符集，但不同平台间的兼容性较

差。

- **char16_t**: 固定为2个字节（16位），专门用于表示UTF-16编码的字符。UTF-16编码使用16位来表示大部分Unicode字符，对于一些不常用的字符会使用代理对来表示，其大小介于char和char32_t之间，在表示常用字符时较为高效，但对于特殊字符的处理相对复杂。
- **char32_t**: 固定为4个字节（32位），用于表示UTF-32编码的字符。UTF-32编码使用32位来表示每一个Unicode字符，每个字符占用固定长度的空间，能够表示的字符数量非常多，编码方式简单直观，但在存储和传输大量文本时会占用较多空间。

16 const关键字

2025年1月14日 20:36

const关键字的作用类似类和结构体内部的可见性，只对组织代码起作用，对实际程序运行没有影响。

使用const就像我们承诺某个变量是常量，我们不会改变它的值，但也仅仅是承诺。

1、声明一个常量：

```
const int MAX_AGE = 90;
```

这是const最简单的应用场景，声明一个不可改变的值。

2、与指针搭配使用（常量指针和指针常量）：

正常情况下，我们可以对指针做两件事：改变指针指向的地址、改变指针指向的内存的内容。当指针搭配const时，情况就会稍微复杂一些：

```
#include <iostream>
using namespace std;

int main() {
    ///禁止改变指针指向的内容：比如指针指向一个值为20的内存，那么这个内存的值将会
    变成常量，始终为20，无法被修改。但是可以让指针重新指向别的地址。///
    ///常量指针
    int const* a = new int;

    ///与int const*相同//
    const int* b = new int;

    ///禁止改变指针指向的地址：这个指针只能指向内存中的某个地址，但是这个地址上的
    内存的值可以进行任意的修改。///
    ///指针常量
    int* const c = new int;

    ///既不能改变指针指向的地址，又不能改变指针指向的内存的内容
    const int* const d = new int;

    cin.get();
}
```

```
}
```

3、在类中使用（常量成员函数）：

```
class Entity {  
private:  
    int m_X,m_Y;  
public:  
    int GetX() const {  
        return m_X;  
    }  
  
    void SetX(int x) {  
        m_X = x;  
    }  
};
```

在类中的成员函数方法名（准确来说是参数列表）后面加上const，说明我们“承诺”不在这个方法内对类的对象进行任何修改，也就是说，这个方法只进行读取操作。这种方法称为**常量成员函数**。

有个地方必须要用到常量成员函数：

```
void PrintEntity(const Entity& e) {  
    cout << e.GetX() << endl;  
}
```

这里PrintEntity函数接收的参数就是我们之前提到的**常量引用**。如果此时GetX()方法不是常量成员函数，那么就会出现编译错误。因为传入的e被const修饰，说明承诺e不会被修改。而只有GetX()方法为常量成员函数，才能保证不会出现e被修改的风险。

但是有时候我们出于某些原因，需要在常量成员函数内修改一些变量，那么就需要mutable关键字：

```
class Entity {  
private:  
    int m_X,m_Y;  
    mutable int var;  
public:  
    int GetX() const {  
        var = 2;  
        return m_X;  
    }  
  
    void SetX(int x) {  
        m_X = x;  
    }  
};
```

mutable允许让一些变量成为特例，可以在常量成员函数内被修改。

17 成员初始化列表

2025年1月15日 11:58

之前在12节提到过初始化列表，它是一种更加高效的类初始化方法。

```
class Entity {
private:
    const char* m_Name;
    int m_Score;
public:
    Entity() : m_Name("Unknown"), m_Score(0) {
    }

    Entity() {
        m_Name = "Unknown";
        m_Score = 0;
    }
};
```

示例中的两个构造方法实现的功能完全一样，第二种是我们在其他语言中比较熟悉的写法，第一种就是使用了初始化列表来进行成员初始化。

在C++中我们更应该使用初始化列表，不仅因为它性能更佳，而且是有一些情况必须使用它，否则会出现错误。

值得一提的是，初始化列表中各个变量的初始化顺序应该与变量声明的顺序保持一致。而且即使你不这么做，编译器在实际工作的时候也是按照这个顺序进行初始化的。

现在来讨论一下为什么初始化列表更优的问题。

比如例子中的代码，如果是第二个构造函数（不使用初始化列表），那么实际上字符串m_Name变量会被初始化两次，为了更直观的看到，我加入一个类的对象作为成员变量：

```
#include <iostream>
using namespace std;

class Example {
public:
    Example() {
        cout << "Created Entity!" << endl;
    }

    Example(int x) {
        cout << "Created Entity with " << x << " !" << endl;
    }
};

class Entity {
```

```

private:
    const char* m_Name;
    int m_Score;
    Example m_Example;

public:
    Entity() {
        m_Name = "Unknown";
        m_Score = 0;
        m_Example = Example(8);
    }
};

int main() {
    Entity e;
    cin.get();
}

```

```

Created Entity!
Created Entity with 8 !

```

可以看到，我们只创建了一个Entity对象，却初始化了两个Example对象，且第一个Example对象完全没有任何作用，只是占用性能，然后被废弃。

现在把构造函数修改成使用初始化列表的形式：

```

Entity() : m_Name("Unknown"), m_Score(0), m_Example(Example(8)) {
}

```

```

Created Entity with 8 !

```

现在只有一个Example对象被创建了。

所以结论就很简单了：**无论何时都要使用初始化列表**。它不仅仅是代码风格的问题，最重要的是，不使用它就会浪费性能。

18 三目操作符

2025年1月15日 12:40

三目操作符本质上就是if&else语句，它是一个语法糖。

C++中三目操作符的使用方式跟其他语言没有什么区别，就是由？和：组成的一个操作符。

```
#include <iostream>
using namespace std;

int main() {
    int level = 0;
    int speed = 0;

    if (level > 5) {
        speed = 10;
    }
    else {
        speed = 5;
    }

    speed = level > 5 ? 10 : 5;

    cin.get();
}
```

一个简单的例子，这里的if语句和三目操作符做了同样的事情。可以看到三目操作符的代码简洁的多。

既然它是if语句的语法糖，那么理所应当地三目操作符也可以进行嵌套操作：

```
speed = level > 5 ? level > 10 ? 15 : 10 : 5;
```

不过一旦进行嵌套操作，三目操作符的可读性就会变差。上面的代码实际上是这个意思：

```
if (level > 5) {
    if (level > 10) {
        speed = 15;
    }
    else {
        speed = 10;
    }
}
else {
    speed = 5;
}
```

三目操作符也可以和逻辑运算符进行搭配使用：

```
speed = level > 5 && level < 100 ? level > 10 ? 15 : 10 : 5;
```

比如标红的片段，即使没有括号也能作为条件正常工作。

不过，**大多数情况下并不建议对三目操作符进行嵌套**，尤其是团队合作时，这样会让你的代码变得晦涩难懂。

19 对象的栈分配与堆分配

2025年1月15日 13:04

C++中对象的创建一般分为两大类：栈分配与堆分配。

顾名思义，这两种分配方式的区别就在于对象的内存所在的空间位置。

1、栈分配：

```
#include <iostream>
using namespace std;

class Entity {
public:
    const char* m_Name;

public:
    Entity() : m_Name("Unknown") {
    }

    Entity(const char* const& name) : m_Name(name){
    }
};

int main() {
    Entity e("Ljj");
    cin.get();
}
```

上面的写法是典型的栈分配。此时对象e分配在栈上，在main函数结束后自动被销毁。栈分配的对象的生命周期与其被创建时的代码块的作用域相关，离开作用域，该对象就会自动被销毁。

一般情况下尽可能使用栈分配，因为这样不必手动管理内存。

但是栈分配是有局限性的。首先栈的空间非常小，如果一个类的体积过于庞大，或者说需要创建非常多的对象，那么就不建议使用栈分配。或者有时由于栈分配的作用域限制而无法实现某些功能，那么也需要使用堆分配。

2、堆分配：

```
Entity* e = new Entity("Ljj");
```

从栈分配转化到堆分配，只需要更改这一行代码即可。

区别是e的类型从Entity变成了Entity类型的指针，以及**new关键字**。实际上，new关键字才是

最核心的区别，正是因为它返回的类型是指针，所以需要变更e的类型。

堆分配除了需要手动释放内存之外，它也需要花费更多的时间。

因为对象的类型变成了指针，后续有关这个对象的一些操作符也要改变。

栈分配时的写法：

```
e.m_Name;
```

堆分配时的写法：

```
(*e).m_Name;
```

```
//或者
```

```
e->m_Name;
```

综上所述，只有当你不能使用栈分配的时候，再考虑堆分配。

20 new关键字

2025年1月15日 17:35

new关键字的本质就是向堆内存申请空间，找到一个满足需求的内存块，然后返回一个指向那块内存的指针。

实际上，new是一个操作符，它同样可以被重载（操作符重载）。

C语言中有一个用于申请内存的函数malloc()，它返回一个空指针（void*），new会调用这个函数。

```
Entity* e = new Entity();  
Entity* e = (Entity*)malloc(sizeof(Entity));
```

这两句代码的功能基本一样，它们都做到了申请内存并返回指针。只有一个区别，就是new还会调用Entity类的构造函数。注意，后者是C语言的写法，并不推荐在C++中这样写。

new关键字和delete关键字是形影不离的。使用new申请的内存，必须使用delete来手动释放。

delete也是一个操作符，它会调用free()函数，然后额外地还会调用类的析构函数。这与new和malloc的关系非常类似。

简单的使用案例，主要注意何时使用delete和delete[]：

```
int main() {  
    int* a = new int;  
    *a = 2;  
    int* b = new int[20];  
  
    delete a;  
    delete[] b;  
  
    cin.get();  
}
```

可能会困惑的一点是，上述例子中的b究竟是int指针还是数组？我们可以回顾数组那一节，数组的本质其实就是指针，只不过有些地方稍有改动。

所以本质上来说，b是一个int指针，它指向存储20个整型的内存块的第一个元素的地址。这在之前也提到过，也就是所谓的动态数组。这里的20实际上可以改成一个变量。

我们可以通过b[10]来获取这个数组的第11个元素，因为尽管b本质上是指针，但它也是数组。即：b = b[0]; b[i] = *(b + i)。

21 隐式转换与explicit关键字

2025年1月15日 18:15

C++允许编译器做一次**隐式转换。**

1、隐式转换：

```
#include <iostream>
using namespace std;

class Entity {
public:
    const char* m_Name;
    int m_Age;

public:
    Entity(int age) : m_Name("Unknown"),m_Age(age) {
    }

    Entity(const char* const& name) : m_Name(name), m_Age(-1){
    }
};

int main() {
    Entity e(19);
    Entity e2 = 19;

    Entity e3("Ljj");
    Entity e4 = "Ljj";

    cin.get();
}
```

例子中标红的两句代码看起来很奇怪，这在Java或者C#中绝对是错误的写法，但是在C++中却是正确的。

原因是Entity恰好有只以int为参数/只以字符串为参数的构造函数，那么就可以进行一次隐式转换来把int或字符串变量转换成Entity对象。

这在以Entity对象为参数的函数中同样适用：

```
void PrintEntity(const Entity& e) {
    cout << e.m_Name << " " << e.m_Age << endl;
}

int main() {
    PrintEntity(19);
    cin.get();
}
```

但是注意隐式转换只能进行一次，**多次隐式转换是不被允许的**。

比如Example类有一个以Entity对象为参数的构造函数，Entity对象有以int为参数的构造函数。

我可以直接让Example对象等于19，然后期待编译器先将整数19隐式转换成Entity对象，再将Entity对象隐式转换成Example对象吗？答案是：不行。

但是几乎没有人会这样写代码：

```
Entity e2 = 19;  
Entity e4 = "Ljj";
```

因为这并没有使代码更加简洁，反而会降低可读性。所以，了解即可。

2、explicit关键字：

用explicit来修饰构造函数，可以禁用该构造函数的隐式转换。

这时除了显式调用构造函数，显式转换也是被允许的。也就是说，可以有这些写法：

```
int main() {  
    //显式调用构造函数  
    Entity e(19);  
    Entity e2 = Entity(19);  
  
    //显式转换  
    Entity e3 = (Entity)19;  
  
    cin.get();  
}
```

22 运算符及其重载

2025年1月15日 18:41

C++支持运算符重载，C#部分支持这个功能，而Java则是完全不支持。

比如这个场景，我们需要定义二维向量以及它的一些运算：

```
#include <iostream>
using namespace std;

struct Vector2 {
    float x, y;
    Vector2(float x, float y) : x(x),y(y){

    }

    Vector2 Add(const Vector2& other) const {
        return Vector2(x + other.x, y + other.y);
    }

    Vector2 Multiply(const Vector2& other) const {
        return Vector2(x * other.x, y * other.y);
    }
};

int main() {
    Vector2 position(4.0f, 4.0f);
    Vector2 speed(0.5f, 1.5f);
    Vector2 powerup(1.1f, 1.1f);

    Vector2 result = position.Add(speed.Multiply(powerup));

    cin.get();
}
```

各种函数的嵌套调用使得可读性比较差，所以我们需要使用运算符重载。

```
#include <iostream>
using namespace std;

struct Vector2 {
    float x, y;
    Vector2(float x, float y) : x(x),y(y){

    }

    Vector2 Add(const Vector2& other) const {
        return Vector2(x + other.x, y + other.y);
    }

    Vector2 operator+(const Vector2& other) const {
```

```

        return Add(other);
    }

    Vector2 Multiply(const Vector2& other) const {
        return Vector2(x * other.x, y * other.y);
    }

    Vector2 operator*(const Vector2& other) const {
        return Multiply(other);
    }
};

int main() {
    Vector2 position(4.0f, 4.0f);
    Vector2 speed(0.5f, 1.5f);
    Vector2 powerup(1.1f, 1.1f);

    //Vector2 result = position.Add(speed.Multiply(powerup));
    Vector2 result = position + speed * powerup;

    cin.get();
}

```

这样一来，我们就可以使用简洁的符号来实现向量运算了。但是此时我们无法直接打印二维向量，因为我们没有对 << 符号进行重载。我们还需要加上：

```

std::ostream& operator<<(std::ostream& stream, const Vector2& other) {
    stream << other.x << ", " << other.y;
    return stream;
}

```

这时就可以直接打印输出向量了：

```

cout << result << endl;

```

```

4.55, 5.65

```

最后，我们还需要让向量之间可以通过 == 和 != 直接进行比较：

```

bool operator==(const Vector2& other) const{
    return x == other.x && y == other.y;
}

bool operator!=(const Vector2& other) const {
    return !(*this == other);
}

```

最后贴上完整的示例代码：

```

#include <iostream>

```

```

using namespace std;

struct Vector2 {
    float x, y;
    Vector2(float x, float y) : x(x),y(y){

    }

    Vector2 Add(const Vector2& other) const {
        return Vector2(x + other.x, y + other.y);
    }

    Vector2 operator+(const Vector2& other) const {
        return Add(other);
    }

    Vector2 Multiply(const Vector2& other) const {
        return Vector2(x * other.x, y * other.y);
    }

    Vector2 operator*(const Vector2& other) const {
        return Multiply(other);
    }

    bool operator==(const Vector2& other) const{
        return x == other.x && y == other.y;
    }

    bool operator!=(const Vector2& other) const {
        return !(*this == other);
    }
};

std::ostream& operator<<(std::ostream& stream, const Vector2& other) {
    stream << other.x << "," << other.y;
    return stream;
}

int main() {
    Vector2 position(4.0f, 4.0f);
    Vector2 speed(0.5f, 1.5f);
    Vector2 powerup(1.1f, 1.1f);

    Vector2 result_1 = position.Add(speed.Multiply(powerup));
    Vector2 result_2 = position + speed * powerup;

    if (result_1 == result_2) {
        cout << "yes" << endl;
    }
    if (result_1 != result_2) {
        cout << "no" << endl;
    }

    cin.get();
}

```


23 this关键字

2025年1月15日 19:17

C++中，在类的内部，this是一个指向当前对象实例的指针。

类似Java，只不过这里的this是一个指针。

```
class Entity {
public:
    int x, y;

    Entity(int x, int y) {
        this->x = x;
        this->y = y;
    }
};
```

比如当参数与成员变量的名字一模一样的时候，就需要这样来区分。

或者当我们需要在类中调用以当前类的对象为参数的非成员函数时，this也会派上用场：

```
class Entity {
public:
    int x, y;

    Entity(int x, int y) {
        this->x = x;
        this->y = y;
        PrintEntity(this);
    }
};

void PrintEntity(Entity* e) {
    //.....
}
```

还有一种情况可能会用到this，比如上一节在类中进行运算符重载的时候：

```
bool operator==(const Vector2& other) const{
    return x == other.x && y == other.y;
}

bool operator!=(const Vector2& other) const {
    return !(*this == other);
}
```

24 对象生存期（栈的作用域）

2025年1月16日 12:00

1、作用域：

任何有花括号出现的地方，就会有作用域。比如函数、if语句、for语句、while语句、类，甚至只是一对花括号，都存在作用域。

```
int main() {  
  
    {  
        Entity e;  
    }  
  
    cin.get();  
}
```

上面这个例子只是在main函数中加了一对花括号，这样就已经限制了e的生存期。当程序进行到花括号外面时，e就会被销毁。

在**栈**上创建的变量会在作用域之外自动被销毁。

```
int* CreateArray() {  
    int array[50];  
    return array;  
}
```

比如这段代码，看起来好像没什么问题，但是它是绝对错误的。因为array被创建在栈上，当函数结束后，它就会被销毁，此时返回的指针指向的内存已经被释放了，但是它仍然指向那片内存。

这个错误通常被称为“返回局部变量的引用或指针”错误，或者更具体地称为“**返回悬挂指针**（Returning a Dangling Pointer）”错误。悬挂指针是指向已经被释放或不再有效的内存区域的指针。也就是说，该指针曾经指向一个有效的内存对象，但这个对象已经被销毁或释放了，而指针本身没有被置为NULL或更新为指向其他有效对象。

如果我们没有意识到这一点，而继续通过这个悬挂指针访问或修改已释放的内存区域，这被称为**未定义行为**，是非法的内存访问操作。

2、作用域指针：

作用域指针是一个类，它是一个指针的包装器，在构造时用堆分配指针，然后在析构时删除指针，实现了自动化new和delete。如果我想要在堆上创建一个变量，但是我又想让它离开当前作用域时自动销毁，那么就需要使用作用域指针。

在C++标准库中封装了unique_ptr类，就是作用域指针。不过为了更清晰地了解它的作用原理，我们跟着教程手动写一个作用域指针。

```
#include <iostream>
using namespace std;

class Entity {
public:
    int x, y;

    Entity() {
        cout << "Created Entity!" << endl;
    }

    ~Entity() {
        cout << "Destroyed Entity!" << endl;
    }
};

class ScopedPtr {
private:
    Entity* m_Ptr;
public:
    ScopedPtr(Entity* ptr) : m_Ptr(ptr){
    }

    ~ScopedPtr() {
        delete m_Ptr;
    }
};

int main() {
    {
        ScopedPtr e = new Entity();
    }

    cin.get();
}
```

这段代码就实现了使用作用域指针管理Entity对象的操作。尽管Entity对象是在堆上创建的，但是离开当前花括号的作用域之后，它依然会被自动销毁。

ScopedPtr e = new Entity(); 这句代码似乎有点难懂：

怎么回事？这里的ScopedPtr的对象e不是在堆上创建的吗？它怎么能够做到出作用域自动销毁？

实际上，这句代码使用了我们之前学过的隐式转换，new操作符在堆上新建了Entity的对象，然后把这个对象作为参数传给了ScopedPtr的构造函数，在栈上创建e。这句代码等价于下面的写法：

```
ScopedPtr e(new Entity());
```

这样看起来就清晰了。

最后附上如果使用标准库中的unique_ptr该怎么写：

```
#include <iostream>
#include <memory> // 引入智能指针头文件
using namespace std;

class Entity {
public:
    int x, y;

    Entity() {
        cout << "Created Entity!" << endl;
    }

    ~Entity() {
        cout << "Destroyed Entity!" << endl;
    }
};

int main() {
    {
        // 使用std::unique_ptr创建并管理Entity对象
        unique_ptr<Entity> e = make_unique<Entity>();
    }

    cin.get();
}
```

25 智能指针

2025年1月16日 12:41

智能指针是一个统称，实际上有很多种智能指针。

1、作用域指针：

补充一点：为什么作用域指针叫做unique_ptr？

unique可以理解为独一无二的意思，事实上确实是这样，你不能复制一个unique_ptr，否则会有多个作用域指针指向同一片内存，当第一个作用域指针将这片内存释放之后，其他的作用域指针就变成了之前提到的**悬挂指针**，这是很危险的行为。

```
#include <iostream>
#include <memory>
using namespace std;

class Entity {
public:
    int x, y;

    Entity() {
        cout << "Created Entity!" << endl;
    }

    ~Entity() {
        cout << "Destroyed Entity!" << endl;
    }
};

int main() {

    {
        std::unique_ptr<Entity> e = make_unique<Entity>();
    }

    cin.get();
}
```

这是上节最后放出的如何使用作用域指针的代码。

标红的一行还可以这样写：

```
std::unique_ptr<Entity> e(new Entity);
```

但是不能这样写：

```
std::unique_ptr<Entity> e = new Entity();
```

也就是说，使用unique_ptr时不可以进行隐式转换，因为它的构造函数是被explicit修饰的。

不过最好的方法还是原始例子中的，使用make_unique方法。它更安全，可以保证构造函数出现异常时及时停止创建指针，从而防止内存泄漏。

作用域指针是最简单的智能指针，同时也是开销最小的，几乎没有开销。

2、共享指针：

共享指针的基本功能与作用域指针一样，但是它可以被复制，所以它叫shared_ptr。

共享指针内部会有一个引用计数，来统计指向当前内存的指针数，每有一个共享指针被销毁，引用计数就会减一，直到等于零，这时被引用的内存就会被释放。

```
#include <iostream>
#include <memory>
using namespace std;

class Entity {
public:
    int x, y;

    Entity() {
        cout << "Created Entity!" << endl;
    }

    ~Entity() {
        cout << "Destroyed Entity!" << endl;
    }
};

int main() {
    {
        std::shared_ptr<Entity> e0;
        {
            std::shared_ptr<Entity> e1 = std::make_shared<Entity>();
            e0 = e1;
        }
    }

    cin.get();
}
```

比如这个例子，我们可以将e1复制给e0，且在离开红色花括号的作用域时，Entity对象并没有被销毁，因为此时e0依然维持对它的引用，引用计数不为0，直到离开蓝色花括号的作用域时，Entity对象才被销毁。

值得一提的是：

```
std::shared_ptr<Entity> e1(new Entity);
```

你可以这样写，但是并不推荐。因为前面提到，创建共享指针时，除了为entity对象分配内存，还需要为引用计数的整数分配内存。这种写法会让两个分配行为分开进行，从而降低效率。make_shared函数把这两个行为统一在一起，效率更高。

弱指针：

弱指针是一种特殊的共享指针，它不参与引用计数。通常用于观察者模式，比如我需要创建一个Entity对象的列表，但是我不希望因为创建这个列表而导致引用计数增加，那么就可以使用弱指针。

```
int main() {  
    {  
        std::weak_ptr<Entity> e0;  
        {  
            std::shared_ptr<Entity> e1 = std::make_shared<Entity>();  
            e0 = e1;  
        }  
    }  
    cin.get();  
}
```

比如这段代码将e0换成了弱指针，这时程序运行到红色花括号外时，Entity对象就会被销毁。因为引用计数已经变成0了。此时弱指针指向了一片无效的内存，不过为了内存安全，弱指针提供了一些方法来判定指针是否有效：

```
if (auto lockedEntity = weakEntity.lock()) {  
    // 对象仍然存在，可以安全地使用  
    std::cout << "Entity is still alive!" << std::endl;  
} else {  
    // 对象已经被销毁  
    std::cout << "Entity has been destroyed." << std::endl;  
}
```

弱指针依赖于共享指针工作，它必须由共享指针复制，无法独立创建。

智能指针旨在实现自动化管理内存，以防你使用new之后忘记使用delete而造成内存泄漏。但是它也并非完全取代了new与delete，二者都有应用场景。

26 复制与拷贝构造函数

2025年1月16日 14:06

在C++中，复制无处不在。

```
int a = 1;
int b = a;
b = 2;
```

比如这段简单的代码，就实现了将a复制给b。因为b是a复制出来的，它们是两个不同的变量，只是值相同而已，所以对b的修改不会影响到a。

```
int* a = new int;
*a = 1;
int* b = a;

b += 2;
*b = 2;
```

而这种情况就有不同了。a, b都成了指针，此时b仍然是a的复制体，它们还是两个不同的变量。但是它们指向的是同一片内存。

b += 2; 这句代码仍然不会对a产生影响。但是我们很少会对指针做这种运算。

*b = 2; 但是这句代码就不同了，因为它修改了b指向的内存，而a也指向这一片内存。

这个例子想要说明的是，**任何把一个变量赋值给另一个变量的操作都是复制**。指针变量和非指针变量的区别导致结果稍显不同，但是复制的本质其实是一样的。

为了更好的演示，我们跟着教程先自己实现了string类：

```
#include <iostream>
#include <memory>
using namespace std;

class String {
private:
    char* m_Buffer;
    unsigned int m_Size;
public:
    String(const char* string) {
        m_Size = strlen(string);
        m_Buffer = new char[m_Size + 1];
        memcpy(m_Buffer, string, m_Size);
        m_Buffer[m_Size] = 0;
    }

    ~String() {
        delete[] m_Buffer;
    }
}
```



```

        friend std::ostream& operator<<(std::ostream& stream, const String&
string);
};

std::ostream& operator<<(std::ostream& stream, const String& string){
    stream << string.m_Buffer;
    return stream;
}

int main() {
    String string = "Hello, World!";
    cout << string << endl;
    cin.get();
}

```

接下来，我们要开始进行复制操作了：

```

int main() {
    String string1 = "Hello, World!";
    String string2 = string1;

    cout << string1 << endl;
    cout << string2 << endl;

    cin.get();
}

```

```

Hello, World!
Hello, World!

```

一切正常，我们打印出了两个相同的字符串。
但是当我按下回车准备结束程序时，程序遇到了错误。

```

void __CRTDECL operator delete(void* const block) noexcept
{
    #ifdef _DEBUG
        _free_dbg(block, _UNKNOWN_BLOCK);
    #else
        free(block);
    #endif
}

```

显然，delete这一步出了问题。

这是怎么造成的？很简单

```

char* m_Buffer;
unsigned int m_Size;

```

我们的string类本质上就是这两个变量的集合，当我复制string1的时候，就是把这两个变量

的值原封不动的复制到了另一片内存中，再把它给string2。

但问题是，m_Buffer变量是一个指针，这会导致两个字符串拥有值相同的指针，它们指向同一片内存。然后当两个字符串相继被销毁的时候，问题出现了：

```
~String() {  
    delete[] m_Buffer;  
}
```

我们在析构函数中使用了delete，这时程序就会尝试释放同一片内存两次，第二次自然就会崩溃。

通过调试时观察可以看到，它们两个的m_Buffer确实指向同一片内存。

```
▸ string1 {m_Buffer=0x00000232ec0c6010 "Hello, World!" m_Size=13 } -□  
▸ string2 {m_Buffer=0x00000232ec0c6010 "Hello, World!" m_Size=13 } -□
```

也就是说，此时对两个字符串中的任意一个进行任何修改，都会影响到另一个。这不是我们想要的，我们希望复制出来的字符串有自己的指针，有自己唯一的内存块。

事实上，我们刚才进行的操作称为**浅拷贝**，要实现我们的目的，我们需要进行**深拷贝**。

要实现深拷贝，我们就需要依靠**拷贝构造函数**。拷贝构造函数的函数签名是**对同样的类对象的常引用**。C++中的默认拷贝构造函数就是简单的内存复制，也就是浅拷贝：

```
String(const String& other) : m_Buffer(other.m_Buffer),  
m_Size(other.m_Size){  
}  
//或者  
String(const String& other){  
    memcpy(this, &other, sizeof(String));  
}
```

顺便一提，如果我们不希望某个类的对象可以被复制，那么就可以把拷贝构造函数删除（=delete），比如之前提到的unique_ptr就是这样做的。

回到正题，如果我们想要实现深拷贝，就要重写拷贝构造函数：

```
String(const String& other) : m_Size(other.m_Size){  
    m_Buffer = new char[m_Size + 1];  
    memcpy(m_Buffer, other.m_Buffer, m_Size + 1);  
}
```

好了，现在问题解决了。

最后补充一点，我在拷贝构造函数中打印一条消息来提醒我程序进行了拷贝操作，然后新建一个函数：

```
String(const String& other) : m_Size(other.m_Size){
```

```

        cout << "Copied String!" << endl;
        m_Buffer = new char[m_Size + 1];
        memcpy(m_Buffer, other.m_Buffer, m_Size + 1);
    }

    void PrintString(String string) {
        cout << string << endl;
    }

```

主函数如下：

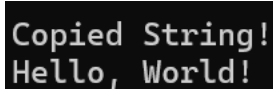
```

int main() {
    String string1 = "Hello, World!";

    PrintString(string1);

    cin.get();
}

```



```

Copied String!
Hello, World!

```

打印结果告诉我，程序进行了一次拷贝。显然是每次调用PrintString函数都会产生一份String的拷贝，这是非常浪费性能和内存的。所以我们最好使用常量引用：

```

void PrintString(const String& string) {
    cout << string << endl;
}

```

如果只是为了解决性能问题，只需要加上引用就可以了。const的作用是保证不在函数内修改传入的变量，由于传入的是引用，我们就必须考虑这一点。

其实很简单，**总是通过常量引用去传递对象**就可以了，因为你可以在函数中自行决定是否复制，而不是任由它到处滥用复制操作。

27 箭头操作符

2025年10月13日 8:11

1、基本用法：

```
#include <iostream>
#include <string>

class Entity {
public:
    void Print() const {
        std::cout << "Hello!" << std::endl;
    }
};

int main() {
    Entity e1;
    e1.Print();

    Entity* e2 = new Entity();
    (*e2).Print();
    e2->Print();

    std::cin.get();
}
```

上述代码中两行标红的部分实现了同样的功能，即调用e2的Print方法。
显然e1是在栈上创建的，e2是在堆上创建的，所以e1是实例本身，e2是指针。

所以**箭头操作符就是反向引用指针的快捷方式。**

2、重载：

众所周知C++的操作符是可以进行重载的。那么什么场景下我们需要重载箭头操作符呢？
——**当我们自己写智能指针类的时候。**

```
#include <iostream>
#include <string>

class Entity {
public:
    void Print() const {
        std::cout << "Hello!" << std::endl;
    }
};
```

```

class ScopedPtr {
private:
    Entity* m_Obj;
public:
    ScopedPtr(Entity* entity)
        : m_Obj(entity)
    {
    }

    ~ScopedPtr()
    {
        delete m_Obj;
    }

    Entity* GetObject() { return m_Obj; }
};

int main() {
    ScopedPtr entity = new Entity();
    entity.GetObject()->Print();

    std::cin.get();
}

```

上述代码实现了简单的智能指针，确保在堆上创建的Entity对象也可以出作用域自动销毁。但是此时如果需要使用箭头操作符进行反向引用，就会显得非常臃肿（如标红部分所示）。因此我们需要重载箭头操作符。

```

Entity* operator->() {
    return m_Obj;
}

```

我们只需将GetObject方法换成上述代码，就可以像这样调用类的方法：

```

ScopedPtr entity = new Entity();
entity->Print();

```

这样做的目的就是让代码保持干净易读。

3、获取内存中某个成员变量的偏移量（拓展内容）：

```

#include <iostream>
#include <string>

struct Vector3 {
    float x, y, z;
};

```

```
int main() {
    int offsetX = (int)&((Vector3*)0)->x;
    int offsetY = (int)&((Vector3*)0)->y;
    int offsetZ = (int)&((Vector3*)0)->z;

    std::cout << offsetX << std::endl;
    std::cout << offsetY << std::endl;
    std::cout << offsetZ << std::endl;

    std::cin.get();
}
```

上述代码先写了一个简单的三维向量结构体，它非常简单，只是将三个浮点数放在一块，不只是书面意义上的一块，他们在内存中也是紧挨着的，因此获取他们的内存偏移量在一些情况下是有意义的。

由于浮点数占用4字节的内存，上述程序运行后打印的结果如下图：



```
0
4
8
|
```

```
(int)&((Vector3*)0)->x;
```

这段代码有些难懂。我们来逐步拆解：

1. `(Vector3*)0`

- 这行代码将数字 0 强制转换 为一个指向 Vector3 结构体的指针。
- 换句话说，它创建了一个“假”的指针，指向内存地址 0。我们绝对不会去解引用这个指针（即访问地址0的数据），因为它是一个非法地址。我们只是利用它来进行地址计算。

2. `((Vector3*)0)->x`

- 通过这个假指针，我们使用 `->` 操作符来访问其成员 `x`。
- 同样，我们并不是要真正读取 `x` 的值（这会导致程序崩溃），我们只是为了定位到成员 `x`。

3. `&((Vector3*)0)->x`

- 这是关键的一步。我们使用取地址操作符 `&` 来获取成员 `x` 的地址。
- 既然我们的假结构体起始于地址 0，那么其成员 `x` 的地址就正好等于 `x` 在结构体内部的偏移量（以字节为单位）。
- 例如，如果 Vector3 在内存中是这样布局的：

- 地址 0: `x` (起始位置)
- 地址 4: `y` (假设 `float` 占4字节)
- 地址 8: `z`
- 那么 `&((Vector3*)0)->x` 计算出的地址就是 0。

4. `(int)&((Vector3*)0)->x`

- 最后，我们将计算出的地址（一个指针类型）强制转换 为 `int` 类型。
- 这样，我们就得到了一个整数值，它代表了成员 `x` 相对于结构体起始位置的字节偏移量。

至于为什么要获取内存中某个成员变量的偏移量？因为当我们进行图形编程、游戏引擎编程时，我们需要将数据序列化为一串字节流，然后处理字节流。这时我们是没有变量名等标识来让我们识别内存中的各个变量的，因此我们需要依赖内存偏移量来处理数据。

28 std::vector (动态数组)

2025年10月13日 9:13

1、C++标准库：

从这节内容开始，我们正式开始使用C++标准库的内容，首先就是动态数组。
不过在学习动态数组之前，我们需要先了解C++标准库。

C++标准库应该叫标准模板库，里面承载了许多**容器类型**，这些容器包含特定的数据。

之所以称之为标准模板库，因为它可以模板化任何东西，而容器内部包含的数据类型由程序员自己决定。

简单来说，模板可以帮助我们处理底层数据类型，我们不再需要自己编写数据结构（只是通常情况下，因为标准模板库并没有把运行速度放在第一位，如果需要更快速的运行，仍然不能只依赖标准库）。

2、Vector类：

C++标准库中，Vector类就是动态数组，并不是数学向量，这个命名会让很多初学者感到困惑，包括我自己。
其实它更应该叫ArrayList。

vector在创建的时候不需要声明大小，它的大小会随元素数量自动改变。

本节只学习vector的基本使用方法，参考代码如下：

```
#include <iostream>
#include <string>
#include <vector>

struct Vertex {
    float x, y, z;
};

std::ostream& operator<<(std::ostream& stream, const Vertex& vertex) { //<<操作符重载
    stream << vertex.x << "," << vertex.y << "," << vertex.z;
    return stream;
}

void Function(const std::vector<Vertex>& vertices) { //确保总是通过引用来传递vector类的参数，复制vector会消耗大量性能
}

int main() {
    std::vector<Vertex> vertices;
    vertices.push_back({ 1,2,3 }); //添加元素，使用了隐式转换
    vertices.push_back({ 4,5,6 });

    for (int i = 0; i < vertices.size(); i++) {
        std::cout << vertices[i] << std::endl;
    }

    for (const Vertex& v: vertices) { //强化for循环用法
        std::cout << v << std::endl;
    }

    vertices.erase(vertices.begin()+1); //删除元素，注意参数不能直接填索引的数字，需要一个迭代器，所以使用begin方法

    vertices.clear(); //清空数组

    std::cin.get();
}
```


29 std::vector的优化使用

2025年10月13日 10:54

1、vector的工作原理：

vector作为动态数组，可以使用push back方法随时添加新的元素。

当使用push back时，如果vector的容量不足，那么它会重新分配内存，即申请一块新的内存，需要足以容纳vector原有的内容和添加的新元素，然后从内存的旧位置复制~~复制~~到内存中的新位置，最后删除旧位置的内存。

频繁的复制操作会拖慢程序，所以我们有些时候需要进行优化使用。优化的策略也是围绕如何避免复制对象。

```
#include <iostream>
#include <string>
#include <vector>

struct Vertex {
    float x, y, z;

    Vertex(float x, float y, float z)
        :x(x), y(y), z(z)
    {
    }

    Vertex(const Vertex& vertex)
        :x(vertex.x),y(vertex.y),z(vertex.z)
    {
        std::cout << "Copied!" << std::endl;
    }
};

int main() {
    std::vector<Vertex> vertices;

    vertices.push_back(Vertex(1, 2, 3));
    vertices.push_back(Vertex(4, 5, 6));
    vertices.push_back(Vertex(7, 8, 9));

    std::cin.get();
}
```

我们可以通过拷贝构造函数了解复制操作具体是如何进行的。

上述代码运行后打印的结果是：

```
Copied!  
Copied!  
Copied!  
Copied!  
Copied!  
Copied!  
|
```

我们得到了6行Copied，为什么呢？

首先，我们在主函数的栈上创建了vertex对象，我们需要将它放到vector分配的内存中，这就已经需要一次复制了。由于这个原因，我们有3次复制（每个vertex对象都有一次）。

其次，**vector默认容量是1**，当我们添加第二个和第三个元素的时候，都进行了刚才提到的重新分配操作，总共两次，第一次需要复制1个元素，第二次需要复制2个元素，所以这里也有3次复制。

6次复制的来源搞清楚之后，我们就可以对症下药进行优化了。

2、优化操作：

想要优化上述代码，只需要修改主函数：

```
int main() {  
    std::vector<Vertex> vertices;  
  
    vertices.reserve(3);  
  
    vertices.emplace_back(1, 2, 3);  
    vertices.emplace_back(4, 5, 6);  
    vertices.emplace_back(7, 8, 9);  
  
    std::cin.get();  
}
```

首先，我们使用了reserve方法预先设定vector的大小，然后使用emplace back方法代替push back方法。

reserve方法让vector预先给3个元素留除了空间，所以直到添加第4个元素之前，它都不需要进行重新分配。

emplace back方法不再传递vertex对象作为参数，而是将构造函数的参数列表传递过去让vector自己在分配的内存里直接生成vertex对象，从而避免复制。

3、push back 与 emplace back 对比：

根据上述测试，我们为什么不全部使用emplace back呢？push back还有存在的意义吗？

1、在传递**左值（已经存在的变量）**的时候，二者在性能上没有区别，都需要进行复制。

```
vertices.emplace_back(Vertex(1, 2, 3));
```

比如这句使用emplace back的代码，和push back没有区别。

2、emplace back由于是完美转发参数，可能会**调用到一些意想不到的构造函数**，包括那些被 explicit 修饰的构造函数，此外还需要考虑**意外的隐式转换**。

3、push_back 充当了一个“安全哨兵”。它要求你先在外面构造好对象，这样如果构造过程有问题（比如无效参数），错误会在容器外部、更靠近代码的地方暴露出来，更容易调试。而 emplace_back 把构造过程隐藏在了容器内部，如果构造函数抛出异常，错误堆栈会更深，更难排查。

4、push back的语义更清晰，可读性更强。

总结，在以下情况下使用 emplace_back：

- 性能瓶颈：当你确实需要优化性能，并且能证明这里有性能开销时。
- 构造对象昂贵：当对象的构造（尤其是拷贝）成本很高时。
- 无法拷贝/移动的对象：当对象类型不允许拷贝或移动时，你必须使用 emplace_back 来构造。
- 传递多个构造参数：当需要传递多个参数来构造对象时，emplace_back 的语法更简洁。

30 如何使用库（静态）

2025年10月18日 20:11

1、简要介绍：

C++中没有所谓的第三方库管理器(package manager)，比如像python、java。

使用外部库的主要方式有两种：

1. 获取依赖库的源代码自行编译成静态或动态库；
2. 链接二进制文件（GLFW）；

我们首先学习第二种，使用二进制文件。

按照教程，我们先从GLFW网站下载我们需要的二进制文件：[An OpenGL library | GLFW](#)

Windows pre-compiled binaries

These packages contain the GLFW header files, [documentation](#) and release mode static libraries, DLLs and import libraries for Visual C++ 2013-2022 and MinGW-w64.

64-bit Windows binaries

32-bit Windows binaries

注意这里的64位和32位是相对于我们开发程序的目标平台而言的，和开发者自身使用的设备无关。

介绍一下静态库和动态库的区别，即静态链接和动态链接。

他们最大的区别就是是否被编译进exe文件中。**静态链接会把库直接放在exe文件里；而动态链接需要额外附加dll文件放在exe文件旁边，准备随时被调用。**

通常来说，静态链接更快，因为这意味着开发者事先知道自己要链接哪些函数，可以有选择地进行链接；而动态链接并不知道程序运行时会出现什么样的具体情况，所以必须要保证它的完整。

2、实际操作：

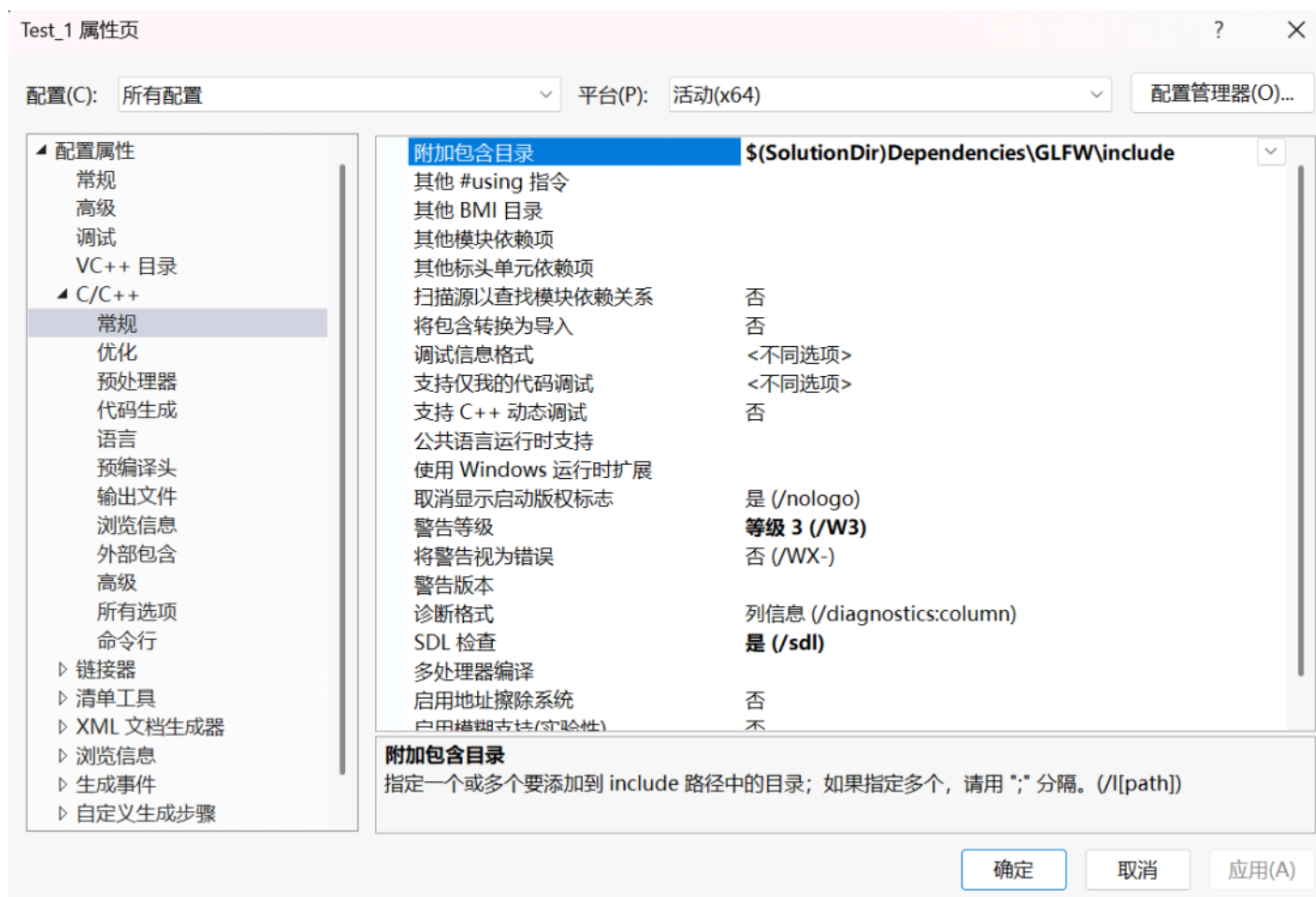
下载完后，我们现在项目文件夹里新建目录来放置我们下载的二进制文件。

地址为：...\Dependencies\GLFW\

把下载包里的include文件夹和对应版本的文件夹放在GLFW文件夹内

然后配置vs解决方案属性设置里的“附加包含目录”：

`$(SolutionDir)`是宏，表示解决方案所在目录地址，避免在不同设备上访问地址出错。



顺便介绍一下，根据使用的vs版本选择对应的文件夹，里面有4个文件：

 glfw3.dll	2024/2/23 23:03	应用程序扩展	207 KB
 glfw3.lib	2024/2/23 23:02	Object File Library	550 KB
 glfw3_mt.lib	2024/2/23 23:03	Object File Library	550 KB
 glfw3dll.lib	2024/2/23 23:03	Object File Library	32 KB

关于这四个文件的用法介绍如下:

1. 作为DLL使用（动态链接）

```
glfw3dll.lib + glfw3.dll
```

glfw3d11.lib 是唯一的导入库，专门用于动态链接。

glfw3.dll 是动态链接库本身。

配对使用：要使用动态链接，你必须链接 `glfw3dll.lib`，并且在运行时需要 `glfw3.dll`。

运行时库：这个DLL是用 多线程DLL运行时库 (/MD) 编译的。

2. 作为静态库使用（静态链接）

静态链接的选择取决于你的项目所使用的C++运行时库类型:

glfw3.lib

使用场景：当你的项目设置使用 多线程DLL运行时库 时使用。

Visual Studio 对应选项: /MD (Release) 或 /MDd (Debug)。这是更常见的设

置。

glfw3_mt.lib

使用场景：当你的项目设置使用 静态多线程运行时库 时使用。

Visual Studio 对应选项：/MT (Release) 或 /MTd (Debug)。

mt 含义：这里的 mt 明确指的是链接到静态的多线程运行时库。

回归正题，这一节我们学习的就是作为**静态库**使用的情况。结束上述操作之后，我们已经可以在编译器中引用头文件并且可以通过编译：

```
#include <iostream>
#include <GLFW/glfw3.h>
```

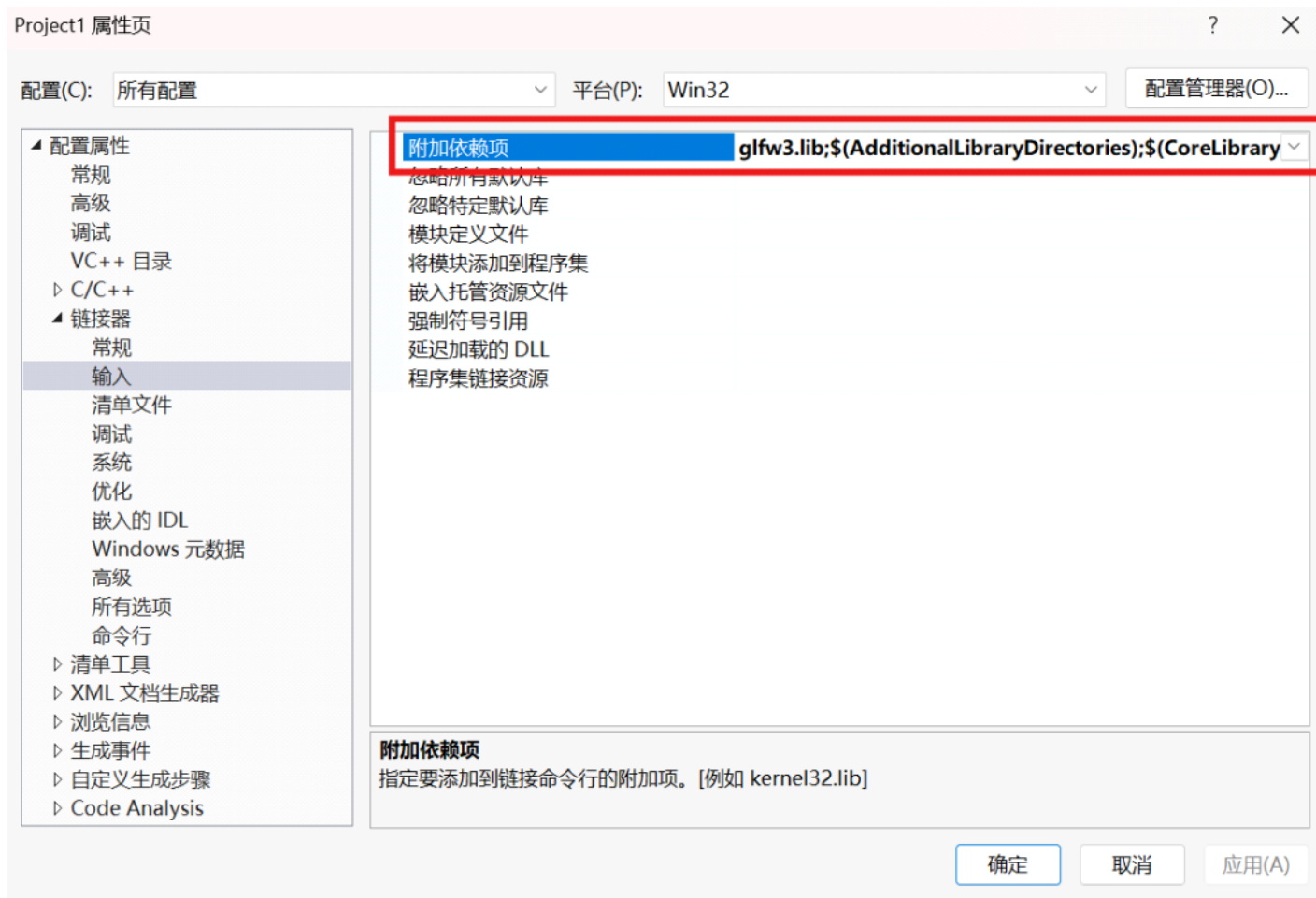
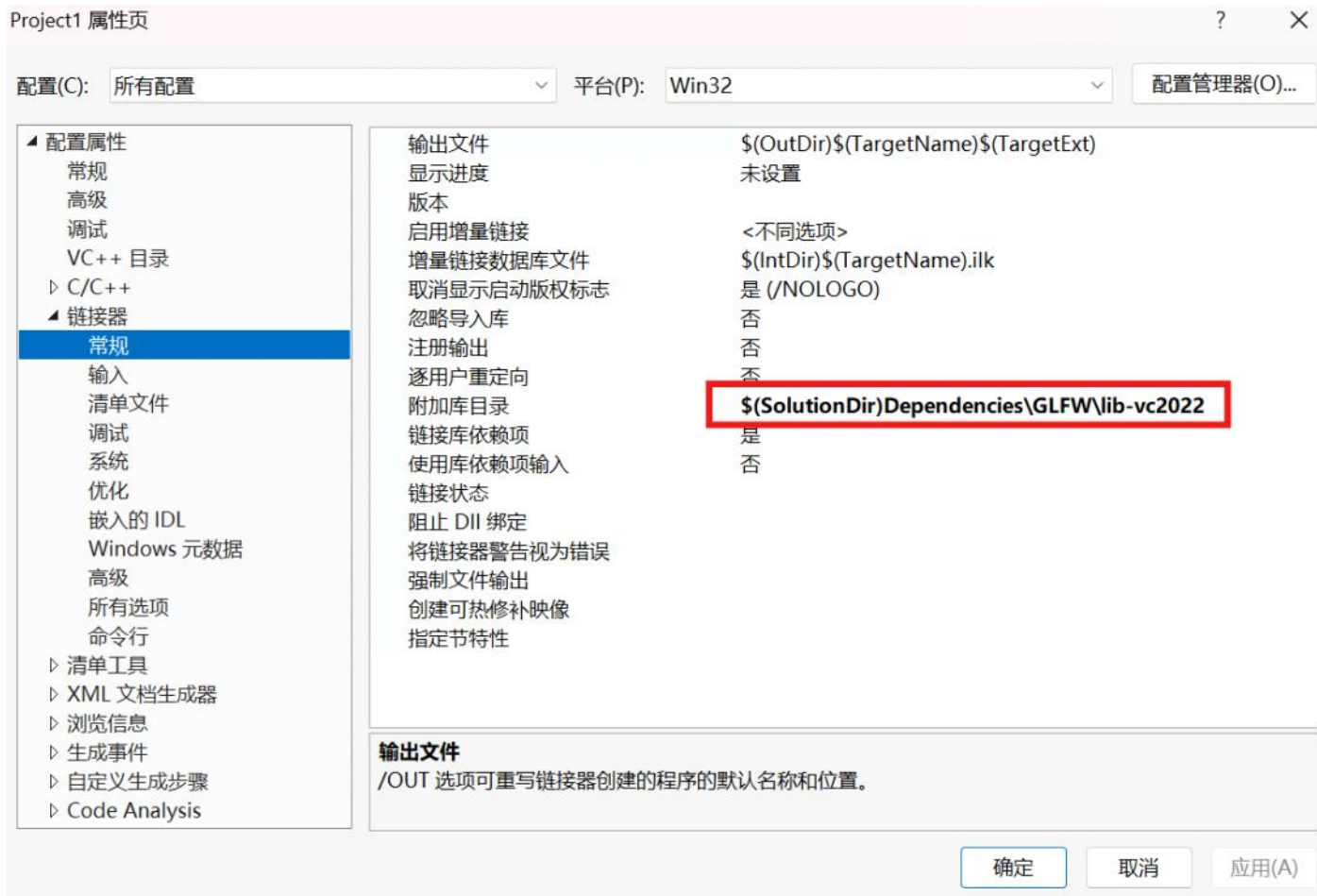
```
int main() {
    glfwInit();
    std::cin.get();
}
```

但是运行时却会出现错误：

	代码	说明
✖	LNK2019	无法解析的外部符号 glfwInit，函数 main 中引用了该符号
✖	LNK1121	1 个无法解析的外部命令

这是因为我们没有实际链接到相应的库中，我们只是获得了库内函数们的声明，而缺少了它们的定义。

接下来我们需要真正链接到库，修改如下设置：



最后额外介绍一下，由于头文件的作用只是声明函数而已，理论上我们可以这样写：


```

#include <iostream>
//#include <GLFW/glfw3.h>

int glfwInit();

int main() {
    int a = glfwInit();
    std::cout << a << std::endl;
    std::cin.get();
}

```

但是这样会报错，这并不是因为语法错误，而是glfw库是C语言库，与C++程序存在兼容问题。

正确写法应该是这样的：

```

#include <iostream>
//#include <GLFW/glfw3.h>

extern "C" int glfwInit();

int main() {
    int a = glfwInit();
    std::cout << a << std::endl;
    std::cin.get();
}

```


31 使用动态库

2025年10月23日 14:50

简单来说，动态链接的链接发生在程序运行的时候，而静态链接的链接发生在编译时。

然而动态库本身也有“静态”和“动态”之分。

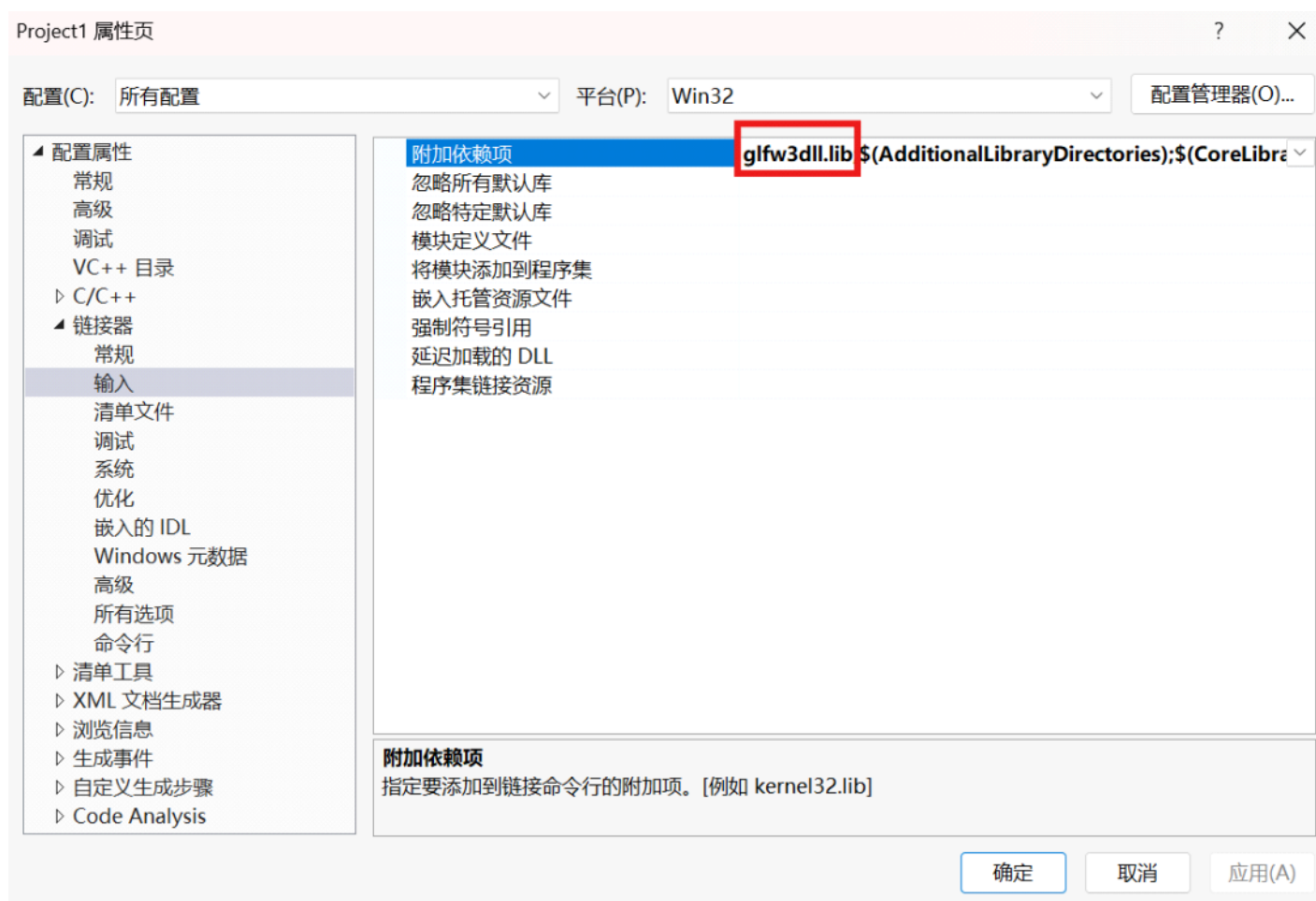
这时“静态”指的是程序运行时需要某个动态链接库，但是我事先已经知道里面的内容，包括函数、变量等等。相对的，“动态”指的就是程序运行时任意加载某个动态库，但是不知道里面有什么。我们这节课学习前者。

“静态”的动态库：

```
#include <iostream>
#include <GLFW/glfw3.h>
```

```
int main() {
    glfwInit();
    std::cin.get();
}
```

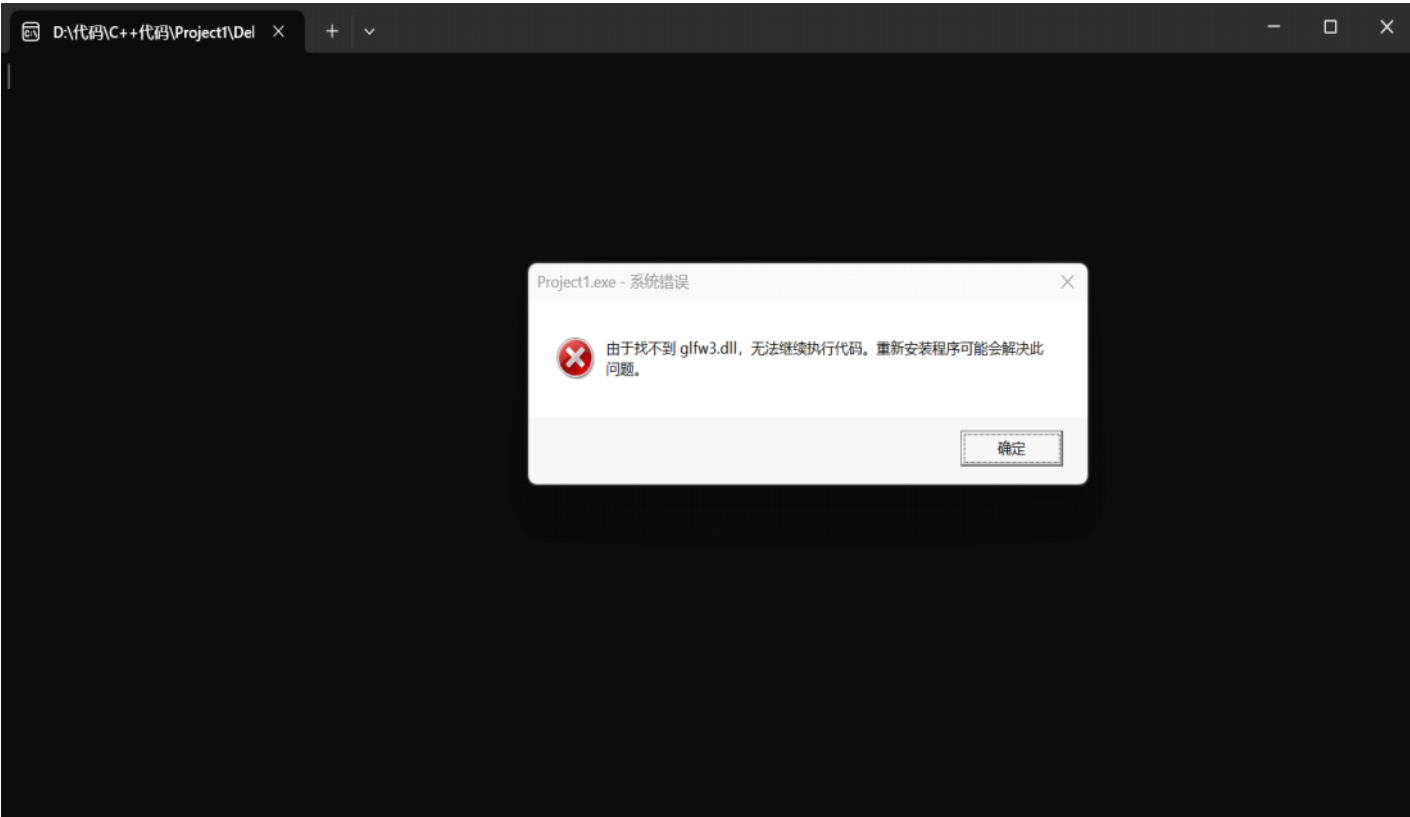
我们不需要修改上节课的代码，GLFW支持头文件静态和动态链接。



首先把依赖项glfw3.lib改成glfw3dll.lib。

- glfw3.lib就是静态库文件；
- glfw3.dll是对应的动态库文件，但是它必须和glfw3dll.lib联合使用；
- glfw3dll.lib是导入库，本质上也是一个静态库，它包含了一堆指向dll文件的指针；

设置完毕，此时编译构建完全没问题，但是运行时会出错：



显然程序找不到dll文件在哪，所以我们要把dll文件放到构建出的exe文件所在目录下，通常是Debug文件夹。

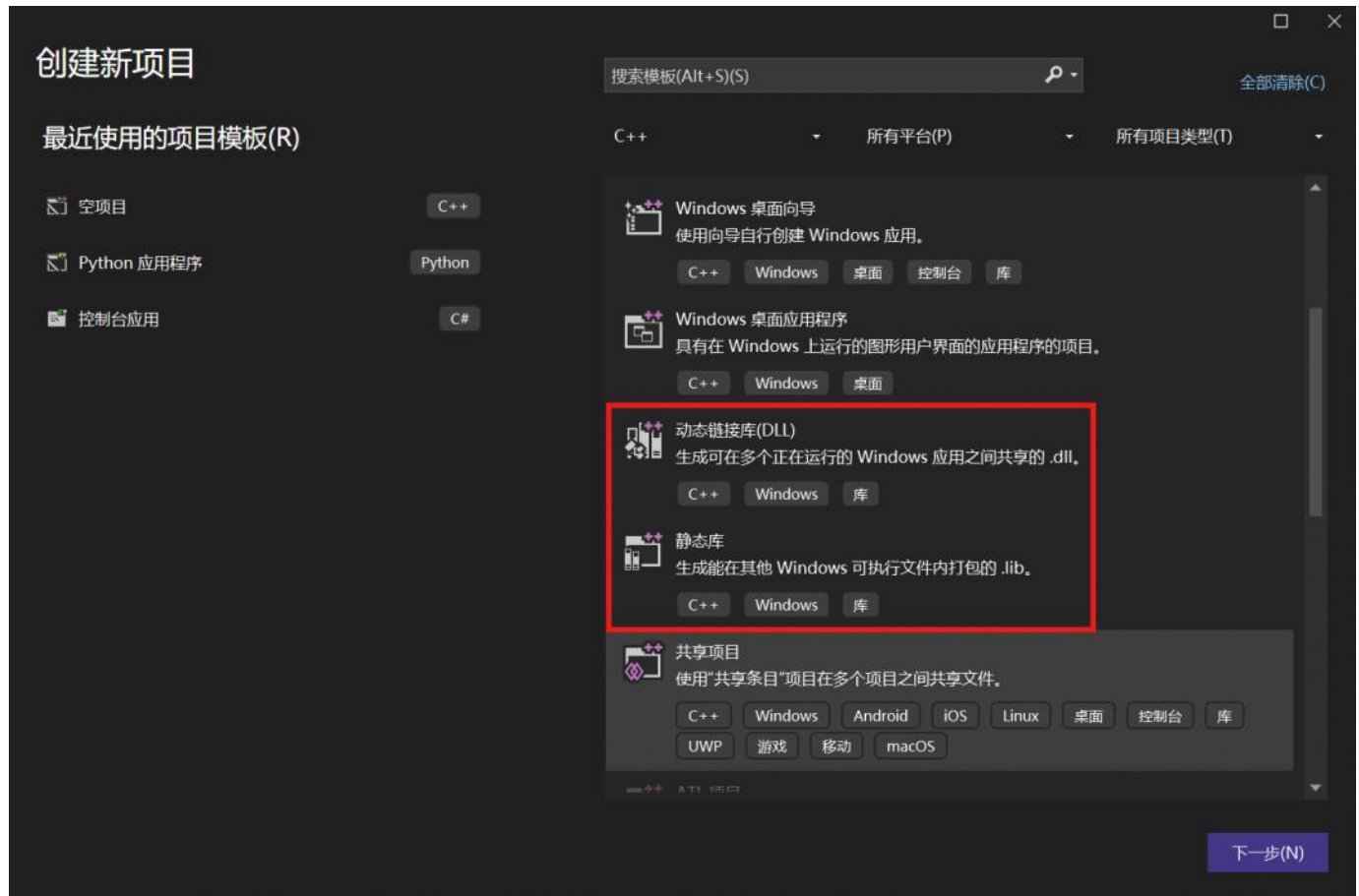
 glfw3.dll	2024/2/23 23:03	应用程序扩展	207 KB
 Project1.exe	2025/10/23 15:10	应用程序	41 KB
 Project1.pdb	2025/10/23 15:10	Program Debug Da...	1,820 KB

此时再运行程序就会得到跟上节课使用静态库完全一致的结果了。

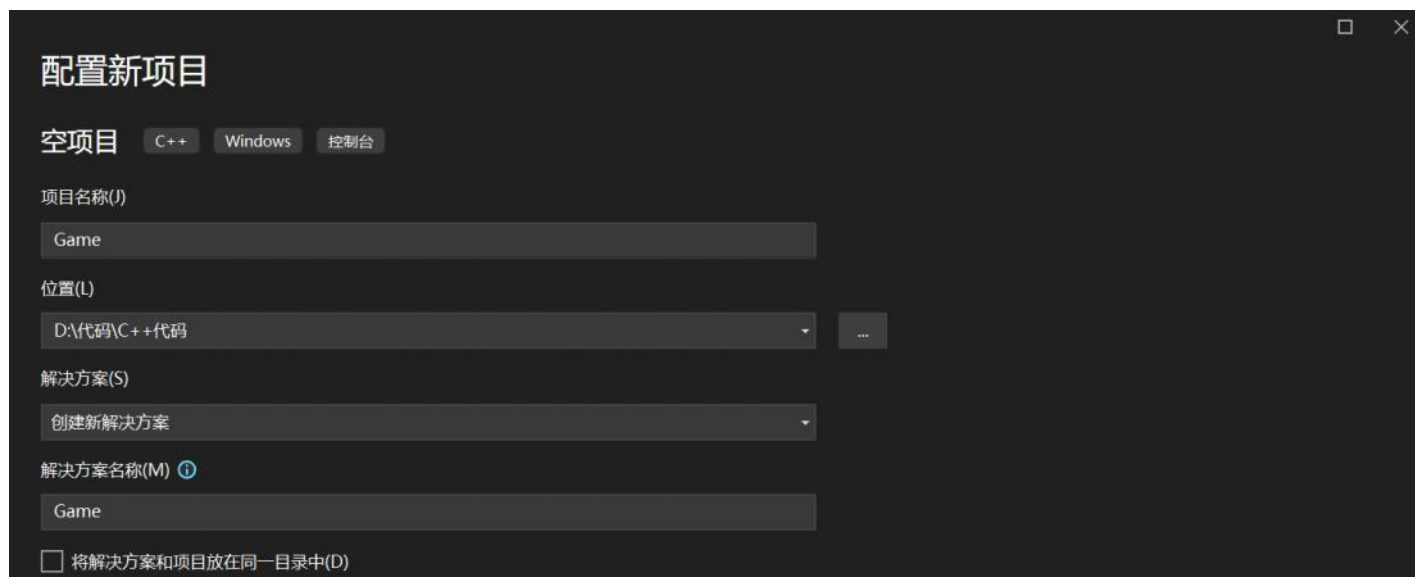
32 创建与使用库

2025年10月23日 15:26

这节课我们的目标是创建一个项目，把它作为库（特别是静态库），然后在别的项目中链接它。

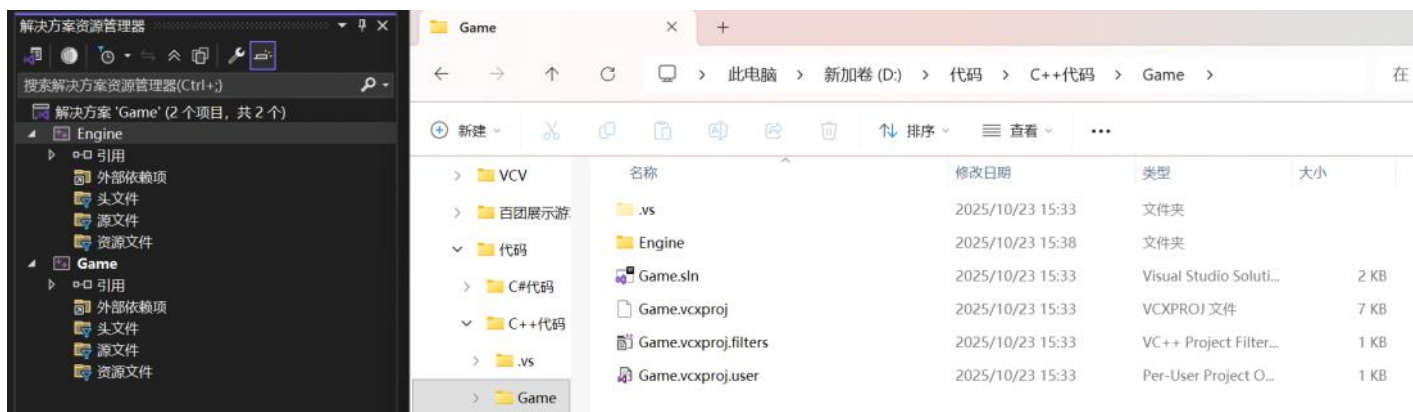


尽管vs中已经有现成的模板，我们还是跟着教学从空项目开始做起，更方便理解其中的原理。如果你的一个解决方案里需要存在多个项目，那么可以把“把解决方案和项目放在同一目录中”取消勾选，让你的项目架构更清晰：



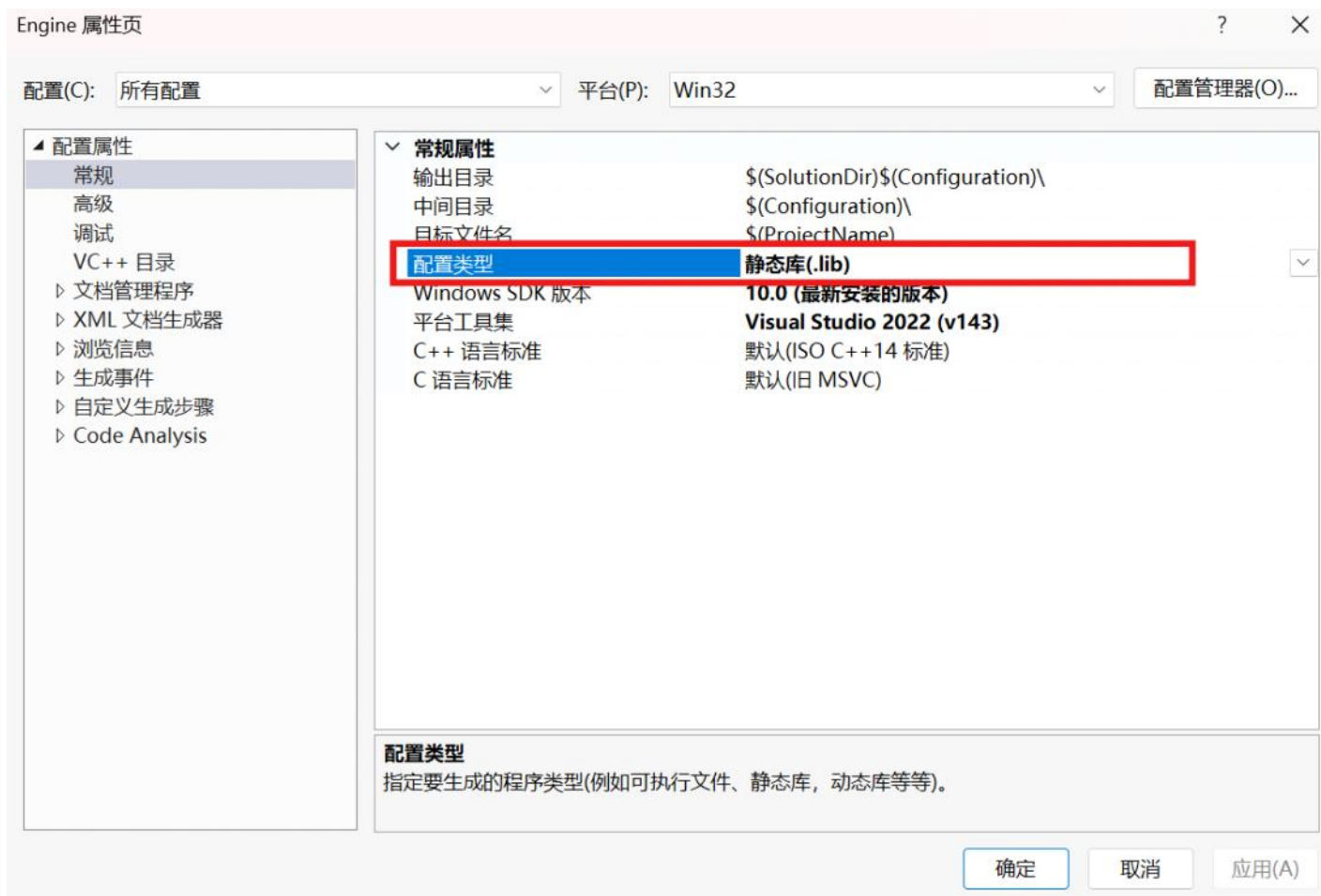


为了模拟真实游戏开发的架构，我们先新建一个Game项目与同名的解决方案，然后在Game解决方案里再新建一个Engine项目（在Game文件夹内）：



然后调整Engine项目的配置类型为静态库(.lib)。

由于Game项目的默认配置类型就是可执行文件(.exe)，所以不需要改动。



配置完成之后，分别在两个项目的src文件夹下新建相应的.cpp或.h文件，就可以正式开始写代码了：

Engine.h:

```
#pragma once

namespace engine {

    void PrintMessage();

}
```

Engine.cpp:

```
#include "Engine.h"
#include <iostream>

namespace engine {

    void PrintMessage() {
        std::cout << "Hello World!" << std::endl;
    }

}
```

Application.cpp:

```
#include "Engine.h"

int main() {
    engine::PrintMessage();
}
```

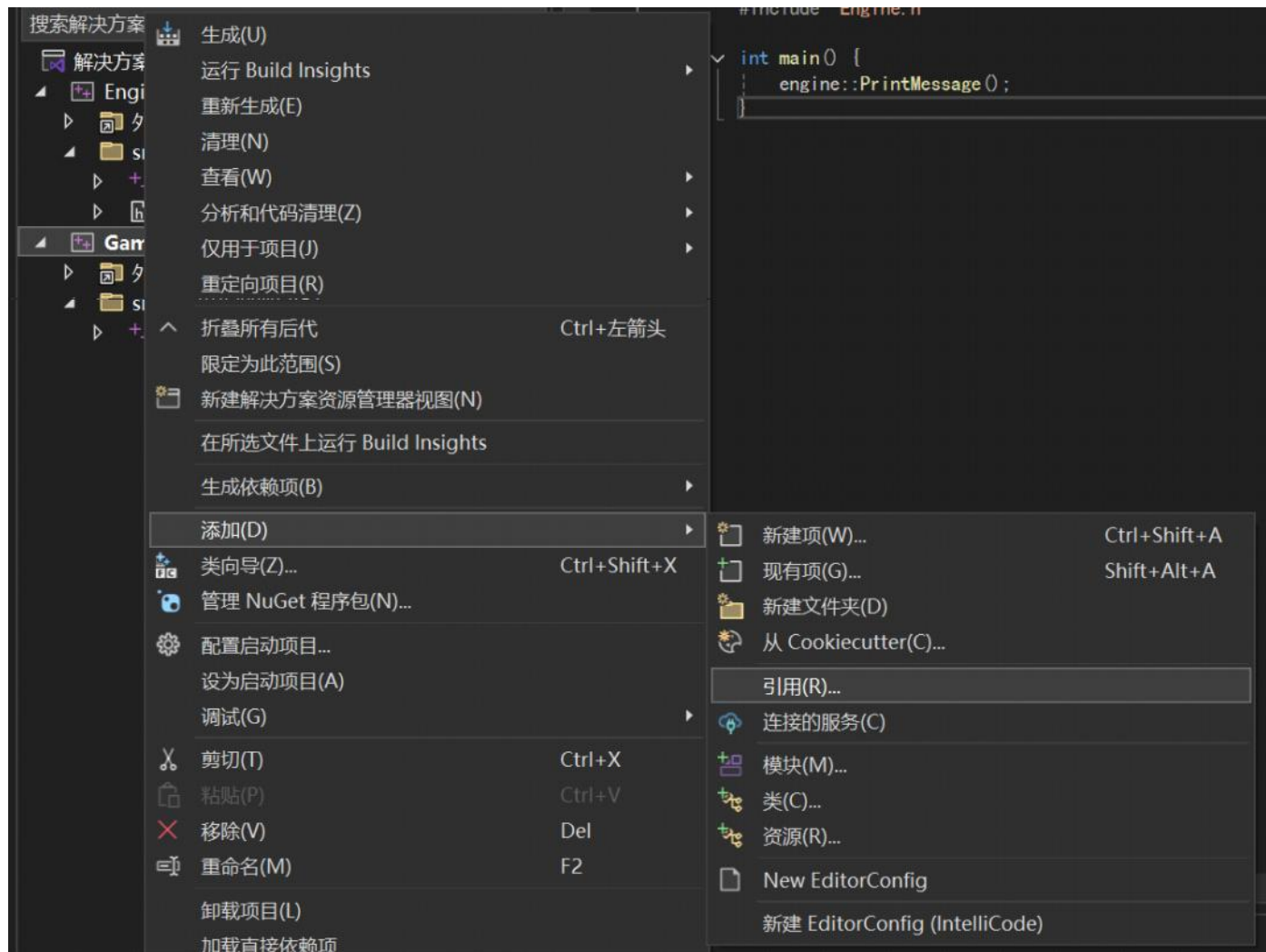
}

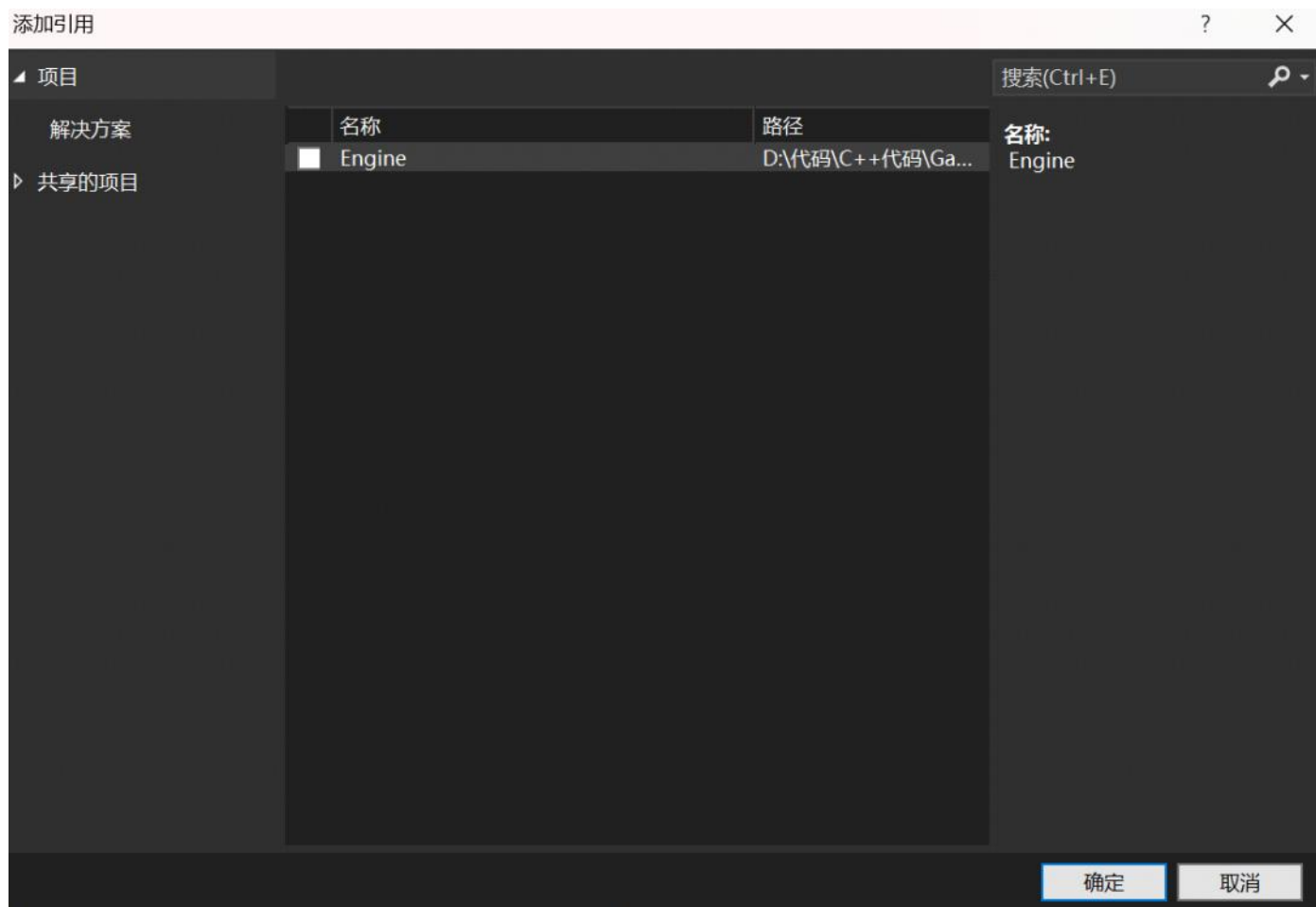
写完上述代码后，生成（CTRL+F7）Game项目，此时不会出现任何错误。

但是运行程序还是会出错。因为我们还没有链接到库，我们只是在Application.cpp中引用了头文件。

我们再去生成（CTRL+F7）Engine项目，会在总解决方案的Debug文件夹内找到生成的Engine.lib静态库文件。

然后我们就可以按照30节所学的方法实现静态链接——没错，不过我们不需要这么做，因为静态库项目在解决方案中，vs有更简便的办法。





直接为Game项目添加引用，勾选Engine项目。

这么做的好处还有一个，就是之后任何Engine项目的更新都会自动应用，我们不需要来回反复更改某些设置。

此时我们再运行程序，就没有任何问题了，PrintMessage函数被顺利调用了。

```
Hello World!  
|
```

补充：由于是静态链接，此时把Debug文件夹内的Game.exe文件单独拿到任何目录下，它都可以正常运行。

33 处理多返回值

2025年10月23日 17:17

由于原视频这一节跟作者另一个教程联动了，没同步的话学习体验不太好，我又从别的地方找了学习教程。

内容来源：[C++ - 函数返回多个返回值的方法总结 c++如何用return返回多个值-CSDN博客](#)

在C++中，标准语法只允许函数返回单个值。然而，有时候我们需要从函数中获取多个结果。虽然C++不支持像Python那样直接返回多个值，但我们可以使用一些技巧来实现类似的功能。

这里的多返回值特别指的是不同类型的返回值，因为同种类型多个值我们可以直接用如vector的容器来存放。

1、使用引用参数（不返回，直接在函数内修改原变量）：

```
#include <iostream>

void GetUserAge(const std::string& user_name, bool& work_status, int& age) {
    if (user_name.compare("xiaoli") == 0) {
        work_status = true;
        age = 18;
    }
    else {
        work_status = false;
        age = -1;
    }
}

int main() {
    bool work_status = false;
    int age = -1;
    GetUserAge("xiaoli", work_status, age);
    std::cout << "查询结果: " << work_status << "\t" << "年龄: " << age << std::endl;
    return 0;
}
```

通过传递引用参数给函数，我们可以在函数内部修改这些参数的值，从而实现返回多个结果的效果。比如这里的GetUserAge函数，在内部直接处理work_status和age。这种方法在语法上不如Python直观，但在功能上可以达到相同的目的。

2、使用std::pair：

```
#include <iostream>

std::pair<bool, int> GetUserAge(const std::string& user_name)
{
    std::pair<bool, int> result;
```



```

        if (user_name.compare("xiaoli") == 0){
            result = std::make_pair(true, 18);
        }
        else{
            result = std::make_pair(false, -1);
        }

        return result;
    }

int main()
{
    std::pair<bool, int> result = GetUserAge("xiaolili");
    std::cout << "查询结果: " << result.first << "    " << "年龄: " << result.second
    << std::endl;
    return 0;
}

```

使用std::pair抽象数据结构，该数据结构可以绑定两个异构成员。这种方式的弊端是只能返回两个值，而且只能用first,second关键字访问元素，可读性不强。

3、使用std::tuple:

```

#include <iostream>
#include <tuple>

std::tuple<bool, int,int> GetUserAge(const std::string& user_name)
{
    std::tuple<bool, int,int> result;

    if (user_name.compare("xiaoli") == 0){
        result = std::make_tuple(true, 18,0);
    }
    else{
        result = std::make_tuple(false, -1,-1);
    }

    return result;
}

int main()
{
    std::tuple<bool, int,int> result = GetUserAge("xiaolili");

    bool work_status;
    int age;
    int user_id;

    std::tie(work_status, age, user_id) = result;

    std::cout << "查询结果: " << work_status << "    " << "年龄: " << age << " "<< "用户
    id:"<<user_id <<std::endl;

    return 0;
}

```

`std::tuple`这种抽象数据结构可以将三个或者三个以上的异构成员绑定在一起，返回`std::tuple`作为函数返回值理论上可以返回三个或者三个以上的返回值。这段代码使用了`std::tie`直接将`tuple`中的元素赋值给变量，我们还可以使用`std::get`等操作，后续再详细学习。

4、使用结构体或类：

```
#include <iostream>

struct UserInfo {
    bool work_status;
    int age;
    int user_id;
};

UserInfo GetUserAge(const std::string& user_name) {
    UserInfo info;
    if (user_name.compare("xiaoli") == 0) {
        info = {true, 18, 0};
    }
    else {
        info = {false, -1, -1};
    }
    return info;
}

int main() {
    UserInfo result = GetUserAge("xiaoli");
    std::cout << "查询结果: " << result.work_status << "\t" << "年龄: " << result.age
    << "\t" << "用户id: " << result.user_id << std::endl;
    return 0;
}
```

定义一个结构体或类，其中包含多个成员变量，然后在函数中返回这个结构体或类的实例。这种方法使得返回的值更加结构化，我们直接可以用变量名访问变量，更容易理解和维护。

34 模板 (templates)

2025年10月27日 9:44

关于C++中的模板，如果你学过其他语言，你可以先把它看成“泛型”。（比如Java和C#）但是模板实际上要强大得多，它有些像宏，可以做许多事情；而泛型却非常受制于类型系统。

实际上，模板就是你给编译器一个规则，然后让它为你写代码。

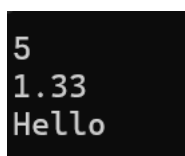
很多情况下，我们需要一个函数可以接收不同类型的参数，很多人会使用方法重载来实现。但是这样也有弊端，每次有一种新的需求出现，我们就要复制一份函数，再修改内容。通过模板，我们就可以只写一次函数代码，通过类似填空的形式实现这个功能。

```
#include <iostream>
#include<string>

template<typename T>
void Print(T value) {
    std::cout << value << std::endl;
}

int main() {
    Print(5);
    Print(1.33f);
    Print("Hello");

    std::cin.get();
}
```



```
5
1.33
Hello
```

可以看到，我们使用模板后，无需函数重载就可以打印多种类型的变量。

实际上，使用template的函数代码块并不是实际的代码，不是真的函数。只有当实际调用它时，函数才基于传递的参数被真正创建，然后作为源代码编译。这就是为什么说模板是让编译器自己写代码。

当然，如果我们永远不调用这个函数，那么它就相当于一块空白，就算内部有语法错误，也不会有任何影响。

（不同编译器报错情况不同，但可以肯定实际运行时都不会有影响）

其实上述代码中调用Print函数的写法不是完全规范的，它包含了隐式传递类型，完整的写法应该是：

```
Print<int>(5);
Print<float>(1.33f);
Print<std::string>("Hello");
```

其实模板在运行过程中实际上就是我们前面提到的方法重载，把模板代码复制一份，填上所需的类型。比如上面的代码在运行时就会创建三个Print函数。只不过这是编译器自己实现的。

模板绝不仅仅局限于类型或函数，它有非常广泛的用途。

```
#include <iostream>

template<typename T, int N>
class Array {
private:
    T m_Array[N];
public:
    int GetSize() const { return N; }
};

int main() {
    Array<int,5> array;
    std::cin.get();
}
```

这段代码使用模板实现了一个简单的Array类，实际上标准库Array的实现方式和这非常类似。

尽管模板非常有用，但是也不要过于依赖。模板最好的用处是简化代码，提高复用性，如果你发现你的模板变得越来越复杂庞大，那么就应该反思是不是做过头了。有时自己手写重载反而会让代码更清晰。

35 栈和堆内存的比较

2025年10月28日 14:02

当程序开始运行时，它被分成了一堆不同的内存区域，包括栈和堆，还有很多东西。不过我们最关心这两个。操作系统会将整个程序加载到内存，并分配物理Ram（随机存取存储器），栈和堆是ram中实际存在的两个区域。栈是一个预定义大小的内存区域，通常约为2MB（兆字节）左右。堆也是一个预定义了默认值的区域，但是它可以生长，并随着应用程序的进行而改变。

Ram是计算机中用于临时存放CPU正在使用或即将使用的数据和程序的地方。比如，你打开一个Word文档、运行一个游戏，它们都会被从速度较慢的硬盘加载到速度极快的RAM上进行处理。

栈和堆本质上的作用是相同的，它们都可以为程序分配内存，但它们给我们内存的方式是不同的。

```
#include <iostream>
#include <string>

struct Vector3 {
    float x, y, z;

    Vector3():x(10),y(11),z(12){}
};

int main() {
    //栈分配
    int value = 5;
    int array[5];
    array[0] = 1;
    array[1] = 2;
    array[2] = 3;
    array[3] = 4;
    array[4] = 5;
    Vector3 vector;

    //堆分配
    int* hvalue = new int;
    *hvalue = 5;
    int* harray = new int[5];
    harray[0] = 1;
    harray[1] = 2;
    harray[2] = 3;
    harray[3] = 4;
    harray[4] = 5;
    Vector3* hvector = new Vector3();

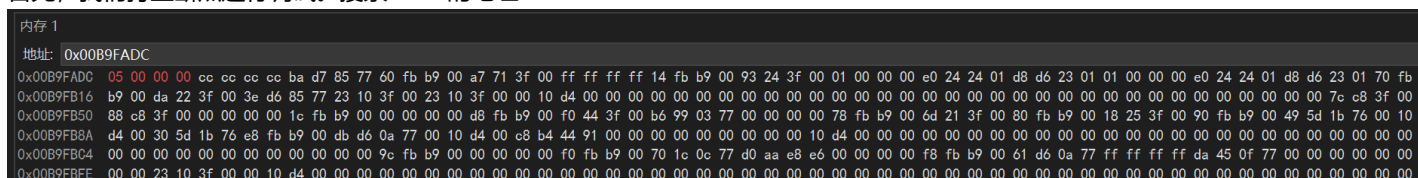
    delete hvalue;
    delete[] harray;
    delete hvector;

    std::cin.get();
}
```

我们通过这段简单的代码来看一下栈和堆在分配内存上到底有什么区别。

1、栈分配：

首先，我们打上断点进行调试。搜索value的地址：



内存 1
地址: 0x00B9FAC0
0x00B9FA00 01 00 00 00 02 00 00 03 00 00 00 04 00 00 00 05 00 00 00 cc cc cc cc cc cc cc cc 05 00 00 00 cc cc cc cc ba d7 85 77 60 fb b9 00 a7 71 3f 00 ff ff ff ff 14 fb b9 00 93 2d
0x00B9FAFA 3f 00 01 00 00 00 e0 24 24 01 d8 d6 23 01 01 00 00 00 e0 24 24 01 d8 d6 23 01 70 fb b9 00 da 22 3f 00 3e d6 85 77 23 10 3f 00 23 10 3f 00 00 10 d4 00 00 00 00 00 00 00 00
0x00B9FB34 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 7c c8 3f 00 88 c8 3f 00 00 00 00 1c fb b9 00 00 00 00 d8 fb b9 00 10 d4 3f 00 b6 99 03 77 00 00
0x00B9FBAE 00 00 78 fb b9 00 6d 21 3f 00 80 fb b9 00 18 25 3f 00 90 fb b9 00 49 5d 1b 76 00 10 d4 00 00 30 5d 1b 76 e8 fb b9 00 db d6 0a 77 00 10 d4 00 c8 b4 44 91 00 00 00 00 00 00 00 00
0x00B9FBA8 00 10 d4 00 f0 fb b9 00 70 1c 0c 77 d0 aa
0x00B9FBE2 c8 e6 00 00 00 00 f8 fb b9 00 61 d6 0a 77 ff ff ff ff da 45 0f 77 00 00 00 00 00 00 00 00 23 10 3f 00 00 10 d4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

为什么他们中间隔开了8个字节？这是调试模式下的安全守卫（safety guards）。在所有变量旁边添加一些空白区域。以确保不会溢出所有的变量。所以按理说，栈上分配的内存是紧挨着的。

[illegible]

实际上，栈分配的过程中，发生的就是栈指针的移动，它分配多少字节，就移动多少字节。这也是为什么栈分配那么快，它全程只做一件事，移动栈指针，然后返回栈指针的地址。

2、堆分配:

new关键字调用了—个叫做malloc的函数，memory allocate（内存分配）的缩写，我们在new关键字那节提到过。

这通常会调用底层操作系统或平台的特定函数，这会在堆上为你分配内存。我们在开始已经知道，在程序启动时，会得到—定数量的物理Ram。

不过还有，我们的程序会维护—个叫做free list（空闲列表）的东西，它会跟踪那些内存块是空闲的，然后当我们调用malloc函数时，它就会在free list寻找足够大的空闲内存，然后分配给你，之后还会进行复杂的记录（比如注明分配内存的大小，标记内存块已被分配等等）。

说了一堆，重点是要知道，**在堆上分配内存是很复杂的过程**，它不像栈分配那样简单如同—条CPU指令。

分区 ChernoのC++教程笔记 的第 94 页

36 宏

2025年10月28日 19:38

C++宏是由预处理器在编译前进行文本替换的代码片段。

上面是对宏的一句话定义，关于什么是预处理器，我们在第一节就谈到过。它实际上就是把我们写的代码检查一遍，然后执行某些查找、替换工作，再交给编译器进行编译。

是的，预处理器不涉及任何编译和运行，它本质上是纯文本处理器。

```
#include <iostream>
#include <string>

#define WAIT std::cin.get()

int main() {
    WAIT;
}
```

上述代码是宏使用的最简单的形式，甚至可以说有点蠢，我们绝不建议这样使用宏。因为这会降低代码可读性（尤其是宏定义在另一个文件的时候），而且没有任何收益。

但是这可以让我们看明白，宏就是文本替换，仅此而已。

另外，宏是可以传递参数的，这会有更实用的用法：

```
#include <iostream>
#include <string>

#define LOG(x) std::cout << x << std::endl

int main() {
    LOG("Hello");
    std::cin.get();
}
```

我们往往需要在调试时使用日志，而我們不希望在发布版本中出现日志信息，这也通常使用宏来实现。

```
#include <iostream>
#include <string>

#ifdef _DEBUG
#define LOG(x) std::cout << x << std::endl
#else
```

```

#define LOG(x)
#endif

int main() {
    LOG("Hello");
    std::cin.get();
}

```

其中_DEBUG是项目设置里的预处理器定义，可以自行设置，我使用的是VS2022的默认定义。

```

#if _DEBUG == 1
#define LOG(x) std::cout << x << std::endl
#elif defined(NDEBUG)
#define LOG(x)
#endif

```

通常这么写会更符合规范，我们可以在设置或直接在代码里初始化_DEBUG的值。

我们还可以使用反斜杠来写多行宏：

```

#include <iostream>

#define MAIN int main()\
{\
    std::cin.get();\
}

MAIN

```

"\"是回车键的转义，而且千万注意不要在它后面写任何东西，包括空格。不过这种写法非常少见且可读性差，仅作参考。

由于宏的一些固有缺陷，它在很多场景下都已经被现代C++的一些方案替代，不过也有其独特的应用场景：

应用场景	传统宏的用法	现代 C++ 的替代方案
定义常量	<code>#define MAX_SIZE 100</code>	<code>const</code> 或 <code>constexpr</code> ：提供类型安全，有明确的作用域，便于调试。
函数式宏	<code>#define SQUARE(x) ((x) * (x))</code>	<code>inline</code> 函数 / 函数模板 / <code>constexpr</code> 函数：避免因文本替换导致的参数多次求值等副作用，类型安全。
类型相关的代码生成	使用宏来定义结构体节点	模板：提供类型安全，更好的调试体验，并支持命名空间等C++特性。
条件编译	<code>#ifdef</code>	（暂无替代）这仍然是宏的核心且不可替代的用途。


```
DEBUG ... #endif
```

37 auto关键字

2025年10月28日 21:27

auto有些像某些语言的var，它让编译器自己推断变量的类型。

就这么一句话，auto的作用其实就这么简单。

```
#include <iostream>
#include <string>

std::string GetName() {
    return "LJJ";
}

int main() {
    auto name = GetName();
    std::cin.get();
}
```

此时auto自动推断出name的类型应该是std::string，如果我再把GetName的返回类型改成char*，auto也能立刻反应过来。也就是说我们只需要改动函数本身，调用函数的语句不需要做任何修改，这确实方便了不少。

现在我们需要面对一个问题，auto好像把C++变成了一个弱类型语言。既然这么方便，我是否可以在任何情况下使用auto，就不必再进行繁琐的类型判断了？

这也有弊端，使用auto意味着我们在后文也不知道这个变量究竟是什么类型。比如当我把GetName的返回类型改成char*后，给name赋值的语句没有任何问题，但是如果后续我调用size方法，就会发生错误。

auto会破坏依赖某种特定类型的语句，此时如果硬要记住变量的类型来编写后续代码，那么为什么不直接写明类型呢？这时auto就显得鸡肋了。

不过auto在一个场景下有百利而无一害的用法，就是“偷懒”。如果当一个类型实在是太长了，而且后续不需要对它做任何依赖类型的操作，那么就用auto吧。

```
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>

class Device{};

class DeviceManager{
private:
```

```

        std::unordered_map<std::string, std::vector<Device*>> m_Devices;
public:
    const std::unordered_map<std::string, std::vector<Device*>>&
    GetDevices() const {
        return m_Devices;
    }
};

int main() {
    DeviceManager dm;
    const std::unordered_map<std::string, std::vector<Device*>>& devices =
    dm.GetDevices();
    std::cin.get();
}

```

可以看到标红的类型是一个非常复杂非常长的类型，我们可以使用using或者typedef来简化，但更简单的方式就是使用auto：

```
const auto& devices = dm.GetDevices();
```

不过注意，我们依然需要使用const和&来避免复制，如果只写auto的话，虽然传递的是个引用，实际上变量还是会被复制一份。

如果当进入更加复杂的代码集，包含了模板等内容，我们就不得不使用auto了，因为那时我们也不知道类型会是什么。但是建议个人代码尽量避免写到这种地步，会极难维护。

38 std::array (标准数组)

2025年10月28日 22:03

C++标准库提供的array是静态数组，它的大小是不可变的。

```
#include <iostream>
#include <array>

int main() {
    std::array<int, 5> data;
    data[0] = 2;
    data[4] = 1;

    std::cin.get();
}
```

它看起来似乎和C风格的数组没什么区别，那我们就需要探究一下我们为什么要用它而不是原始数组。

其实最大的优势只有一个，那就是它知道自己的大小。

使用原始数组的时候我们必须额外维护一个int变量来告诉我们这个数组有多大，否则很容易出现越界访问的错误。而标准数组可以随时随地获取自己的大小。其他还有一些功能比如sort等等，但是不如这个原因重要。

另外，当你尝试越界访问的时候，标准数组会阻止你；而原始数组可能（取决于编译器）会放任你去修改不属于它的内存，导致引发无法预知的错误。

那么速度呢？这并不需要担心，只要使用得当，array的速度与原始数组没有差别。

内存大小？它多存储了一个表示大小的变量。实际上不是，它的大小在初始化的时候就由程序员传入，此后每次调用size函数时它只会返回那个数字（具体实现通过查看源码可以得知是使用了模板参数），这里没有变量。所以也不需要担心占用内存。

这节内容非常简单，就是尽可能地使用标准库提供的标准数组，它更安全易用。

39 原始函数指针

2025年11月1日 18:05

函数指针是将函数赋值给一个变量的方法。

到目前为止，我们仅仅尝试过调用一个函数，其实我们还可以将函数赋值给变量，或者将函数作为参数再传递给函数。此时如果不使用函数指针，代码就会变得非常复杂。

```
#include <iostream>

void HelloWorld() {
    std::cout << "Hello World!" << std::endl;
}

int main() {
    auto function = HelloWorld; //包含一次隐式转换，等效于: auto function =
    &HelloWorld;

    function();

    std::cin.get();
}
```

是的，之前学的auto关键字在这里使用会很方便。因为function的类型会非常奇怪，如果用鼠标悬停在上面可以看到它真正的类型：

```
int main() {
    void(*function)() = HelloWorld;

    function();

    std::cin.get();
}
```

这样写和之前用auto的那句代码是等效的，但会非常难懂。这种写法我们平时不用，仅作了解。我们通常直接使用auto或者搭配typedef使用。

```
void(*function)()
```

实际上，这里标红的部分才是类型；标蓝的部分是名字，可以随意更改。

```
#include <iostream>

void HelloWorld() {
    std::cout << "Hello World!" << std::endl;
}
```

```
int main() {
    typedef void(*HelloWorldFunction)();

    HelloWorldFunction function = HelloWorld;

    function();

    std::cin.get();
}
```

再复杂一点，可以接受参数的函数是怎样的：

```
#include <iostream>

void HelloWorld(int a) {
    std::cout << "Hello World!" << a << std::endl;
}

int main() {
    typedef void(*HelloWorldFunction)(int);

    HelloWorldFunction function = HelloWorld;

    function(8);

    std::cin.get();
}
```

那么，函数指针具体该怎么用，比如：

```
#include <iostream>
#include <vector>

void PrintValue(int value) {
    std::cout << "Value: " << value << std::endl;
}

void ForEach(const std::vector<int>& values, void(*func)(int)) {
    for (int value : values) {
        func(value);
    }
}

int main() {
    std::vector<int> values = { 1,2,3,4,5 };

    ForEach(values, PrintValue);

    std::cin.get();
}
```

我们想在某个函数中再多次使用某些操作，而且希望这些操作可以根据需要改变，就可以将一些操作写成函数，再使用函数指针，这样代码的灵活性和可读性都会变强。

比如这个例子里，我希望在遍历vector每个元素时打印它的值，func就是打印操作。如果我
希望将每个元素的值减一，那么我可以再写一个函数MinusOne，然后调用ForEach时传入
它的函数指针。

40 lambda匿名函数

2025年11月1日 18:47

1、匿名函数：

我们上节课学习了函数指针，将打印函数传给遍历函数。

不过我们往往不需要为如此简单的操作再额外写一个函数，比如打印，所以我们更多时候使用lambda匿名函数。

```
#include <iostream>
#include <vector>
#include <functional>

void ForEach(const std::vector<int>& values, const std::function<void(int)>& func) {
    for (int value : values) {
        func(value);
    }
}

int main() {
    std::vector<int> values = { 1,2,3,4,5 };

    auto lambda = [](int value) -> void {std::cout << "Value: " << value << std::endl; };

    ForEach(values,lambda);

    std::cin.get();
}
```

标红的一行实际上就是lambda表达式，可以发现它的内容实际上跟lambda这个单词没有关系，在C++中我们只是习惯用这个单词来命名匿名函数。

2、捕获列表：

匿名函数语法的重点就是中括号[]，它在这里叫“**捕获列表**”。

其实上述代码中的ForEach函数相比于上节课写的有所不同。func不再使用原始函数指针，而是使用标准库里function类型的引用，就是因为原始函数指针不兼容lambda捕获列表。

捕获列表允许匿名函数使用外部变量，比如：

```
int main() {
    int a = 1;

    int b = 2;

    auto lambda = [a,b]() -> int {return a+b; };

    std::cin.get();
}
```

这里是最简单的**值捕获**，即将变量值直接传入匿名函数。也可以用"="一次对所有变量采用**隐式值捕获**：


```
auto lambda = [=]() -> int {return a+b; };
```

对应的，还有**引用捕获**和**隐式引用捕获**：

```
auto lambda = [&a,&b]() -> int {return a+b; };
```

```
auto lambda = [&]() -> int {return a+b; };
```

隐式捕获和显式捕获可以混用，但不能两个都是值捕获或两个都是引用捕获：

```
auto lambda = [&,b]() -> int {return a+b; };
```

```
auto lambda = [=,&b]() -> int {return a+b; };
```

最后，还有**初始化捕获**：

```
auto lambda = [c = a + 5]() -> int {return c; };
```

补充一点，和普通函数不同，匿名函数中通过值捕获传入的变量，并不能在函数中进行修改，这似乎和引用捕获没有区别了。这时我们需要mutable关键字：

```
auto lambda = [a]() mutable -> int { a = 5; return a; };
```

3、使用示例：

附上教程里的使用示例，用匿名函数和algorithm库快速实现：查找并返回动态数组里第一个大于3的数。

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> values = { 1,2,3,4,5 };

    auto it = std::find_if(values.begin(), values.end(), [](int value) -> bool { return
    value > 3; });

    std::cout << *it << std::endl;

    std::cin.get();
}
```

这里的返回值是一个迭代器，所以it需要解引用。

41 using namespace std

2025年11月1日 19:31

这一节是博主专门为了回答一个问题而制作的：为什么他从来不用using namespace std？

我们都知道写了这句代码后，就不用每次都写一遍"std::"了，但是大佬选择不用，肯定有他的理由。

首先，每次都写一遍std::并不一定是坏事。它让我们清晰地知道代码中那些部分使用了标准库，哪些部分不是。别的第三方库中也许会有重名的变量或函数，甚至大型项目中，尤其是协作项目，自定义的一些类或函数会和标准库撞名。此时如果前面不加std::就会难以分辨，最终导致出错，而且难以排查。

其次，如果同时对两个不同的库using namespace，且两个库中有同名元素（变量或函数），这个元素变成了歧义符号，此时不加命名空间前缀直接使用，编译器并不会报错，但是运行起来，到底最终会调用哪个库中的元素，变成了未知数。这种错误也会非常致命且难以排查。

最后，也是最重要的一点，**永远不要在头文件内使用using namespace**。作者在提到前面的内容时可能还保有“见仁见智”的意味，到这一点后却变得十分笃定。

在头文件中使用using namespace意味着引用头文件即在整个程序作用域中使用using namespace，而且是跨文件使用。这种情况很容易发生错误，而且极难排查问题所在。

总结来说，如果真的要用using namespace，在尽量小的作用域中使用来简化代码。

42 名称空间

2025年11月1日 20:00

C++中的名称空间有什么作用？我们可以先回忆之前使用过的GLFW库。

它是一个C语言库，所以它没有名称空间，所以你可以发现它内部所有的函数名都以glfw开头，这正是为了防止与其他库的函数重名。没有名称空间，C语言别无选择，只能这样做。

这就反映出名称空间的主要作用，允许同一文件内出现同名符号（这里指的是类、函数、变量、常数等，同名也不只是名字相同，同时还有函数签名相同），也就是**避免命名冲突**。通过作用我们也可以反过来更容易理解它为什么叫“名称空间”了。

补充一点，类本身也是命名空间，我们访问类中的内部类，枚举数和静态成员的时候都需要使用“::”而不是“.”。

对名称空间，我们可以进行以下这些操作：

```
using namespace apple;

using apple::print;

namespace a = apple;
```

- 导入名称空间的所有内容；
- 导入名称空间的某个内容；
- 重命名名称空间；

名称空间也可以进行嵌套，嵌套时的导入操作如下：

```
namespace apple {
    namespace functions {
        void print() {}
    }
}
int main() {

    using namespace apple::functions;
    //或者
    using namespace apple;
    using namespace functions;

    print();

    std::cin.get();
}
```

最后再次强调最好不要在文件顶层使用using namespace，如果我们总是那么做的话，名称空间避免命名冲突的作用就完全失去了。

43 线程

2025年11月2日 22:21

到目前为止，课程中所写的代码全都是单线程。

比如当我们用std::cin.get()来等待用户输入时，程序什么都不做，就只是在那等着，好像进入了while true。

为了让程序等待时也能做些有用的事情——简单来说就是优化——我们就需要学习线程。

```
#include <iostream>
#include <thread>
#include <chrono>

static bool is_Finished = false;

void DoWork() {
    using namespace std::chrono_literals;

    std::cout << "Started thread id-" << std::this_thread::get_id()
    << std::endl;

    while (!is_Finished) {
        std::cout << "Working...\n";
        std::this_thread::sleep_for(1s);
    }
}

int main() {
    std::thread worker(DoWork);

    std::cin.get();
    is_Finished = true;

    worker.join();

    std::cout << "Finished." << std::endl;
    std::cout << "Started thread id-" << std::this_thread::get_id()
    << std::endl;

    std::cin.get();
}
```

这段代码实现了简单的多线程。程序可以一边等待用户输入，一边打印“working...”。其中使用到了thread库（线程相关）和chrono库（计时相关）。

代码主要流程：

1. 线程创建：主线程创建并启动工作线程
2. 并行执行：
 - 工作线程：循环打印“Working...”
 - 主线程：等待用户输入

3. 线程同步：用户按下回车后，主线程设置结束标志
4. 线程结束：工作线程检测到标志退出循环，主线程等待其结束
5. 资源清理：通过`join()`等待工作线程正常结束

44 计时

2025年11月2日 22:51

计时在C++中非常重要，无论是希望某些事情在特定时间发生，还是评估性能或基准测试，我们都需要计时。

chrono库就是专门计时的工具，我们上一节学习线程的时候已经初步接触了。

```
#include <iostream>
#include <chrono>
#include <thread>

struct Timer {
    std::chrono::time_point<std::chrono::steady_clock> start,end;
    std::chrono::duration<float> duration;

    Timer() {
        start = std::chrono::high_resolution_clock::now();
    }

    ~Timer()
    {
        end = std::chrono::high_resolution_clock::now();
        duration = end - start;

        std::cout << "Timer took " << duration.count()*1000.0f << "ms"
        << std::endl;
    }
};

void Function() {
    Timer timer;

    for (int i = 0; i < 100; i++) {
        std::cout << "Hello\n";
    }
}

int main() {
    Function();

    std::cin.get();
}
```

上述代码实现了一个简单的自动计时器，写法类似之前写过的智能指针，都是借助析构函数来实现的。

可以看到chrono内部的类型都比较复杂，不是想当然的返回float类型，所以我们通常会使用auto，不过定义变量的时候不能用auto。

顺带一提，std::endl会更耗时间，如果非常追求速度，那么换行时应该直接使用转义字符"\n"。

45 多维数组

2025年11月3日 8:11

我们从二维数组为例子开始介绍多维数组。

二维数组就是“数组的数组”。数组实际上是内存块，管理内存的简单方式是指针。

那么很显然，二维数组首先自身是一个指针，指向的内存块存储了一堆指针，分别指向内存中的其他数组。

```
#include <iostream>

int main() {
    int** array_2d = new int*[50];

    std::cin.get();
}
```

以原始数组为例。如果是别的语言，写这一句可能就没问题了，编译器会自动把数组内容填充好。

但是C++不行，此时我们只是申请了一系列内存，比如第一句就只是申请了50 * sizeof(int*)字节的内存，仅此而已，我们实际上还没有获得一个真正的数组。

关于int*（整型指针）的大小到底是多少：

指针本质上就是一个整数，它的大小是固定的，与类型无关。32位系统中为4字节，64位系统中为8字节。

```
#include <iostream>

int main() {
    int** array_2d = new int*[5];
    for (int i = 0; i < 5; i++) {
        array_2d[i] = new int[5];
    }

    int*** array_3d = new int**[5];
    for (int i = 0; i < 5; i++) {
        array_3d[i] = new int*[5];
        for (int j = 0; j < 5; j++) {
            array_3d[i][j] = new int[5];
        }
    }

    std::cin.get();
}
```

新建二维数组和三维数组的完整写法如上。

但是这还没完，因为我们是在堆上新建的数组，所以还需要delete。

但是多维数组不能一次delete就万事大吉了，C++并没有 delete[][] 这种写法，所以还需要遍历。

```
#include <iostream>

int main() {
    int** array_2d = new int*[5];
    for (int i = 0; i < 5; i++) {
        array_2d[i] = new int[5];
    }

    for (int i = 0; i < 5; i++) {
        delete[] array_2d[i];
    }
    delete[] array_2d;

    std::cin.get();
}
```

可以看到它有些类似处理输入输出流的代码，操作是“轴对称”的。

写法上介绍到这里就差不多了。不过还有一个问题：

这样处理多维数组会导致内存碎片的问题，因为刚才也提到了，二维数组里面存储了一系列指针，它们都指向各自的数组。这些数组的内存分配实际上是随机的，它们散落在堆内存各处，这在之前学习堆栈的时候也提到过。

因此，遍历5*5大小的二维数组要比遍历一个25大小的一维数组慢得多。但是我又需要使用二维数组，该怎么办？

```
#include <iostream>

int main() {
    int* array = new int[5 * 5];
    for (int y = 0; y < 5; y++) {
        for (int x = 0; x < 5; x++) {
            array[x + y * 5] = 1;
        }
    }

    std::cin.get();
}
```

教程给出的解决方法是使用抽象二维数组，用数学计算来模拟二维的访问方式，但是实际上我们使用一维数组储存，这样数组内所有的元素都储存在一个连续的内存块中。

46 排序

2025年11月3日 9:06

在C++中，我们当然可以自己写排序算法，冒泡排序、快速排序.....

但是通常我们没有必要自己写一个算法来进行排序，我们可以使用algorithm库里的std::sort。

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> values = { 3,5,1,4,2 };
    std::sort(values.begin(), values.end());

    for (int value : values) {
        std::cout << value << std::endl;
    }

    std::cin.get();
}
```

上述代码对一个整型数组进行了简单的排序。

下面我们介绍一下sort函数：sort(begin, end, cmp)

- begin：开始迭代器，也可以理解是指向待排序数组的第一个元素的指针；
- end：结束迭代器，同理可以理解是指向待排序数组最后一个元素的下一个位置的指针；
- cmp：排序准则，默认升序，可以使用内置预设，也可以自定义传入返回类型为bool的函数指针；
- 时间复杂度通常为 $O(N \cdot \log N)$

刚才的代码就是默认的升序，下面是使用预设的整数降序排序准则：

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> values = { 3,5,1,4,2 };
    std::sort(values.begin(), values.end(), std::greater<int>());

    for (int value : values) {
        std::cout << value << std::endl;
    }

    std::cin.get();
}
```

如何自定义排序准则：

- 接收两个同类型的参数, 如 int a, int b;
- 定义比较a, b的逻辑;
- 返回bool值, true表示a在b前, false反之;

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> values = { 3,5,1,4,2 };
    std::sort(values.begin(), values.end(), [](int a, int b) -> bool
        {
            return a < b;
        });

    for (int value : values) {
        std::cout << value << std::endl;
    }

    std::cin.get();
}
```

上述使用匿名函数实现了自定义升序准则。

47 类型双关

2025年11月3日 10:37

类型双关只是个花哨的术语，用来在C++中绕过类型系统。

作为强类型语言，C++当然有类型系统，但是它没有那么“强制”，比如Java和C#，绕过它们的类型系统需要做更多的工作。

C++中虽然类型是编译器强制执行的，但是我们可以直接访问内存，因此我们可以做很多微操，特别是处理字节流的时候很常用。

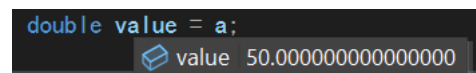
我们先把类型双关和类型转化做个区分：

```
#include <iostream>

int main() {
    int a = 50;
    double value = a;

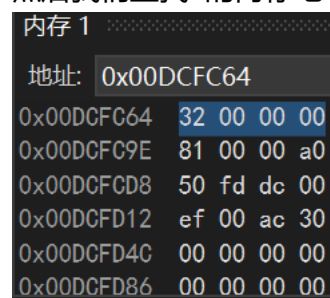
    std::cin.get();
}
```

这段代码是个很简单的隐式转换，此时value被赋值成double类型的50：



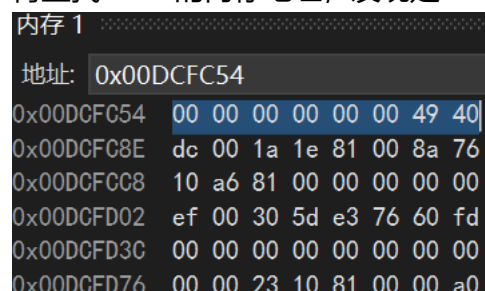
```
double value = a;
value 50.000000000000000
```

然后我们查找a的内存地址，正好是16进制的50：



```
内存 1
地址: 0x00DCFC64
0x00DCFC64 32 00 00 00
0x00DCFC9E 81 00 00 a0
0x00DCFCd8 50 fd dc 00
0x00DCFD12 ef 00 ac 30
0x00DCFD4C 00 00 00 00
0x00DCFD86 00 00 00 00
```

再查找value的内存地址，发现是49 40，这是因为double遵循 IEEE 754 标准：



```
内存 1
地址: 0x00DCFC54
0x00DCFC54 00 00 00 00 00 00 49 40
0x00DCFC8E dc 00 1a 1e 81 00 8a 76
0x00DCFCc8 10 a6 81 00 00 00 00 00
0x00DCFD02 ef 00 30 5d e3 76 60 fd
0x00DCFD3C 00 00 00 00 00 00 00 00
0x00DCFD76 00 00 23 10 81 00 00 a0
```

然后是类型双关：

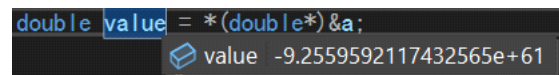
```
#include <iostream>

int main() {
    int a = 50;
    double value = *(double*)&a;

    std::cin.get();
}
```

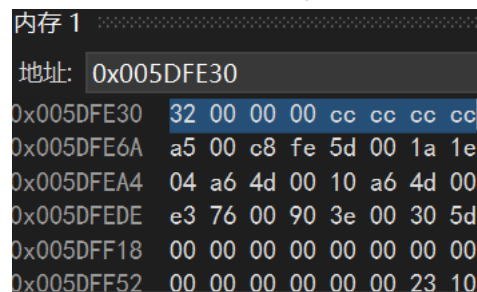
这段代码先取a的地址，我们得到一个int*，然后显式转换为double*，再解引用获得值，赋给value。

此时再进行调试，a的值当然不变，但是value的情况跟刚才不同了：



```
double value = *(double*)&a;
value -9.2559592117432565e+61
```

value的内存也跟刚才不同：



```
内存 1
地址: 0x005DFE30
0x005DFE30 32 00 00 00 cc cc cc cc
0x005DFE6A a5 00 c8 fe 5d 00 1a 1e
0x005DFEA4 04 a6 4d 00 10 a6 4d 00
0x005DFEDE e3 76 00 90 3e 00 30 5d
0x005DFF18 00 00 00 00 00 00 00 00
0x005DFF52 00 00 00 00 00 00 23 10
```

此时value几乎就是把a的内存照搬到了它这里，多余的4个字节就直接不初始化了。然后编译器依然按照double遵循的 IEEE 754 标准试图解读数据，于是value显示的值就变得如此奇怪了。

如果我们想直接对a进行类型双关，而不新建别的变量，那么可以使用引用类型：

```
double& value = *(double*)&a;
```

引用而不是拷贝成一个新的变量，这样我们就可以直接编辑a的内存。但是这样会有风险：a实际上还是一个4字节的整数，但是此时编译器认为它占用8个字节，如果我不小心对value进行赋值，那么它会额外覆写不属于a的4个字节。

```
#include <iostream>

struct Entity {
    int x, y;
};

int main() {
    Entity e = { 5, 8 };

    int* position = (int*)&e;

    std::cout << position[0] << ", " << position[1] << std::endl;
```

```
        std::cin.get();  
    }
```

这段代码把一个包含两个int变量的结构体类型双关成了一个int数组。因为结构体里的变量都是相邻存储的，当它们类型完全一致时，从内存来看与数组没有任何区别，所以我们可以这么做。

```
int y = e.y;  
  
int y = *(int*)((char*)&e + 4);
```

同理，我们也可以用这种奇怪的方式获取结构体的y变量，这种写法我们在之前学习指针的时候就接触过：取e的地址，显式转换成char类型的指针，然后向后移4个字节（1个整数所占的内存大小），再显式转换成int指针，最后解引用。

C++之所以是一个强大的语言，很重要的一点就是它可以自如操纵内存。尽管这大大提高了它的学习难度，但是作为回报，它可以把性能发挥到极致。

刚才的代码在实际情况中不可能出现，仅仅用来教学展示。我们再写一段更贴近现实的代码：

```
#include <iostream>  
  
struct Entity {  
    int x, y;  
  
    int* GetPositions() {  
        return &x;  
    }  
};  
  
int main() {  
    Entity e = { 5,8 };  
  
    int* position = e.GetPositions();  
  
    std::cin.get();  
}
```

我想把Entity对象类型双关成一个数组，毕竟它内部只有两个整数。

我们封装了一个函数来返回数组，但是我们并不需要新建一个数组、将x, y加入进去等等操作，那样太慢、太不优雅了。

我们只需要返回x的地址，作为一个整数指针。这很好理解——如果你认真看过之前数组那一节，知道数组本质上就是一个指针的话。再加上这个结构体又可以等效为数组，我们可以这样认为：

结构体（成员变量类型全部相同时） --> 数组 --> 指针

这样再看上面的代码就很好懂了。

总结一句，类型双关就是要把拥有的一段内存当作不同类型的内存来对待。

48 联合体 (Union)

2025年11月3日 12:34

联合体有点类似类或结构体，但是它一次只能占用一个成员的内存。

比如一个结构体包含四个整数，那么这个结构体应该占16个字节的内存。

但是如果一个联合体包含四个整数，它还是只占4个字节的内存，这些变量共用同一块内存，所以它们的值也是相等的。共用内存这个特点可能有点熟悉，有点像上节学习的类型双关。实际上，联合体的使用通常就伴随着类型双关。

如果我想给同一个变量取多个不同的名字时，联合体就很有用了。比如我希望有一个 `vector3(x,y,z)`，它同时也可以表示 `color(R,G,B)`，类似这样的情况。

通常 `union` 是匿名使用的，不过匿名联合体不能使用成员函数。

```
#include <iostream>

int main() {
    struct Union {
        union {
            float a;
            int b;
        };
    };

    Union u;
    u.a = 2.0f;
    std::cout << u.a << " " << u.b << std::endl;

    std::cin.get();
}
```

这段代码就在 `Union` 结构体内使用了匿名联合体，打印出的结果如下：

```
2 1073741824
```

可以看到，和类型双关那节举的 `double` 类型的例子一样，我通过 `a` 将这块内存赋值成了浮点数 2，而 `b` 试图将它读取成一个整数，所以它的值会变得很奇怪。这也说明它们确实是共用同一块内存。

现在，我们有两个类，二维向量和四维向量：

```
struct Vector2 {
```

```

        float x, y;
    };

    struct Vector4 {
        float x, y, z, w;

        Vector2& GetA() {
            return *(Vector2*)&x;
        }

        Vector2& GetB() {
            return *(Vector2*)&z;
        }
    };

```

我希望把四维向量看成两个二维向量的组合，封装了两个函数。这里的写法就不需要拷贝新的二维向量，更节省性能。但是我们还有更好的写法，使用联合体：

```

    struct Vector2 {
        float x, y;
    };

    struct Vector4 {
        union {
            struct {
                float x, y, z, w;
            };

            struct {
                Vector2 A, B;
            };
        };
    };

```

我们在Vector4内部使用了匿名联合体，然后在联合体里又写了两个匿名结构体，这两个结构体共用一块16字节的内存。其中一个和原来的Vector4一样，包含x, y, z, w四个浮点数，还有一个只包含两个Vector2。

这就相当于给四维向量定义了两种划分，一种是四个维度的变量，一种是看成两个二维向量，其中A正好包含x, y; B正好包含z, w。

完整测试代码如下：

```

#include <iostream>

struct Vector2 {
    float x, y;
};

struct Vector4 {
    union {

```



```

        struct
        {
            float x, y, z, w;
        };
        struct
        {
            Vector2 A, B;
        };
    };

};

void PrintVector2(const Vector2& vector) {
    std::cout << vector.x << ", " << vector.y << std::endl;
}

int main() {
    Vector4 vector = { 1.0f, 2.0f, 3.0f, 4.0f };
    PrintVector2(vector.A);
    PrintVector2(vector.B);

    std::cout << "-----" << std::endl;
    vector.z = 500.0f;

    PrintVector2(vector.A);
    PrintVector2(vector.B);

    std::cin.get();
}

```

```

1, 2
3, 4
-----
1, 2
500, 4

```

可以看到，完美地对应上了。我们只修改了四维向量里的z，但是它分解出的第二个二维向量的x也被改变了。

49 虚析构函数

2025年11月5日 18:20

虚析构函数可以理解为虚函数和析构函数的组合。这比较简单，但本质上虚析构函数对于处理多态和继承是非常重要的。

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "Base Constructor\n"; }
    ~Base() { std::cout << "Base Destructor\n"; }
};

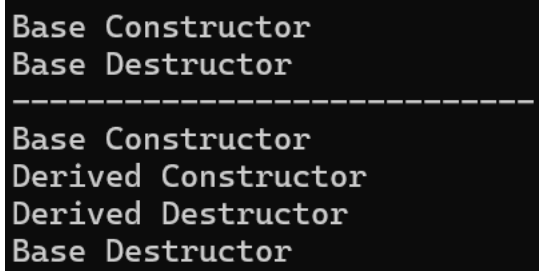
class Derived :public Base {
public:
    Derived(){ std::cout << "Derived Constructor\n"; }
    ~Derived() { std::cout << "Derived Destructor\n"; }
};

int main() {
    Base* base = new Base();
    delete base;
    std::cout << "-----\n";

    Derived* derived = new Derived();
    delete derived;

    std::cin.get();
}
```

首先是只有析构函数，没有虚函数的例子。



```
Base Constructor
Base Destructor
-----
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor
```

打印结果是这样的，创建和删除子类时，基类和本身的构造函数和析构函数都会被调用，一切正常。

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "Base Constructor\n"; }
    ~Base() { std::cout << "Base Destructor\n"; }
```

```
};

class Derived :public Base {
public:
    Derived(){ std::cout << "Derived Constructor\n"; }
    ~Derived() { std::cout << "Derived Destructor\n"; }
};

int main() {
    Base* base = new Base();
    delete base;
    std::cout << "-----\n";

    Derived* derived = new Derived();
    delete derived;
    std::cout << "-----\n";

    Base* poly = new Derived();
    delete poly;

    std::cin.get();
}
```

此时尝试新建一个子类实例，当作基类的指针来处理，打印结果就会出问题：

```
Base Constructor
Base Destructor
-----
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor
-----
Base Constructor
Derived Constructor
Base Destructor
```

可以看到，构造函数还是一样正常，但只有基类的析构函数被调用了，子类的析构函数没有被调用。这有可能导致内存泄漏。

由于poly被当作基类的指针，所以编译器在删除时就只调用了该类的析构函数。但如果此时我把基类的析构函数标记为virtual，也就是虚函数，就能解决问题。

这也许有点难懂，我们可以认为对于析构虚函数来说，它的意义在于“添加”，而不是普通虚函数的“覆写”。编译器看到析构函数是虚函数的时候，相当于被提醒“你应该去调用派生类的析构函数，如果有的话”。

```
#include <iostream>

class Base {
public:
```

```

    Base() { std::cout << "Base Constructor\n"; }
    virtual ~Base() { std::cout << "Base Destructor\n"; }
};

class Derived :public Base {
public:
    Derived(){ std::cout << "Derived Constructor\n"; }
    ~Derived() { std::cout << "Derived Destructor\n"; }
};

int main() {
    Base* base = new Base();
    delete base;
    std::cout << "-----\n";

    Derived* derived = new Derived();
    delete derived;
    std::cout << "-----\n";

    Base* poly = new Derived();
    delete poly;

    std::cin.get();
}

```

```

Base Constructor
Base Destructor
-----
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor
-----
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor

```

一般来说，如果你允许一个类拥有子类，那么你百分百需要声明析构函数为虚函数，为了安全拓展。

50 类型转换

2025年11月5日 18:46

类型转换有隐式转换和显式转换，我们这一节主要讨论显式。

```
#include <iostream>

int main() {
    double value = 5.3;
    int a = (int)value;

    std::cin.get();
}
```

我们之前熟悉的显式转换，像上面这样的，其实是C风格类型转换。当然我们还有C++风格的类型转换，这需要用到cast：

```
#include <iostream>

int main() {
    double value = 5.3;
    int a = static_cast<int>(value);

    std::cin.get();
}
```

除了static_cast，还有reinterpret_cast、dynamic_cast和const_cast。值得注意的是，它们相比较于C风格的类型转换，并没有添加新的功能。它们能实现的类型转化，C风格也能实现，它们只是提供了一些语法糖而已。

使用这些cast的一大好处就是：我们可以在代码库中搜索类型转换的代码都在哪里。我可以CTRL+F搜索static_cast，但是如果是C风格的类型转换，我们就没办法有效搜索。

还有一点就是，它们增加了编译时的检查，可以提前警告不兼容的类型转换，保证安全性。

最后介绍一下四种cast的用法和它们的区别：

1. static_cast

- 用途：用于非多态类型的转换，通常用于基本数据类型之间的转换（如int转double），以及具有继承关系的指针或引用之间的转换（向上转换是安全的，向下转换不安全但允许）。
- 特点：在编译时进行类型检查，但不能用于转换掉const、volatile或__unaligned属性。

2. dynamic_cast

- 用途：主要用于处理多态类型的转换，通常用于在继承体系中进行安全的向下转换

（即从基类指针或引用转换为派生类指针或引用）。它还可以用于交叉转换（`cross cast`）。

- 特点：在运行时检查类型转换的安全性。如果转换是指针类型并且失败，则返回空指针；如果是引用类型并且失败，则抛出`std::bad_cast`异常。
- 注意：使用`dynamic_cast`需要基类至少有一个虚函数（即多态类型）。

3. `const_cast`

- 用途：用于修改类型的`const`或`volatile`属性。常用于去掉`const`属性，但也可以加上。注意，去掉`const`属性后修改原为常量的对象是未定义行为，除非原对象本身不是常量。
- 特点：可以改变对象的常量性，但不能改变对象的类型。

4. `reinterpret_cast`

- 用途：用于进行低级别/types转换，通常用于将指针转换为整数，或将整数转换为指针，或者在不同类型的指针之间进行转换，甚至在不相关的类型之间进行转换。
- 特点：转换时不做任何类型检查，因此非常不安全。它通常用于底层的编程，如设备驱动或序列化等。

总结：

- `static_cast`：用于相关类型之间的转换，编译时检查，相对安全。
- `dynamic_cast`：用于多态类型的向下转换，运行时检查，安全但有一定开销。
- `const_cast`：用于修改`const`或`volatile`属性，谨慎使用。
- `reinterpret_cast`：用于不相关类型之间的转换，不安全，应尽量避免。

51 预编译头文件 (PCH)

2025年11月5日 19:15

1、为什么使用PCH:

预编译头文件 (Precompiled Header, PCH) 实际上是让你抓取一堆头文件, 并将它们转换成编译器可以使用的格式, 而不必一遍又一遍地读取这些头文件。

我们都知道引用头文件实际上就是复制粘贴, 而我们一般使用的标准库动辄上万行代码, include以后, 每次我们修改自己的代码后再重新编译, 会连带头文件内的所有代码一起重新解析和编译, 也许你只是多加了一行注释或空格。而且如果在不同的cpp文件中都引用同一个头文件, 它们也是独立编译的, 也就是说这个头文件会被重复编译若干次。

所以为了节约时间, 我们可以使用预编译头文件。它只编译一次你需要用到的头文件, 以二进制格式储存, 以便编译器能够快速处理。这能大大提高编译速度, 特别是对于大型项目来说。

2、为什么不使用PCH:

不要滥用预编译头文件, 比如需要频繁修改的内容, 是绝对不建议放入预编译头文件的, 每次修改后, 预编译头文件也要重新构建, 这就毫无意义了。它**主要用于那些不是你写的代码**, 如标准库, Windows库等等。

PCH还有一个缺点: 隐藏依赖关系。如果所有的头文件都放入PCH中, 那么我们实际在cpp文件中就看不到这个文件实际引入了哪些依赖, 如果不仔细的读代码, 我们无法确定它使用了PCH中的具体哪一个头文件。这在模块化和代码重用方面会造成困难。

所以也不是只要不用动的代码统统都放进PCH, 如果一个头文件只有少数某几个cpp文件需要使用的话, 建议直接include。

3、使用示例:

pch.h:

```
#pragma once
#include <iostream>
#include <algorithm>
#include <functional>
#include <memory>
#include <thread>
#include <utility>

#include <string>
#include <stack>
#include <deque>
#include <array>
#include <vector>
#include <set>
#include <map>
#include <unordered_set>
#include <unordered_map>

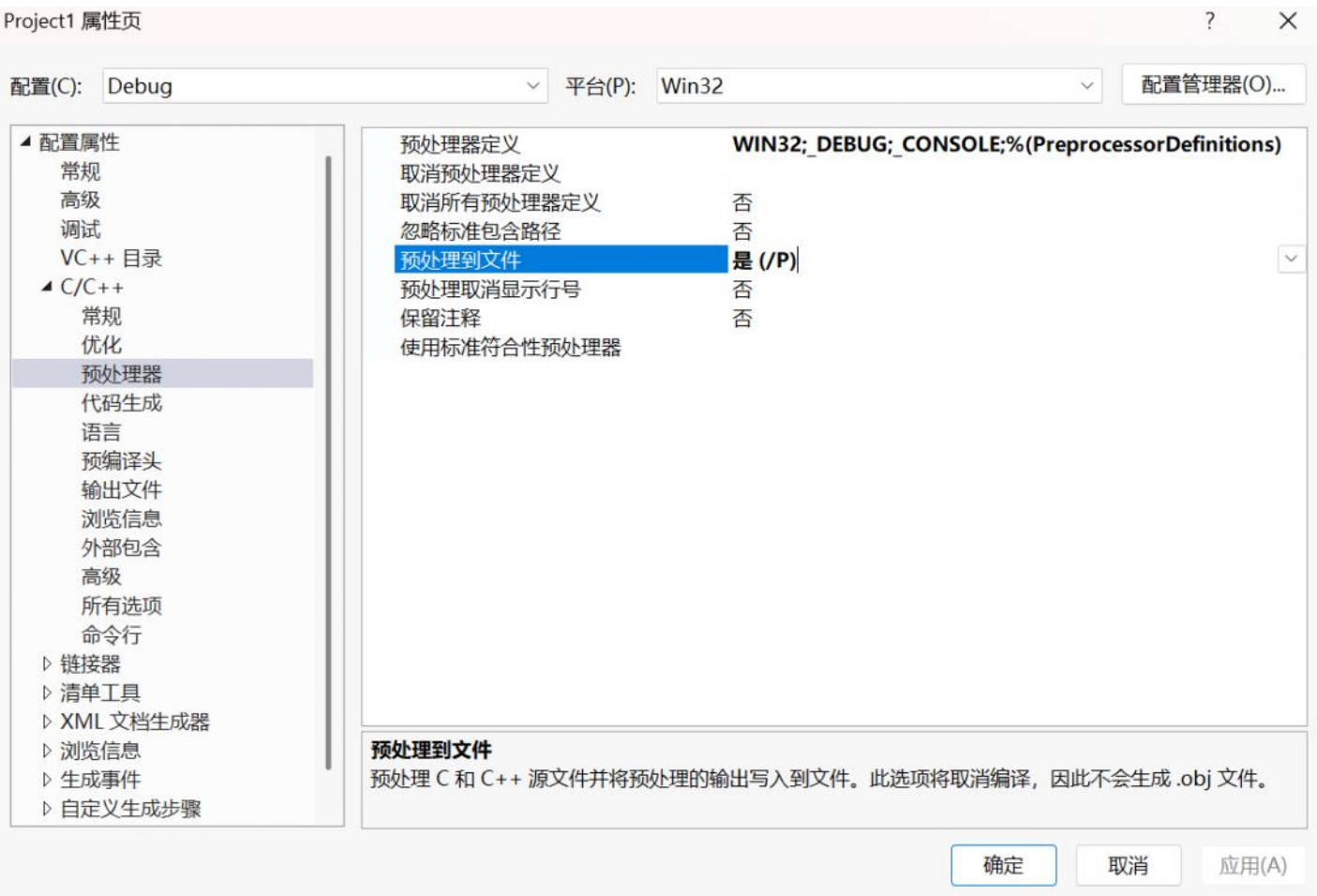
#include <Windows.h>
```

```
main.cpp:
#include "pch.h"

int main() {
    std::cout << "Hello World" << std::endl;
}
```

我们自己写了一个头文件pch.h，里面引用了一大堆头文件，当然此时它还只是个普通的头文件。然后我们在main.cpp文件里引用它，然后写上一句简单的hello world。

之后我们在属性里将**预处理到文件**选择是，CTRL+F7构建一下（后面了解完.i文件之后别忘了改回否）：



勾选这个选项会让编译器在构建时生成一个.i文件，把实际的代码储存在里面。

Project1.tlog	2025/11/5 21:12	文件夹	
main.i	2025/11/5 21:12	Preprocessed C/C+...	6,456 KB
main.obj	2025/11/5 19:21	OBJ 文件	56 KB
Project1.exe.recipe	2025/11/5 19:21	RECIPE 文件	1 KB
Project1.ilink	2025/11/5 19:21	Incremental Linker ...	557 KB
Project1.log	2025/11/5 21:12	文本文档	1 KB
vc143.idb	2025/11/5 19:21	VC++ Minimum Re...	227 KB
vc143.pdb	2025/11/5 19:21	Program Debug Da...	492 KB


```

428010     #pragma external_header(pop)
428011     #line 20 "D:\\代码\\C++代码\\Project1\\pch.h"
428012     #line 2 "D:\\代码\\C++代码\\Project1\\main.cpp"
428013
428014     int main() {
428015         std::cout << "Hello World" << std::endl;
428016     }
428017

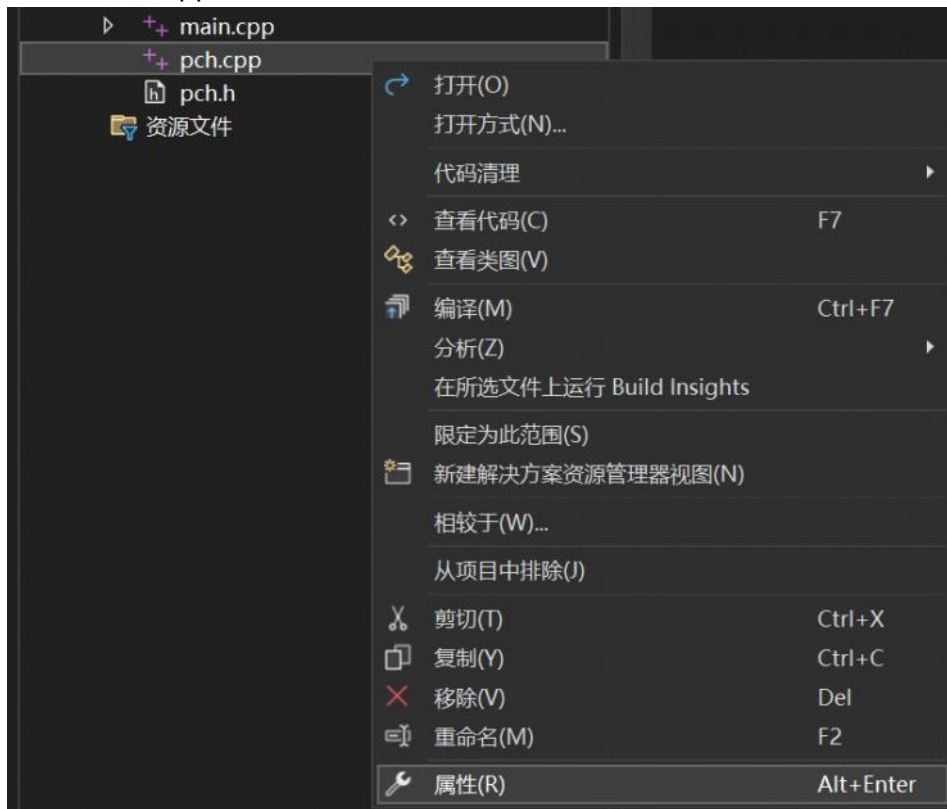
```

打开这个文件，会发现包含这么多头文件之后的hello world，有42万8千行。除去我们写的3行，不使用pch的情况下，额外42万8千多行的代码每次都要重新编译。所以我们才需要使用pch。

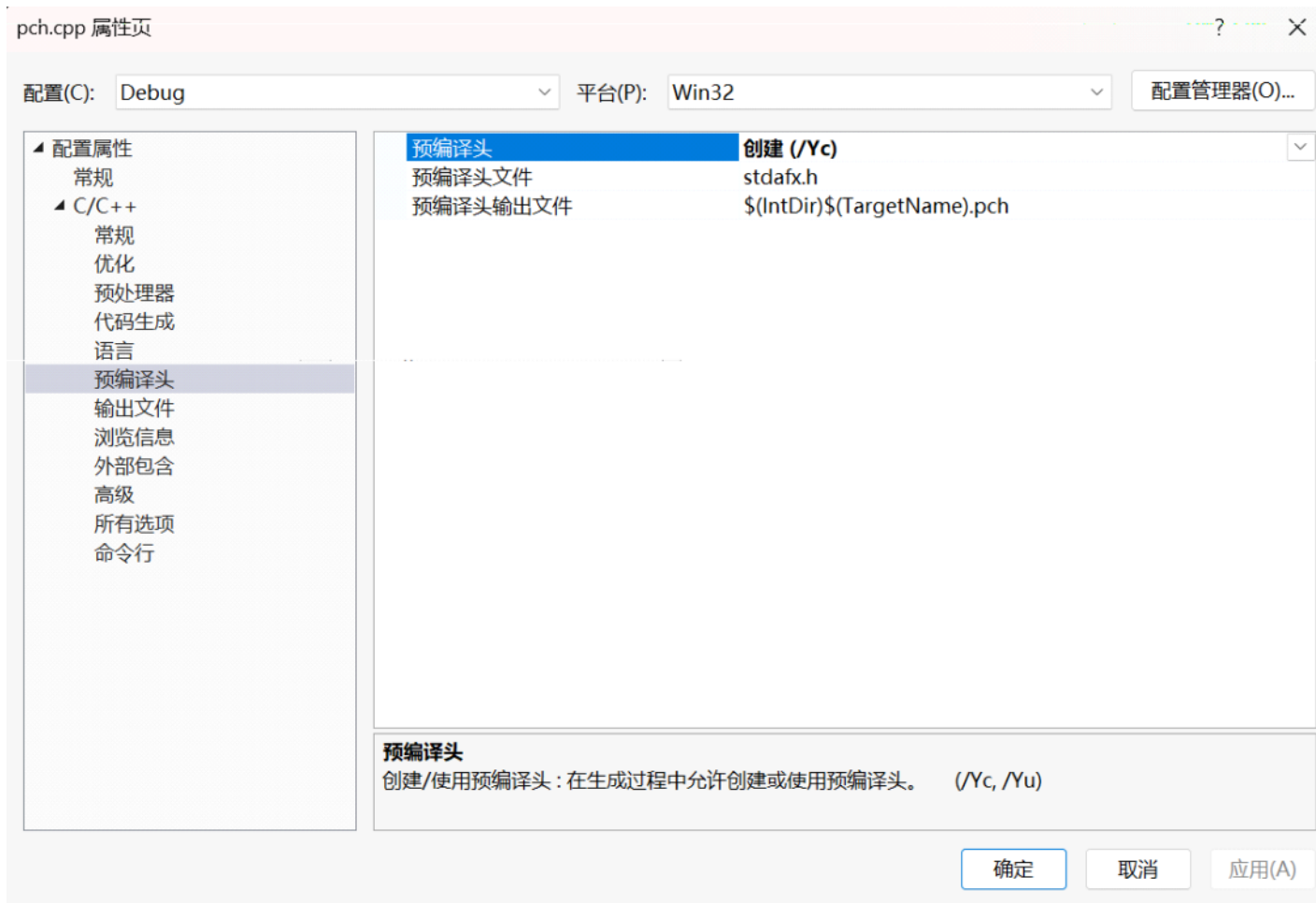
现在我们把pch变成真正的预编译头文件：

首先，只有一个头文件是不够的，至少在vs里是这样，我们还需要新建一个cpp文件，内容只有一行，就是include这个头文件。

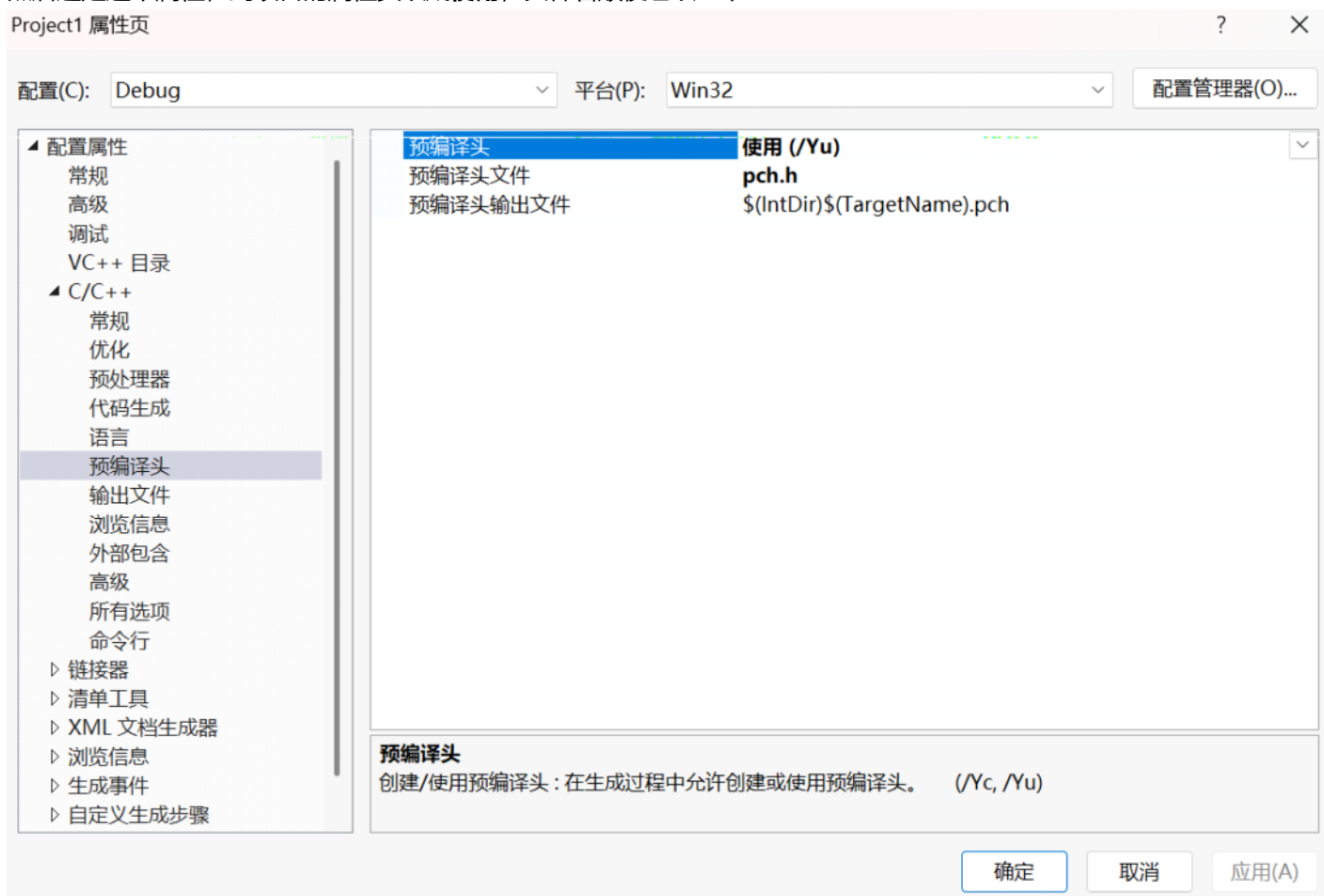
然后打开这个cpp文件的属性（注意不是这个项目的属性）：



在预编译头属性栏里把同名属性改成**创建**：



然后还是这个属性，到项目的属性页改成**使用**，文件名顺便也改一下：



```
生成开始于 21:12...
1>----- 已启动生成: 项目: Project1, 配置: Debug Win32 -----
1>main.cpp
===== 生成: 1 成功, 0 失败, 0 最新, 0 已跳过 =====
===== 生成 于 21:12 完成, 耗时 03.949 秒 =====
```

```
生成开始于 21:26...
1>----- 已启动生成: 项目: Project1, 配置: Debug Win32 -----
1>pch.cpp
1>main.cpp
===== 生成: 1 成功, 0 失败, 0 最新, 0 已跳过 =====
===== 生成 于 21:26 完成, 耗时 02.038 秒 =====
```

使用pch后, 直接构建的速度就快了将近一倍。此时我们随便修改一下代码再重新构建:

```
生成开始于 21:35...
1>----- 已启动生成: 项目: Project1, 配置: Debug Win32 -----
1>main.cpp
===== 生成: 1 成功, 0 失败, 0 最新, 0 已跳过 =====
===== 生成 于 21:35 完成, 耗时 00.586 秒 =====
```

可以看到只花了半秒钟, 也就是说头文件没有再被重新编译了。

这还只是一个简单的演示项目, 只有一个cpp文件。对于大型项目的效率提升有多大, 可想而知。

52 dynamic cast

2025年11月10日 9:56

我们在类型转换的那节了解过C++风格的类型转换，我们有各种的cast。这节我们主要学习dynamic cast。

相比于C风格的类型转换，dynamic_cast更像是一个函数，它不是编译时进行的类型转换，而是在运行时计算。

dynamic_cast是专门用于沿继承层次结构进行的强制类型转换，具体来说就是基类和派生类之间的类型转换。

通常把派生类转换成基类都比较容易，我们可以直接用隐式转换；但是如果要把基类转换成派生类就有点复杂了，我们不知道它是被声明成基类的派生类实例，还是真的只是基类的实例。这就涉及到安全性的问题，所以我们通常借助dynamic_cast来帮我们进行验证。

dynamic_cast在转换失败的时候会返回NULL指针，也就是0，我们可以用它来检查一个对象是否为给定类型。

```
#include <iostream>
#include <string>

class Entity {
};

class Player :public Entity {
};

class Enemy :public Entity {
};

int main() {
    Entity* e1 = new Player();
    Entity* e2 = new Enemy();

    Player* p = dynamic_cast<Player*>(e1);
}
```

上述代码尝试使用dynamic_cast将Entity转换成Player，但是标红区域是会报错的：“**运行时dynamic_cast的操作数必须包含多态类类型**”。我们之前也提到过，要使用dynamic_cast，基类必须是**多态类型**，即**至少有一个虚函数**。这个虚函数是什么都无所谓，甚至可以是个空函数。

```
#include <iostream>
#include <string>

class Entity {
public:
    virtual void VFunction() {}
};

class Player :public Entity {
};
```

```

class Enemy :public Entity {

};

int main() {
    Entity* e_player = new Player();
    Entity* e_enemy = new Enemy();

    Player* p1 = dynamic_cast<Player*>(e_player);
    Player* p2 = dynamic_cast<Player*>(e_enemy);
}

```

名称	值	类型
▸  p1	0x01505e70 {...}	Player *
▸  p2	0x00000000 <NULL>	Player *
添加要监视的项		

可以看到尝试将本质为enemy的entity转化成player就会失败，返回NULL指针。

现在的问题是，它是怎么知道实例本质上是什么类型的？C++本身并不是托管语言。能做到这一点，是因为它储存了**运行时类型信息 (runtime type information/RTTI)**。如果我们在VS中将RTTI选项关闭，此时再运行程序，dynamic_cast函数就会报错，因为它没有得到类型信息。

既然RTTI可以关闭，那就说明它是存在性能开销的，以及dynamic_cast本身的验证功能也是如此。但是如果不需要编写快到极致的代码，RTTI一般是需要打开的，动态强制转换也是建议使用的。

53 基准测试

2025年11月10日 10:48

我们通常需要测试代码的性能，一般是通过计时的方法来实现。

要进行基准测试，首先是准备好需要测试的代码：

```
#include <iostream>
#include <memory>

int main() {
    int value = 0;

    for (int i = 0; i < 1000000; i++) {
        value += 1;
    }

    std::cout << value << std::endl;

    __debugbreak();
}
```

__debugbreak()是VS专门用于Windows的函数，相当于一个断点。

```
#include <iostream>
#include <memory>
#include <chrono>

class Timer {
public:
    Timer() {
        m_StartTimepoint = std::chrono::high_resolution_clock::now();
    }

    ~Timer()
    {
        Stop();
    }

    void Stop() {
        auto endTimepoint = std::chrono::high_resolution_clock::now();

        auto start =
            std::chrono::time_point_cast<std::chrono::microseconds>
            (m_StartTimepoint).time_since_epoch().count();
        auto end =
            std::chrono::time_point_cast<std::chrono::microseconds>
            (endTimepoint).time_since_epoch().count();

        auto duration = end - start;
        double ms = duration * 0.001;
    }
};
```

```

        std::cout << "Timer took " << duration << "us (" << ms
        << "ms)\n";
    }
private:
    std::chrono::time_point<std::chrono::steady_clock> m_StartTimepoint;
};

int main() {
    int value = 0;
    {
        Timer timer;
        for (int i = 0; i < 1000000; i++) {
            value += 1;
        }

        std::cout << value << std::endl;

        __debugbreak();
    }
}

```

我们写了一个计时器类，基本上和学习计时那节写的内容一致。然后我们人为创造一个作用域，也就是一对花括号，来应用计时器。

不过这种计时方式尤其需要注意的一点是：**必须确保你真的测量了你想测量的代码**。这么说可能有点奇怪，我们在最开始学习的时候就了解过，编译器会自己对代码进行优化，比如整数折叠之类的行为。就上述代码来说，我们想要测量的是1000000次整数增量的耗时，但是在运行而非调试情况下，测量实际上是无效的。

```

Timer took 1547us (1.547ms)
1000000

```

```

Timer took 684us (0.684ms)
1000000

```

可以看到调试模式下运行的时间是运行模式的两倍多。这是因为编译器在运行模式时会优化掉不必要的代码，我们进入反汇编查看汇编码就会发现运行模式下编译器只是直接打印了1000000这个数字，没有任何的整数增量计算过程。我们实际上只是记录了打印操作的耗时。

还有一点，一般都是在release版本进行基准测试，因为我们希望知道的是实际发布版本的性能情况，debug版本会有许多为了方便调试但对于业务逻辑没有帮助的冗余功能，导致影响计时。

54 结构化绑定

2025年11月10日 11:25

我们在之前学习过在C++中处理多返回值，而结构化绑定就是专门为了优化多返回值处理而在C++17中新增的功能。

```
#include <iostream>
#include <string>
#include <tuple>

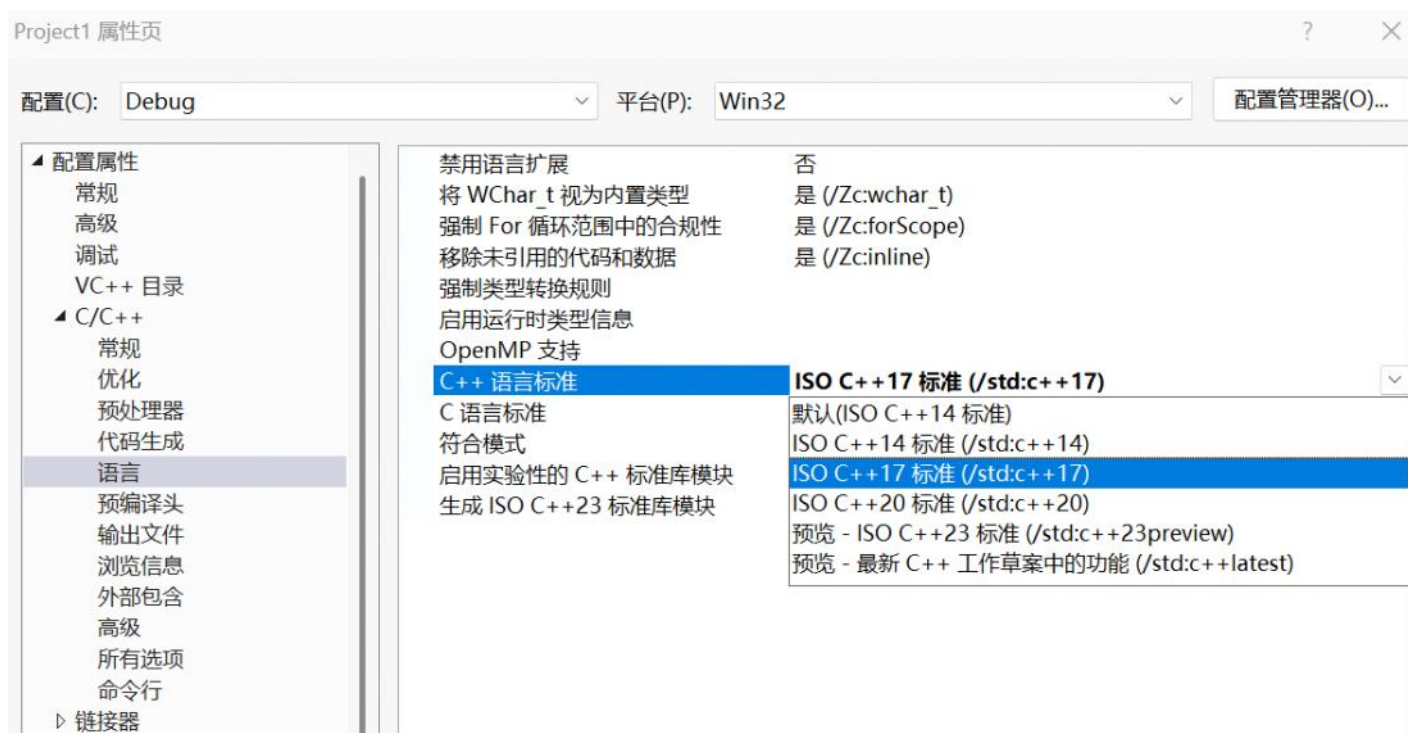
std::tuple<std::string, int> CreatePerson() {
    return{ "LJJ", 24 };
}

int main() {
    //使用std::get<>()
    auto person = CreatePerson();
    std::string& name = std::get<0>(person);
    int age = std::get<1>(person);

    //使用std::tie()
    std::string name;
    int age;
    std::tie(name, age) = CreatePerson();
}
```

我们之前讨论过，结论是最好使用结构体，原因就是tuple访问元素的代码可读性差。如上述代码，使用get的话，只能按照索引访问元素，可读性最差；使用tie的情况稍好，但每个变量都需要新建，复杂的情况下代码行数会很多。所以我们可以使用结构化绑定，一行代码解决。

不过使用结构化绑定之前需要检查C++版本是否在17以上，VS的默认版本是C++14，是不能直接使用结构化绑定的，需要调整版本。





```
#include <iostream>
#include <string>
#include <tuple>

std::tuple<std::string, int> CreatePerson() {
    return{ "LJJ", 24 };
}

int main() {
    auto [name, age] = CreatePerson();
    std::cout << name << " " << age << std::endl;
}
```

但是此时name和age两个变量实际上还是孤立的，没有一个东西把它们统筹在一起。我个人还是觉得用结构体来管理多返回值更清楚。

但是如果整个工程中某个结构体只使用过一次或两次，而且再也不会再出现，那么就没有必要为它新建一个类型，那样只会让代码变得混乱。这种情况下使用结构化绑定是再好不过的了。

55 std::optional

2025年11月10日 12:44

当数据可能存在，也可能不存在的时候，该怎么办？比如我们有一个返回数据的函数，我们正在用它读取一个文件。如果文件是不可读写的会怎么样？如果文件是空的会怎么样？如果文件类型不是我们所期望的会怎么样？这些意外情况我们仍需要函数有一个返回值，而不是直接 error。

这就是需要用到std::optional的地方了，它同样是C++17的新东西。在此之前我们也有别的方式处理此类问题，不过现在有了更好的方法了。

```
#include <iostream>
#include <string>
#include <fstream>

std::string ReadFileAsString(const std::string& filepath, bool&
outSuccess) {
    std::ifstream stream(filepath);

    if (stream) {
        std::string result;
        //read file...
        stream.close();
        outSuccess = true;
        return result;
    }

    outSuccess = false;
    return std::string();
}

int main() {
    bool fileOpenedSuccessfully;
    std::string data = ReadFileAsString("data.txt",
fileOpenedSuccessfully);
    if (fileOpenedSuccessfully) {

    }
}
```

这是不使用std::optional的写法，做了简单的异常处理，比异常直接返回空字符串要好一些，但还是不够好。我们需要两个变量，一个字符串用来存储信息，一个布尔值为了标记是否正常打开文件（区别打开失败和文件为空）。这实际上有些麻烦且占用内存。

```
#include <iostream>
#include <string>
#include <fstream>
#include <optional>
```

```

std::optional<std::string> ReadFileAsString(const std::string& filepath) {
    std::ifstream stream(filepath);

    if (stream) {
        std::string result;
        //read file...
        stream.close();
        return result;
    }

    return {};
}

int main() {
    std::optional<std::string> data = ReadFileAsString("data.txt");

    std::string value = data.value_or("Not present"); //设置默认值

    std::cout << value << std::endl;

    if (data) {
        std::cout << "File read successfully!\n";
    }
    else {
        std::cout << "File could not be opened!\n";
    }

    std::cin.get();
}

```

这段代码使用std::optional很简洁地实现了需要的功能。函数中返回空的花括号表示返回一个默认构造的std::optional对象，即一个不包含值的optional（等同于std::nullopt）。

假如data不为空optional，我们可以使用data.value()或*data来使用接收的字符串数据，data->后面也可以直接用std::string类的方法。

56 std::variant

2025年11月10日 22:17

C++17中新的标准库提供了std::variant，方便我们在一个变量中存储多种类型的数据。它和std::optional很像，它的作用是让我们不用担心处理的确切数据类型。

```
#include <iostream>
#include <variant>

int main() {
    std::variant<std::string, int> data;

    data = "LJJ";
    std::cout << std::get<std::string>(data) << "\n";

    data = 2;
    std::cout << std::get<int>(data) << "\n";

    std::cin.get();
}
```

上述代码简单使用了std::variant，我们指定data只能是字符串或整数这两种类型之一，具体是哪种并不确定。此时data无法被赋值成除这两种之外的其他类型，但是布尔值可以，那是因为布尔值会被编译器识别成0和1，本质上还是整数。

如果我们尝试获取错误的类型，比如data实际上是字符串，但是我们尝试将它获取为整数，那么编译器就会抛出异常。我们可以使用try语句来进行验证，但是还有另外两种方法，不需要使用异常：

```
#include <iostream>
#include <variant>

int main() {
    std::variant<std::string, int> data;

    data = "LJJ";
    std::cout << std::get<std::string>(data) << "\n";
    std::cout << data.index() << "\n";

    data = 2;
    std::cout << std::get<int>(data) << "\n";
    std::cout << data.index() << "\n";

    std::cin.get();
}
```

std::variant类型有一个索引属性，按照声明时尖括号里的类型排列顺序。比如这里当data为字符串的时候index就是0，为整数的时候index就是1。

另一种方法:

```
#include <iostream>
#include <variant>

int main() {
    std::variant<std::string, int> data;

    data = "LJJ";
    std::cout << std::get<std::string>(data) << "\n";
    if (auto value = std::get_if<std::string>(&data)) {

    }

    data = 2;
    std::cout << std::get<int>(data) << "\n";
    if (auto value = std::get_if<int>(&data)) {

    }

    std::cin.get();
}
```

get_if会尝试获取尖括号内类型的数据并返回指向它的指针，如果类型不符，就会返回空指针。这种写法相对于通过索引获取可读性更强。

很多人认为std::variant和union很像，实际上它们还是有一定区别的：

首先是内存方面，union所占的内存大小是它包含的所有类型中最大的那个的内存大小，而std::variant是它包含的所有类型的内存大小的和。本质上std::variant是创建了一个结构体，为每个可能的类型都预留好了空间，而union就是共用内存而已。

其次用途上union是更高效的，但是std::variant是类型安全的，不会出现未定义的行为。就大多数情况而言，不过度追求性能的话，都应该使用std::variant。

现在我们可以用std::variant优化上节写的读取文件的代码：

```
enum class ErrorCode {
    None = 0, NotFound = 1, NoAccess = 2
};

std::variant<std::string, ErrorCode> ReadFileAsString() {
    //...
    return{};
}
```

现在我们不局限于判断文件能否打开，可以进行更详细的状态分析。

57 std::any

2025年11月10日 22:53

这节我们学习如何在单个变量中存储任意类型的数据，这可以用void指针来做，但是C++17有更安全的处理方式std::any。不过最大的问题不在于我们如何使用它，而是我们在什么场景下需要使用它。

```
#include <iostream>
#include <any>

int main() {
    std::any data;
    data = 2;
    data = "LJJ";
    data = std::string("LJJ");

    std::string str = std::any_cast<std::string>(data);

    std::cin.get();
}
```

std::any很像范围无限大的std::variant，它允许变量可能是任何类型而不需要一个一个列出来。这似乎让它看起来更好用，实则不然。正因为它可以是任何类型，它就无法保证类型安全，我们最终需要确定类型、拿取数据的时候就要花上更多功夫。

更深入一些，通过观察std::any的源码，发现它在小类型的处理方式上和std::variant是一致的，但是对大类型会使用动态内存分配（堆分配），我们知道动态内存分配是很消耗性能的。所以对于小类型的变量，使用两种方式本质上是一样的，对于大类型的变量，std::variant会表现得更好。一般来说，大小类型的分界线是32字节内存。

实际上，对于教程作者来说，他也不认为std::any会有什么特别突出的适合场景。std::variant基本上就是std::any的内存安全版本，而且性能更优。std::any本身允许使用所有类型的功能也很鸡肋，如果程序需要在单个变量中存储任何数据类型，那么往往程序的设计是有问题的。

所以.....尽可能避免使用它吧，学习仅作了解作用。

58 让字符串更快

2025年11月11日 22:57

C++标准库的string实际上是比较慢的，因为它做了很多堆分配的操作。我们之前讨论过这个问题，在可以避免的情况下就应该避免堆分配，因为它底层是个很复杂的过程，会拖慢我们的程序。

```
#include <iostream>
#include <string>

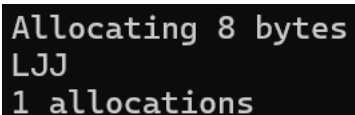
static uint32_t s_AllocCount = 0;

void* operator new(size_t size) {
    s_AllocCount++;
    std::cout << "Allocating " << size << " bytes\n";
    return malloc(size);
}

void PrintName(const std::string& name) {
    std::cout << name << std::endl;
}

int main() {
    std::string name = "LJJ";
    PrintName(name);

    std::cout << s_AllocCount << " allocations\n";
    std::cin.get();
}
```



```
Allocating 8 bytes
LJJ
1 allocations
```

这段代码中我们重写了new操作符以便可以观察堆分配的详细情况。可以看到我们从始至终没有使用过new操作符，但是堆分配还是发生了。

也许你会尝试这么写：PrintName("LJJ");

但是很不幸，直接传递一个临时的字符串不会有任何改善，它同样会产生一次堆分配。

```
int main() {
    std::string name = "LJJ";

    std::string firstName = name.substr(0, 1);
    std::string lastName = name.substr(1, 2);

    PrintName(firstName);

    std::cout << s_AllocCount << " allocations\n";
}
```

```

        std::cin.get();
    }

```

主函数改成如上，会发现现在有三堆分配了：

```

Allocating 8 bytes
Allocating 8 bytes
Allocating 8 bytes
L
3 allocations

```

对于如此简单的程序来说这点开销没什么，但是当你的程序每一帧都要进行成百上千次类似操作的时候，它对于性能的影响是非常严重的。

任何优化首先都是从数据入手。这里我们需要获取人的姓名、姓和名，本质上就是字符串和它的子字符串。那么我有必要新建字符串来存储子字符串吗？当然不是。我们可以想象一个小的视口，它的大小只能容纳子字符串的长度，我们用它去观察完整的字符串，然后把看到的東西输出，就像管中窥豹一样。

```

void PrintName(std::string_view name) {
    std::cout << name << std::endl;
}

int main() {
    std::string name = "LJJ";

    std::string_view firstName(name.c_str(), 1);
    std::string_view lastName(name.c_str() + 1, 2);

    PrintName(firstName);
    PrintName(lastName);

    std::cout << s_AllocCount << " allocations\n";
    std::cin.get();
}

```

我们使用了C++17中的新特性std::string_view，其实在C++17之前人们就开始这么做了，只不过现在官方做出了专门的工具。它本质上就是通过原始字符串的指针和子字符串的大小来截取内容，和学习原始数组时说过的操作基本一致。

```

Allocating 8 bytes
L
JJ
1 allocations

```

现在我们只有新建原始字符串时的那一次堆分配了。不过还可以更好，我们完全不使用std::string，就可以实现0次堆分配：

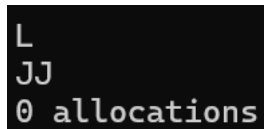
```

int main() {
    const char* name = "LJJ";
}

```



```
std::string_view firstName(name, 1);  
std::string_view lastName(name + 1, 2);  
  
PrintName(firstName);  
PrintName(lastName);  
  
std::cout << s_AllocCount << " allocations\n";  
std::cin.get();  
}
```



```
L  
JJ  
0 allocations
```

可以看到new操作符完全没有被调用。不过我们通常优化到上一步的代码就可以了，因为我们一般处理的字符串来源于外部，大多是情况下也都会是std::string类型。

59 单例模式

2025年11月12日 19:04

当我们想要拥有应用于全局数据集的功能，而且只是想要重复使用一个对象的时候，单例是非常有用的。

比如随机数生成器，我们只希望访问它，向它索要一个随机数，仅此而已，所以我们希望只把它实例化一次就够了。或者渲染器，它通常是一个非常全局的东西，我们通常不会有一个渲染器的多个实例。

单例是一个类的单一实例，也就是一个类只实例化一个对象。这样看似和类的用途背道而驰，我们费劲写出的类就是为了应对可能多次使用它的对象的情况。但现在我们只需要实例化一次，那么我们是否还有必要让它成为一个类？

其实从根本上来说，在C++中，单例类就像一个名称空间，它只是一种组织一堆全局变量和静态函数的方式。它确实有点“里外不是人”了，我们完全可以使用C风格代码实现单例的功能，就是只使用一堆全局变量和静态函数。

不过在一些强类型语言中，单例类的存在是没有歧义的，比如Java。因为Java所有的代码必须在某个类中，它不允许有脱离类孤立存在的东西。所以就算只实例化一次的东西，也需要成为一个类，也许单例类就是从这里起源的。

话虽如此，在C++中使用单例模式仍需要遵守一些规则。

```
#include <iostream>

class Singleton {
public:
    Singleton(const Singleton&) = delete;

    static Singleton& Get() {
        return s_Instance;
    }

    void Function(){}

private:
    Singleton(){}

    static Singleton s_Instance;
};

Singleton Singleton::s_Instance;

int main() {
    Singleton& instance = Singleton::Get();
```

```

        instance.Function();

        std::cin.get();
    }
}

```

这段代码实现了最简单的单例，可以认为它是经典单例模式的模板，它包含了以下要点：

1. 禁止拷贝构造：通过 `Singleton(const Singleton&) = delete;` 显式删除了拷贝构造函数，防止通过拷贝创建多个实例，确保单例的唯一性。
2. 静态成员函数获取实例：提供了一个公共的静态成员函数 `Get()`，用于获取单例实例。该函数返回一个对静态实例的引用。
3. 私有化构造函数：将构造函数 `Singleton()` 设置为私有，防止外部直接创建实例，强制通过 `Get()` 函数获取实例。
4. 静态成员变量：在类内声明了一个静态成员变量 `s_Instance`，并在类外进行了定义（`Singleton Singleton::s_Instance;`）。这意味着在程序启动时（在 `main` 函数之前）就已经创建了实例，属于饿汉式单例。
5. 线程安全：由于静态成员变量在程序开始时就初始化，因此该单例实现是线程安全的，因为在线程开始之前实例已经创建完毕。

最后我们把单例模板代码修改成一个实际应用场景——随机数生成器：

```

#include <iostream>

class Random {
public:
    Random(const Random&) = delete;

    static Random& Get() {
        return s_Instance;
    }

    float Float(){
        return m_RandomGenerator;
    }

private:
    Random(){}

    float m_RandomGenerator = 0.5f;
    static Random s_Instance;
};

Random Random::s_Instance;

int main() {
    float random_number = Random::Get().Float();

    std::cin.get();
}

```

不过我们没有真的写生成随机数的逻辑，它只是简单的返回0.5f。只是提供一个供参考的例子，来帮助更好地理解单例是怎么运作和使用的。

最后补充一下，懒汉式单例的写法：

```

#include <iostream>

class Singleton {
public:
    Singleton(const Singleton&) = delete;
    static Singleton& Get() {
        static Singleton s_Instance; // 第一次调用时创建
        return s_Instance;
    }
    void Function(){}

private:
    Singleton(){}
// 移除静态成员变量定义
    // static Singleton s_Instance;
};
// 删除这行: Singleton Singleton::s_Instance;

```

相比于饿汉式单例的写法，改动并不大，主要是标红的语句。它意味着这个单例只在它第一次被调用时创建，也就是第一次调用静态函数Get()时。它不需要在类外部定义静态变量s_Instance，所以看起来更干净。由于s_Instance是静态变量，所以也不用担心多次调用Get()会发生不必要的复制。

60 小字符串优化 (SSO)

2025年11月12日 19:50

字符串在编程语言中一直都是比较有争议的话题，因为它太慢了。所以，字符串优化的讨论热度很高，甚至会有人鄙视使用字符串的程序员。

不过好在C++标准库也注意到这个问题，它们的解决方式跟之前学过的std::any非常像。之前说过，std::any在小类型的处理上和std::variant一致，都是栈分配，但是对大类型使用较慢的堆分配。

字符串也是这样，小于某个长度的字符串会被认为是“小字符串”，也会在栈上分配内存，这个阈值因平台而异（vs是15个字符长度），超过阈值的字符串照常使用堆分配。

所以说当你有一个非常小的字符串时，就不再需要使用const char*了。但是在之前“让字符串更快”这一节中，我们的代码明明使用的是小字符串，却还是发生了堆分配：

```
#include <iostream>
#include <string>

static uint32_t s_AllocCount = 0;

void* operator new(size_t size) {
    s_AllocCount++;
    std::cout << "Allocating " << size << " bytes\n";
    return malloc(size);
}

void PrintName(const std::string& name) {
    std::cout << name << std::endl;
}

int main() {
    std::string name = "LJJ";
    PrintName(name);

    std::cout << s_AllocCount << " allocations\n";
    std::cin.get();
}
```

```
Allocating 8 bytes
LJJ
1 allocations
```

这是因为我当时使用的是Debug模式，如果改成Release模式，小字符串优化就会生效。

```
LJJ
0 allocations
```

61 跟踪内存分配的简单方法

2025年11月12日 20:24

在编写C++代码中，明确的知道自己的代码在什么地方分配了内存、分配了多少内存，尤其是堆分配、在性能关键代码中时，是极其重要的。它不仅让我们优化程序有迹可循，也能加深我们对程序运行的理解程度。

```
#include <iostream>
#include <memory>

struct AllocationMetrics {
    uint32_t TotalAllocated = 0;
    uint32_t TotalFreed = 0;

    uint32_t CurrentUsage() { return TotalAllocated - TotalFreed; }
};

static AllocationMetrics s_AllocationMetrics;

void* operator new(size_t size) {
    s_AllocationMetrics.TotalAllocated += size;

    return malloc(size);
}

void operator delete(void* memory, size_t size) {
    s_AllocationMetrics.TotalFreed += size;

    free(memory);
}

struct Object {
    int x, y, z;
};

static void PrintMemoryUsage() {
    std::cout << "Memory Usage: " << s_AllocationMetrics.CurrentUsage()
    << " bytes\n";
}

int main() {
    PrintMemoryUsage();
    std::string str = "114514";
    PrintMemoryUsage();
    {
        std::unique_ptr<Object> obj = std::make_unique<Object>();
        PrintMemoryUsage();
    }
    PrintMemoryUsage();
}
```

这段代码实现了一个简单的内存追踪器，关键点就是重写new和delete操作符，再额外写一个单例类来管理获取的数据。这段代码可以插入到任何应用程序中来帮助优化性能。

62 左值和右值

2025年11月13日 14:49

我们后续将会学习移动语义及其相关的一系列知识，其中左值和右值是我们首先要了解的。

```
int i = 10;
```

简单来看，绝大多数情况下，左值就是等号左边的值，右值就是等号右面的值。但是这并不是总是适用的，左值也可能出现在等号右边，比如我把左值赋值给另一个左值：

```
int a = i;
```

此时等号左右两边都是左值。也有人说字面量就是右值，这也不完全对，函数返回的值通常也是右值：

```
#include <iostream>

int GetValue() {
    return 10;
}

int main() {
    int i = GetValue();
}
```

不过既然说是“通常”，那就说明函数也能返回左值：

```
#include <iostream>

int& GetValue() {
    static int value = 10;
    return value;
}

int main() {
    GetValue() = 5;
}
```

一般函数接受的参数可以是左值或右值。但本质上都是左值，因为传入的右值会被用来创建一个左值。但是当函数接收左值引用的时候，就无法使用右值了，因为右值不能赋值给左值引用。

```
#include <iostream>

void SetValue1(int value) {

}
```

```

void SetValue2(int& value) {

}

int main() {
    int i = 10;
    SetValue1(i);
    SetValue1(10);

    SetValue2(i);
    SetValue2(10);
}

```

这里涂红的代码会报错，就是因为尝试把右值赋值给左值引用，但是这也并非绝对不可以：

```

int& a = 10;
const int& a = 10;

```

在前面加个const就可以了。常量左值引用允许右值赋值，但本质上还是左值赋值，内部的完整过程是这样的：

```

int temp = 10;
const int& a = temp;

```

我们再举一个字符串的例子：

```

#include <iostream>

void PrintName(std::string& name) {
    std::cout << name << std::endl;
}

int main() {
    std::string firstName = "L";
    std::string lastName = "JJ";

    std::string fullName = firstName + lastName;

    PrintName(fullName);
    PrintName(firstName + lastName);
}

```

std::string是支持字符串相加的，但是它返回的其实是一个右值，所以上述涂红的代码会报错。这也就是为什么我们通常函数接收字符串的时候都喜欢用常量引用，因为它兼容临时的右值和实际存在的左值：

```

#include <iostream>

void PrintName(const std::string& name) {
    std::cout << name << std::endl;
}

```



```
int main() {
    std::string firstName = "L";
    std::string lastName = "JJ";

    std::string fullName = firstName + lastName;

    PrintName(fullName);
    PrintName(firstName + lastName);
}
```

这样写就没有任何问题了。不过我们还有右值引用，就是连用两个“&”符号，这时函数就只会接收临时的右值，且它接收右值的优先级高于常量左值引用：

```
#include <iostream>

void PrintName(const std::string& name) {
    std::cout << "[lvalue]" << name << std::endl;
}

void PrintName(std::string&& name) {
    std::cout << "[rvalue]" << name << std::endl;
}

int main() {
    std::string firstName = "L";
    std::string lastName = "JJ";

    std::string fullName = firstName + lastName;

    PrintName(fullName);
    PrintName(firstName + lastName);
}
```

```
[lvalue]LJJ
[rvalue]LJJ
```

说了这么多，感觉绕来绕去的。简单来说：

- **左值是有某种存储支持的变量，右值是临时值；**
- **左值引用仅仅接收左值，除非使用const；**
- **右值引用仅仅接受右值，不会与const搭配使用；**

补充一点，右值引用对于优化是有帮助的。因为它明确告诉我们正在使用一个临时值，我们可以随意从临时值中偷取资源而不用担心安全性。

63 参数计算顺序

2025年11月13日 15:44

```
#include <iostream>

void PrintSum(int a, int b) {
    std::cout << a << " + " << b << " = " << (a + b) << std::endl;
}

int main() {
    int value = 0;
    PrintSum(value++, value++);

    std::cin.get();
}
```

这是一段非常简单的代码，但是传参的写法跟我们平时的习惯不太一致。因为我们需要引出一个问题：最后打印出的结果应该是什么样子的？换句话说，当我们给函数的多个参数传入的是多个计算式的时候，它们的计算顺序是什么样子的？

事实上，这个问题并没有答案，因为这种写法是一种“**未定义行为**”。C++标准并没有规定这种情况下应该怎么处理，它完全按照编译器的执行顺序来处理，也就是说不同的编译器在不同的设置下结果都是不同的。

在VS2022中，我们在Debug模式下会得到：“1 + 0 = 1”，在Release模式下会得到：“0 + 0 = 0”。

不过在C++17中新增定义：**后缀表达式必须在别的表达式之前被计算**。对于这些参数，它们必须一个接一个地被计算，但是谁先谁后仍然是不确定的。

64 移动语义

2025年11月13日 16:06

我们之前学习过右值和右值引用，它们最大的一个用处就是移动语义。移动语义本质上允许我们移动对象，它算是C++11后的新特性。因为右值引用是在C++11中引入的，而它基本是移动语义所必需的。

简单来说，移动语义就像是“剪切”。我们都知道复制是比较消耗性能的行为，尤其是涉及到堆分配的时候。当我们别无选择，直接剪切，将变量本身移动到我们想要的位置，可以帮我们最大限度节约性能。

```
#include <iostream>

class String {
public:
    String() = default;
    String(const char* string) {
        printf("Created!\n");
        m_Size = strlen(string);
        m_Data = new char[m_Size];
        memcpy(m_Data, string, m_Size);
    }

    String(const String& other) {
        printf("Copied!\n");
        m_Size = other.m_Size;
        m_Data = new char[m_Size];
        memcpy(m_Data, other.m_Data, m_Size);
    }

    ~String()
    {
        printf("Destroyed!\n");
        delete m_Data;
    }

    void Print() {
        for (uint32_t i = 0; i < m_Size; i++) {
            printf("%c", m_Data[i]);
        }
        printf("\n");
    }
private:
    char* m_Data;
    uint32_t m_Size;
};

class Entity {
public:
    Entity(const String& name)
        :m_Name(name){
```

```

    }

    void PrintName() {
        m_Name.Print();
    }
private:
    String m_Name;
};

int main() {
    Entity entity(String("LJJ"));
    entity.PrintName();

    std::cin.get();
}

```

这里有一段相对较长的代码，但实际上做的事情很简单。我们实现了一个简单的字符串类和一个实体类。实体类有名字，当我们实例化一个实体对象时临时创建了一个字符串传递给实体类的构造函数，但是还是发生了复制：

```

Created!
Copied!
Destroyed!
LJJ

```

这意味着一次额外的堆分配，虽然并不是什么大问题，但是我们仍然想要解决它。我们想要在main函数里分配它，然后直接把它移到实体类的构造函数里，不出现任何复制操作。这就是移动语义的用武之地。

```

#include <iostream>

class String {
public:
    String() = default;
    String(const char* string) {
        printf("Created!\n");
        m_Size = strlen(string);
        m_Data = new char[m_Size];
        memcpy(m_Data, string, m_Size);
    }

    String(const String& other) {
        printf("Copied!\n");
        m_Size = other.m_Size;
        m_Data = new char[m_Size];
        memcpy(m_Data, other.m_Data, m_Size);
    }

    String(String&& other) noexcept{
        printf("Moved!\n");
        m_Size = other.m_Size;
        m_Data = other.m_Data;

        other.m_Size = 0;
        other.m_Data = nullptr;
    }
}

```

```

    }

    ~String()
    {
        printf("Destroyed!\n");
        delete m_Data;
    }

    void Print() {
        for (uint32_t i = 0; i < m_Size; i++) {
            printf("%c", m_Data[i]);
        }
        printf("\n");
    }
private:
    char* m_Data;
    uint32_t m_Size;
};

class Entity {
public:
    Entity(const String& name)
        :m_Name(name){

    }

    Entity(String&& name)
        :m_Name((String&&)name) {

    }

    void PrintName() {
        m_Name.Print();
    }
private:
    String m_Name;
};

int main() {
    Entity entity(String("LJJ"));
    entity.PrintName();

    std::cin.get();
}

```

```

Created!
Moved!
Destroyed!
LJJ

```

对最初的代码进行一些修改后，我们最终消除了复制操作。

关键点在于：我们在字符串类中使用移动构造函数代替复制构造函数。移动构造函数和复制构造函数类似，都需要先传入一个已有的字符串，不过它使用的是右值引用。

然后我们直接“偷取”这个右值字符串的资源，我们有一个新的指针指向它的内存，然后再把它本身的指针赋值成nullptr。这样一来，当原字符串的生命周期结束时，它的析构函数会delete一个空指针（等于什么都没做）。

移动语义可以理解为偷天换日、狸猫换太子等类似操作。本质上就是将原有数据的资源转移到另一个地方使用。

初学者可能难以区分它和引用传递的用法区别，不都是避免复制操作吗？

引用和指针确实避免了复制，但它们的“避免复制”和移动语义的“避免复制”是在不同层面上解决的问题：

- 引用和指针：传递的是资源的“地址”，不复制资源本身。但资源的所有权没有改变，原始变量仍然拥有并可能修改这个资源。
- 移动语义：转移的是资源的“所有权”。将资源从一个对象“移动”到另一个对象，原对象不再拥有该资源。

总的来说，移动语义让"临时对象"和"不再需要的对象"能够高效地转移资源，在必须创建新对象的场景中（如函数返回、容器插入）提供接近引用的性能。

65 std::move和移动赋值操作符

2025年11月13日 17:24

我们上节课所做的是把一个对象移动赋值给新建的对象，我们这节课主要学习如何把一个对象移动赋值给另一个已经存在的对象。

我们仍需要使用上节课写的字符串类，在这里省略，只写关键的main函数部分：

```
int main() {
    String string = "Hello";
    String destination((String&&)string);

    std::cin.get();
}
```

我想把string移动到destination上，要做到这一点，我们需要调用移动构造函数。那么怎么才能触发移动构造函数呢？很简单，使用右值引用，让它根据函数签名自动选择我们想要的构造函数。

上述代码使用显式类型转换来使用右值引用，但是这种写法的前提是知道我们操作的是什么类型的变量，如果上下文使用的都是auto，那就无从下手了。所以我们需要std::move：

```
int main() {
    String string = "Hello";
    String destination(std::move(string));

    std::cin.get();
}
```

这段代码做的事情是完全一样的，只不过std::move更方便，泛用性更强，可以理解为它是自动检测类型的右值引用类型转换。

不过到这里为止我们还是在把一个对象移动赋值给新建的对象，所以我们才会调用移动构造函数。如果我们想要把一个对象移动赋值给另一个已经存在的对象，就需要重写赋值操作符，也就是我们熟知的“=”。

移动赋值操作符的内容和移动构造函数基本一致，但是需要多一步“覆写”，因为我们是在操作一个已经存在的对象，它已经拥有了一些数据、分配了一些内存。通常我们还需要确保当前对象不等于我们要移动的对象，也就是不能把自己移动赋值给自己，这很愚蠢。

```
String& operator=(String&& other) noexcept {
    printf("Moved!\n");

    if (this != &other) {
```

```

        delete[] m_Data;

        m_Size = other.m_Size;
        m_Data = other.m_Data;

        other.m_Size = 0;
        other.m_Data = nullptr;
    }

    return *this;
}

```

附上完整代码：

```

#include <iostream>

class String {
public:
    String() = default;
    String(const char* string) {
        printf("Created!\n");
        m_Size = strlen(string);
        m_Data = new char[m_Size];
        memcpy(m_Data, string, m_Size);
    }

    String(const String& other) {
        printf("Copied!\n");
        m_Size = other.m_Size;
        m_Data = new char[m_Size];
        memcpy(m_Data, other.m_Data, m_Size);
    }

    String(String&& other) noexcept{
        printf("Moved!\n");
        m_Size = other.m_Size;
        m_Data = other.m_Data;

        other.m_Size = 0;
        other.m_Data = nullptr;
    }

    String& operator=(String&& other) noexcept {
        printf("Moved!\n");

        if (this != &other) {
            delete[] m_Data;

            m_Size = other.m_Size;
            m_Data = other.m_Data;

            other.m_Size = 0;
            other.m_Data = nullptr;
        }

        return *this;
    }
}

```



```

~String()
{
    printf("Destroyed!\n");
    delete m_Data;
}

void Print() {
    for (uint32_t i = 0; i < m_Size; i++) {
        printf("%c", m_Data[i]);
    }
    printf("\n");
}
private:
    char* m_Data;
    uint32_t m_Size;
};

int main() {
    String string = "Hello";
    String destination;

    std::cout << "String: ";
    string.Print();
    std::cout << "Destination: ";
    destination.Print();

    destination = std::move(string);

    std::cout << "String: ";
    string.Print();
    std::cout << "Destination: ";
    destination.Print();

    std::cin.get();
}

```

```

Created!
String: Hello
Destination:
Moved!
String:
Destination: Hello

Destroyed!
Destroyed!

```

可以看到，通过重写赋值操作符，我们轻松实现了两个字符串的内容交换，没有出现任何复制。当我们写了一个类，我们想让它的所有对象都使用移动代替复制的话，就可以重写赋值操作符为移动赋值操作符。

最后补充一下：

```
String destination = std::move(string);
```

```
String destination;  
destination = std::move(string);
```

红绿两种颜色标记的代码虽然都有 “=”，但它们本质上是完全不同的。

红色区域代码虽然出现了 “=”，但是它并没有调用赋值操作符，它调用的是构造函数，在这个例子中也就是移动构造函数。

绿色区域代码就是我们最后给出的代码中的一部分，它才真正意义上实现了两个已有变量之间的移动，使用了移动赋值操作符。